DIPLOMA OF ASSOCIATE ENGINEER

2ND YEAR

COMPUTER INFORMATION TECHNOLOGY

APPROVED BY TEVTA PUNJAB

A TEXT BOOK OF

DATABASE

CIT-263



Developed By

Academics Wing

Technical Education & Vocational Training Authority Punjab

Relational Data Base Management System

CIT-263

FOR DAE 2nd Year

TECHNICAL EDUCATION & VOCATIONAL TRAINING AUTHORITY PUNJAB

PREFACE

The text book has been written to cover the syllabus of database management system 2nd year D.A.E (CIT) according to the new scheme of studies. The book has been written in order to cater the needs of latest concepts and needs of the course i.e. Database management system and to be able to attempt D.A.E Examination of PBTE Lahore.

The aim of bringing out this book is to enable the students to have sound knowledge of the subject. Every aspect has been discussed to present the subject matter in the most concise, compact lucid & simple manner to help the subject without any difficulty. Frequent use of illustrative figures has been made for clarity. Short Questions and Self-tests have also been included at the end of each chapter which will serve as a quick learning tool for students.

The author would like to thank the reviewers whose valuable recommendations have made the book more readable and understandable. Constructive criticisms and suggestions for the improvements in future are welcome.

AUTHORS

MANUAL DEVELOPMENT COMMITTEE

IRAM MUNIR **(CHIEF WRITER)**

GCT (W) LYTTON ROAD, LAHORE

SUMAIRA PERVEEN **(MEMBER)**

GCT (W) BAHAWALPUR

HAFIZ USMAN DILSHAD **(MEMBER)**

GCT (M) BAHAWALPUR

Contents

1 CHAPTER NO. 1 (INTRODUCTION) 11

1.1 DATA 11
1.2 INFORMATION 12
1.3 DIFFERENCE BETWEEN DATA AND INFORMATION 13
1.4 INTRODUCTION TO DATABASE MANAGEMENT SYSTEM 13
1.5 HISTORY OF DBMS 14
1.6 CHARACTERISTICS OF DATABASE MANAGEMENT SYSTEM 15
1.7 DBMS VS. FLAT FILE 16
1.8 FIELD DEFINITIONS AND NAMING CONVENTIONS 16
1.8.1 FIELD 16
1.8.2 RECORD 17
1.8.3 FILE 17
1.8.4 TYPES OF FILES 17
1.9 NAMING CONVENTIONS 20
1.9.1 LOWERCASE 20
1.10 COMPONENTS OF DATABASE SYSTEM 22
1.11 MICROSOFT ACCESS 27
1.12 WHAT IS MYSQL? 27

2 CHAPTER NO. 2 (DATABASE SYSTEM) 33

2.1 LEGACY DATA BASE SYSTEMS 33
2.2 FILE PROCESSING SYSTEM 33
2.3 DATA MODEL AND ITS TYPES 35
2.3.1 HIERARCHICAL DATABASE MODEL 36
2.3.2 NETWORK MODEL 39

3 CHAPTER NO. 3 (DATABASE MODELS) 46

3.1 WHAT DOES DATABASE MODEL MEAN? 46
3.1.1 DATA MODEL 46

3.2 SEMANTIC DATA MODEL 47
3.3 RELATIONAL MODEL 52

4 CHAPTER NO. 4 (RELATIONAL DATABASE MANAGEMENT) 65

4.1 RELATIONAL DATABASE MANAGEMENT 65
4.1.1 ENTITY 65
4.1.2 ATTRIBUTES 66
4.1.3 TYPES OF ATTRIBUTES 66
4.2 WHAT IS A DATABASE TABLE? 67
4.2.1 WHAT IS RELATIONAL DATABASE MANAGEMENT SYSTEM? 67
4.2.2 CUSTOMERS TABLE 68
4.2.3 ORDERS TABLE 69
4.2.4 SHIPPERS TABLE 69
4.3 WHAT IS DATA REDUNDANCY? 72
4.3.1 HOW DOES DATA REDUNDANCY OCCUR? 72
4.3.2 UNDERSTANDING DATABASE VERSUS FILE-BASED DATA REDUNDANCY 72
4.3.3 TOP 4 ADVANTAGES OF DATA REDUNDANCY 73
4.4 RELATIONAL DATABASE MANAGEMENT TERMINOLOGIES 79
4.4.1 TUPLES 80
4.4.2 ATTRIBUTES 81
4.4.3 DOMAIN 81
4.4.4 DEGREE 82
4.5 KEYS 83
4.5.1 INTRODUCTION TO DATABASE KEYS 83
4.5.2 SUPER KEY 84
4.5.3 CANDIDATE KEY 84
4.5.4 PRIMARY KEY 84
4.5.5 ALTERNATE KEY 85
4.5.6 COMPOSITE KEY 86
4.5.7 FOREIGN KEY 86
4.6 RELATIONAL DATA INTEGRITY 88
4.6.1 ENTITY INTEGRITY 89
4.6.2 DOMAIN INTEGRITY 89
4.6.3 REFERENTIAL INTEGRITY 89
4.7 RELATIONAL SET OPERATORS 91

5 CHAPTER NO. 5 (NORMALIZATION OF DATABASE TABLES) 96

5.1 DATABASE ANOMALIES 96
5.1.1 TYPES OF ANOMALIES. 96
5.2 NORMALIZATION 98
5.2.1 PURPOSES OF NORMALIZATION 98
5.2.2 CHARACTERISTICS OF NORMALIZED DATABASE 99
5.3 FUNCTIONAL DEPENDENCY 99
5.4 FIRST NORMAL FORM 99
5.4.1 PROBLEMS IN INF 101
5.5 SECOND NORMAL FORM 102
5.5.1 ANALYSIS OF SECOND NORMAL FORM (2NF) 104
5.6 THIRD NORMAL FORM 105
5.7 BOYCE-CODD NORMAL FORM (BCNF) 108

6 CHAPTER NO. 6 (RELATIONAL ALGEBRA AND SQL) 114

6.1 INTRODUCTION OF RELATIONAL ALGEBRA AND SQL IN DBMS 114
6.1.1 UNARY OPERATIONS 116
6.1.2 SELECT OPERATION (σ) 116
6.2 PROJECT OPERATION (∏) 118
6.3 UNION OPERATION (U) 119
6.4 SET DIFFERENCE (-) 119
6.5 CARTESIAN PRODUCT (X) 120
6.6 RENAME OPERATION (ρ) 121
6.7 SQL INNER JOIN KEYWORD 128
6.7.1 INNER JOIN SYNTAX 128
6.8 DEMO DATABASE 129
6.9 SQL INNER JOIN EXAMPLE 130
6.10 JOIN THREE TABLES 130
6.11 DIFFERENT TYPES OF SQL JOINS 131
6.12 WHAT IS SQL? 132
6.12.1 WHAT IS AN OPERATOR IN SQL? 132
6.12.2 SQL ARITHMETIC OPERATORS 132
6.12.3 SQL COMPARISON OPERATORS 133
6.12.4 SQL LOGICAL OPERATORS 134

6.13 DATABASE LANGUAGE 136
6.13.1 TYPES OF DATABASE LANGUAGE 137
6.13.2 DATA DEFINITION LANGUAGE (DDL) 137
6.13.3 DATA MANIPULATION LANGUAGE 138
6.13.4 DATA CONTROL LANGUAGE 140
6.13.5 TRANSACTION CONTROL LANGUAGE 141
6.14 AGGREGATE FUNCTIONS (TRANSACT-SQL) 141
6.15 INTRODUCTION TO SQL COUNT FUNCTION 142
6.16 SQL COUNT FUNCTION EXAMPLES 143
6.17 INTRODUCTION TO SQL SUM FUNCTION 144
6.18 INTRODUCTION TO THE SQL SERVER MAX ( ) FUNCTION 145
6.19 SQL SERVER MIN () FUNCTION 146
6.20 INTRODUCTION TO SQL SERVER AVG () FUNCTION 147
6.21 SQL SERVER AVG () FUNCTION: ALL VS. DISTINCT 148

7 CHAPTER NO. 7 (DATABASE LIFE CYCLE (DBLC)) 157

7.1 DATABASE LIFE CYCLE (DBLC) 157
7.2 THE DATABASE INITIAL STUDY 158
7.2.1 ANALYZE THE COMPANY SITUATION: 159
7.2.2 DEFINE PROBLEMS AND CONSTRAINTS: 159
7.2.3 DEFINE OBJECTIVES: 159
7.2.4 DEFINE SCOPE AND BOUNDARIES: 159
7.3 DATABASE DESIGN 160
7.3.1 IMPLEMENTATION AND LOADING: 160
7.3.2 TESTING AND EVALUATION: 161
7.3.3 OPERATION 162
7.3.4 MAINTENANCE AND EVOLUTION 162
7.4 DATABASE DESIGN STRATEGIES 164
7.4.1 TOP – DOWN DESIGN METHOD 164
7.4.2 BOTTOM – UP DESIGN METHOD 165
7.5 CENTRALIZED DESIGN 166
7.5.1 DECENTRALIZED DESIGN 166
7.5.2 CENTRALIZED VERSUS DECENTRALIZED DESIGN 168

8 CHAPTER NO. 8 (ENTITY RELATIONSHIP (E-R) MODELING) 174

8.1 Entity Relationship (E-R) Modeling 174
8.2 Basic Modeling Concepts 175
8.2.1 ER model: 175
8.2.2 History of ER models 176
8.2.3 Why use ER Diagrams? 176
8.2.4 Facts about ER Diagram Model 176
8.2.5 ER Diagrams Symbols & Notations 177
8.2.6 ER Model 178
8.3 What is Entity? 179
8.4 Entity set: 180
8.5 Degrees of Data Abstraction: 180
8.5.1 Three levels of data abstraction: 181
8.5.2 View Level or External Schema 181
8.5.3 Conceptual Level or Logical Level 182
8.5.4 Physical Level or Internal Schema 182
8.6 Association and Cardinality 183
8.6.1 Association: 183
8.6.2 Cardinality 183
8.7 Relationship Participation 185
8.7.1 Total Participation 185
8.7.2 Partial Participation 186
8.8 Composite Entities, Entity Super types and subtypes 187
8.8.1 Traditional Entity: 187
8.8.2 Composite Entity: 188
8.8.3 Subtype/Supertype Entity: 188
8.8.4 Weak Entity: 188
8.8.5 Strong Entity: 188
8.8.6 Attributes: 188
8.8.7 Types of Attributes 189
8.9 Enhanced Entity Relationship Diagram 190
8.9.1 Enhanced ER Model 190
8.9.2 Features of EER Model 190
8.9.3 Sub Class and Super Class 190
8.10 Category or Union 193
8.11 Aggregation 194
8.12 Transform ER/EER to Relational Model 195

9 CHAPTER NO. 9 (TRANSACTION MANAGEMENT) 210

9.1 WHAT IS TRANSACTION? 210
9.1.1 PROCESS OF TRANSACTION 211
9.2 EVALUATING TRANSACTION RESULTS 212
9.2.1 TRANSACTION PROPERTIES OR ACID PROPERTIES. 213
9.2.2 STATES OF TRANSACTION 214
9.2.3 ADVANTAGES OF EXECUTION OF TRANSACTION 215
9.3 TRANSACTION MANAGEMENT WITH SQL 216
9.4 TRANSACTION LOG 217
9.4.1 THE TRANSACTION LOG SUPPORTS THE FOLLOWING OPERATIONS: 218
9.4.2 RECOVERY OF ALL INCOMPLETE TRANSACTIONS WHEN SQL SERVER IS STARTED 218
9.4.3 ROLLING A RESTORED DATABASE, FILE, FILE GROUP, OR PAGE FORWARD TO THE POINT OF FAILURE 219
9.4.4 SUPPORTING TRANSACTIONAL REPLICATION 219
9.4.5 SUPPORTING HIGH AVAILABILITY AND DISASTER RECOVERY SOLUTIONS 219
9.4.6 TRANSACTION LOG CHARACTERISTICS 220
9.4.7 TRANSACTION LOG TRUNCATION 220
9.4.8 WHAT IS CONCURRENCY CONTROL? 221
9.4.9 WHY USE CONCURRENCY METHOD? 222
9.4.10 CONCURRENCY CONTROL PROTOCOLS 223
9.5 STRICT TWO-PHASE LOCKING METHOD 226
9.5.1 CENTRALIZED 2PL 226
9.5.2 PRIMARY COPY 2PL 226
9.5.3 DISTRIBUTED 2PL 226
9.5.4 TIMESTAMP BASED PROTOCOL 227
9.6 TYPES OF TRANSACTIONS: 229
9.6.1 IMPLICIT TRANSACTIONS 229
9.6.2 EXPLICIT TRANSACTIONS 230

1 Chapter No. 1 (Introduction)

Objectives

After completion of this chapter students will be able to:

- ✓ Explain Introduction
- ✓ Learn Introduction to DMBS
- ✓ Learn Advantage and disadvantage of DBMS
- ✓ Understand Field Definitions and Naming Conventions
- ✓ Learn Components of DB Applications
- ✓ Learn DB Tools; Microsoft Access, MySQL

1. Introduction:

Explain data and information with examples.

DATA

A collection of raw facts and figures related to an object is called Data.

- ✓ **Object →** person (or student), an organization, an event or any other things etc.
- ✓ **Data →** in the form of text, numbers, images, sounds, and videos.
- ✓ **Processed** to produce meaningful information.

Importance of Data

- ✓ Gives view of current and past activities or history related to the rise and fall of an organization.
- ✓ Helps an organization in making decisions for future activities.

Examples of Data

Ali	22	55	33
Salman	54	44	75
Kamran	56	88	45

This table of student's names and numbers represents data of students.

Information

The processed data.

- ✓ Provides useful meanings.
- ✓ Data is used as input for processing and information is the output of this Processing.



Example of Information

Sr.NO	Name	English	Chemistry	Computer	Total	Grade
1	Ali	44	55	66	165	B
2	Kamran	34	65	34	133	C
3	USMAN	67	76	45	188	A

The above processed data conveys the clear and proper meanings

Difference between Data and Information

DATA	Information
Raw facts and figures.	Information is Processed data.
Does not give useful and proper meanings.	Gives useful and proper meanings.
Input for processing.	Output of processing.
Does not depend on information.	Depends upon data.
Huge in volume.	Normally short in volume.
Data us huge in volume	Normally short in volume

Introduction to database Management system

A database management system (DBMS) refers to the technology for creating and managing databases. DBMS is a software tool to organize (create, retrieve, update, and manage) data in a database.

The main aim of a DBMS is to supply a way to store up and retrieve database information that is both convenient and efficient. By data, we mean known facts that can be recorded and that have embedded meaning. Usually, people use software such as DBASE IV or V, Microsoft ACCESS, or EXCEL to store data in the form of a database. A datum is a unit of data. Meaningful data combined to form information. Hence, information is interpreted data - data provided with semantics. MS. ACCESS is one of the most common examples of database management software.

Knowledge refers to the useful use of information. As you know, that information can be transported, stored, and shared without any problems and difficulties, but

the same cannot be said about knowledge. Knowledge necessarily involves personal experience and practice.
Database systems are meant to handle an extensive collection of information. Management of data involves both defining structures for storage of information and providing mechanisms that can do the manipulation of those stored information. Moreover, the database system must ensure the safety of the information stored, despite system crashes or attempts at unauthorized access.

Let us see a simple example of a university database. This database is maintaining information concerning students, courses, and grades in a university environment. The database is organized as five files:

- ✓ The STUDENT file stores data of each student
- ✓ The COURSE file stores contain data on each course.
- ✓ The SECTION stores the information about sections in a particular course.
- ✓ The GRADE file stores the grades which students receive in the various sections
- ✓ The TUTOR file contains information about each professor.

Definition of DBMS:

DBMS stands for Database Management System. We can break it like this DBMS = Database + Management System. Database is a collection of data and Management System is a set of programs to store and retrieve those data. Based on this we can define DBMS like this: DBMS is a collection of inter-related data and set of programs to store & access those data in an easy and effective manner We need to specify the structure of the records of each file by defining the different types of data elements to be stored in each record.

History of DBMS

Here, are the important landmarks from the history:

1960 - Charles Bachman designed first DBMS system

1970 - Codd introduced IBM'S Information Management System (IMS)

1976- Peter Chen coined and defined the Entity-relationship model also known as the ER model

1980 - Relational Model becomes a widely accepted database component

1985- Object-oriented DBMS develops.

1990s- Incorporation of object-orientation in relational DBMS.

1991- Microsoft ships MS access, a personal DBMS and that displaces all other personal DBMS products.

1995: First Internet database applications

1997: XML applied to database processing. Many vendors begin to integrate XML into DBMS products.

Characteristics of Database Management System

Here are the characteristics and properties of Database Management System:

- ✓ Provides security and removes redundancy
- ✓ Self-describing nature of a database system
- ✓ Insulation between programs and data abstraction
- ✓ Support of multiple views of the data
- ✓ Sharing of data and multiuser transaction processing
- ✓ Database Management Software allows entities and relations among them to form tables.
- ✓ It follows the ACID concept (Atomicity, Consistency, Isolation, and Durability).
- ✓ DBMS supports multi-user environment that allows users to access and manipulate data in parallel.

DBMS vs. Flat File

DBMS	Flat File Management System
Multi-user access	It does not support multi-user access
Design to fulfill the need for small and large businesses	It is only limited to smaller DBMS system.
Remove redundancy and Integrity	Redundancy and Integrity issues
Expensive. But in the long term Total Cost of Ownership is cheap	It's cheaper
Easy to implement complicated transactions	No support for complicated transactions

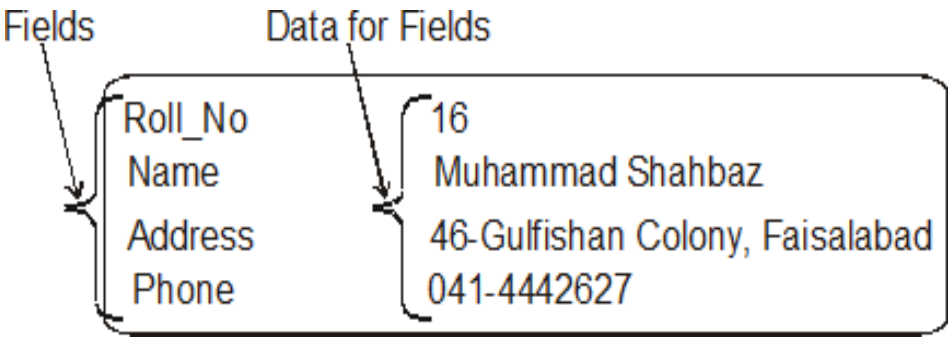
Field Definitions and Naming Conventions

1.1.1 Field

- ✓ A set of related characters
- ✓ Represents smallest unit of data.
- ✓ Field name: Each field is given a unique name. Example: Roll_No, Name, Address and Marks of a student of a class.
- ✓ Each field contains one specific piece of information.
- ✓ Field size: The maximum number of characters that can be stored in a field.
- ✓ Each field can contain only one type of data such as text or numbers or dates and so on.

1.1.2 Record

- ✓ A collection of related fields.
- ✓ A record is treated as a single unit.
- ✓ Example, a student's record includes a set of fields that may contain Roll_No, Name, Address, and Phone etc.



1.1.3 File

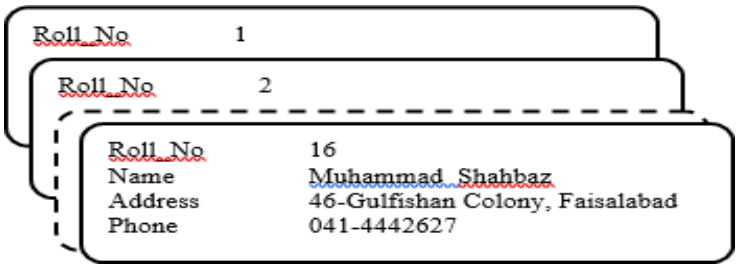
- ✓ File or data set: A collection of related records.
- ✓ These related records are treated as a single unit.
- ✓ Example: A collection of records of students of a college.
- ✓ Files are stored on storage media such as hard disk, CD-ROM, USB drive etc.

1.1.4 Types of Files

(i) Usage point of view

(ii) Functional point of view

(iii) Storage point of view



Types of Files (Usage Point of View, Functional point of view)

Usage Point of View	Functional Point of view
Master File	Program File
Transaction File	Data File
Backup File	

File Types from Usage Point of View

1. Master File

- ✓ Used to store the information that remains constant for a long period of time.
- ✓ Example: Employee's master file, can contain the fields like 'Name', 'Address', 'Telephone Number' etc.
- ✓ Master file is the latest updated file.
- ✓ It is updated when any change in its contents is required.
- ✓ In updating process, records can be edited (changed), deleted or new records can be added in a master file.
- ✓ Once a master file is created, it cannot become empty.

2. Transaction File

- ✓ A type of file that is used to store input data before processing
- ✓ It may be a temporary file.
- ✓ Usually used to update data in master file.
- ✓ It may exist until the master file is updated.
- ✓ It may also be used to maintain a permanent record of data about a transaction.

3. Backup File

- ✓ A type of file that is used to take the backup of important data.
- ✓ Permanent file.
- ✓ Make the duplicate copy of data.
- ✓ Created for the protection of important data or files.
- ✓ The data can be recovered from backup files if any data file is lost or damaged.
- ✓ Backup files are created by using specific software (utility program).

File Types from functional Point of View

- ✓ Different files perform different functions.
- ✓ A file consists of file name and file extension. Separated by dot (.).
- ✓ Recognized by its file extension.
- ✓ The extension of a file is normally assigned by the software in which it is created.

1. Program File

- ✓ A type of file that contains a set of instructions of program.
- ✓ Executable file.
- ✓ File extensions: exe or com.

2. Data File

- ✓ A type of file that contains data
- ✓ Data files are created by the software being used
- ✓ Different programs (or software) store data in the data files using different formats.
- ✓ Created into one application program cannot be used into another application program.
- ✓ Some software or application programs provide facility to use the data files of different formats.
- ✓ Generally recognized by the file extension. For example: .doc is known as document file.

Examples of some file types are given in following table

Software	File Extension	File Type
Word Processor	.doc	Document file
Spreadsheet	.xls, .wks	Worksheet files
Database	.mdb, .dat, .dbf,	Database files
NotePad	.txt	ASCII/Text files
Image Processors	.gif, .jpg, .bmp, .tif	Image files
Audio	.wav, .mid	Audio files
Video	.avi, .mpg	video files

Naming Conventions

Avoid quotes. If you have to quote an identifier then you should rename it. Quoted identifiers are a serious pain. Writing SQL by hand using quoted identifiers is frustrating and writing dynamic SQL that involves quoted identifiers is even more frustrating. This also means that you should never include whitespace in identifier names.
Ex: Avoid using names like "FirstName" or "All Employees"

1.1.5 Lowercase

Identifiers should be written entirely in lower case. This includes tables, views, column, and everything else too. Mixed case identifier names means that every usage of the identifier will need to be quoted in double quotes (which we already said are not allowed). Ex: Use first_name, not "First_Name".

Data types are not names

Database object names, particularly column names, should be a noun describing the field or object. Avoid using words that are just data types such as text or timestamp. The latter is particularly bad as it provides zero context.

Underscores separate words

Object name that are comprised of multiple words should be separated by underscores (ie. snake case).
Ex: Use word_count or team_member_id, not wordcount or wordCount.

Full words, not abbreviations

Object names should be full English words. In general, avoid abbreviations, especially if they're just the type that removes vowels. Most SQL databases support at least 30-character names which should be more than enough for a couple English words. PostgreSQL supports up to 63-character for identifiers.
Ex: Use middle_name, not mid_nm.

Use common abbreviations

For a few long words the abbreviation is both more common than the word itself. "Internationalization" and "localization" are the two that come up most often as i18n and l10n respectively. In these cases, use the abbreviation.
If you're in doubt, use the full English word. It should be obvious where the abbreviation makes sense.

Avoid reserved words.

Avoid using any word that is considered a reserved word in the database that you are using. There aren't that many of them so it's not too much effort to pick a different word. Depending on the context, reserved words may require quoting. This means sometimes you'll write "user" and sometimes just user.

Another benefit of avoiding reserved words is that less-than-intelligent editor syntax highlighting won't erroneously highlight them. Ex: Avoid using words like user, lock, or table.

Components of Database System

In order to facilitate these functions, DBMS has the following key components:

1) Software

DBMS is primarily a software system that can be considered as a management console or an interface to interact with and manage databases. The interfacing also spreads across real-world physical systems that contribute data to the backend databases. The OS, networking software, and the hardware infrastructure is involved in creating, accessing, managing, and processing the databases.

2) Hardware

The hardware is the actual computer system used for keeping and accessing the database. Conventional DBMS hardware consists of secondary storage devices, usually hard disks, on which the database physically resides, together with the associated Input-Output devices, device controllers and· so forth. Databases run on a' range of machines, from Microcomputers to large mainframes. Other hardware issues for a DBMS includes database machines, which is hardware designed specifically to support a database system.

3) Data

DBMS contains operational data, access to database records and metadata as a resource to perform the necessary functionality. The data may include files with such as index files, administrative information, and data dictionaries used to represent data flows, ownership, structure, and relationships to other records or objects.

4) Procedures

While not a part of the DBMS software, procedures can be considered as instructions on using DBMS. The documented guidelines assist users in designing, modifying, managing, and processing databases.

5) Database languages

These are components of the DBMS used to access, modify, store, and retrieve data items from databases; specify database schema; control user access; and perform other associated database management operations. Types of DBMS languages include Data Definition Language (DDL), Data Manipulation Language (DML), Database Access Language (DAL) and Data Control Language (DCL).

6) Query processor

As a fundamental component of the DBMS, the query processor acts as an intermediary between users and the DBMS data engine in order to communicate query requests. When users enter an instruction in SQL language, the command is executed from the high-level language instruction to a low-level language that the underlying machine can understand and process to perform the appropriate DBMS functionality. In addition to instruction parsing and translation, the query processor also optimizes queries to ensure fast processing and accurate results.

7) Runtime database manager

A centralized management component of DBMS that handles functionality associated with runtime data, which is commonly used for context-based database access. This component checks for user authorization to request the query; processes the approved queries; devises an optimal strategy for query execution; supports concurrency so that multiple users can simultaneously work on same databases; and ensures integrity of data recorded into the databases.

8) Database manager

Unlike the runtime database manager that handles queries and data at runtime, the database manager performs DBMS functionality associated with the data

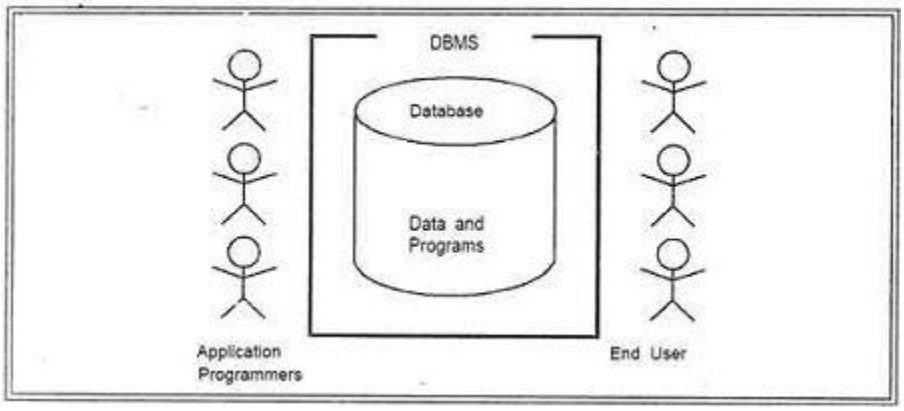
within databases. Database manager allows a set of commands to perform different DBMS operations that include creating, deleting, backup, restoring, cloning, and other database maintenance tasks. The database manager may also be used to update the database with patches from vendors.

9) Database engine

This is the core software component within the DBMS solution that performs the core functions associated with data storage and retrieval. A database engine is also accessible via APIs that allow users or apps to create, read, write, and delete records in databases.

10) Reporting

The report generator extracts useful information from DBMS files and displays it in structured format based on defined specifications. This information may be used for further analysis, decision making, or business intelligence



1REPORTING

Advantages and Disadvantages of DBMS

Following are DBMS advantages and disadvantages.

Advantages of DBMS:

Data Sharing

- ✓ Data can be shared among authorized users of the organization.

- ✓ DBA manages data and gives rights to users to access data.
- ✓ Many users can be authorized to access the same piece of information simultaneously. (remote users)
- ✓ Data of same database can be shared between different application programs.

Data Independence

- ✓ Separation of data structure of database from the application program
- ✓ Separated database and application programs
- ✓ Users can easily change structure of database without modifying application program. Example: user can modify the size or data type of a data items.

Support complex Data Relationships

- ✓ Allows user to design complex data structures.
- ✓ Enables users to view and access data in different ways

Data Security

- ✓ Protection of the database from unauthorized access.
- ✓ DBMS provides several procedures to maintain data security.
- ✓ Only the authorized persons are allowed to access
- ✓ Partial access to users (Part of Database)
- ✓ Some permitted only to retrieve data
- ✓ Others are allowed to retrieve + update data.
- ✓ Database access controlled by the DBA.
- ✓ Creates the accounts of users
- ✓ Gives rights to access the database.
- ✓ Users are given usernames protected by passwords.

Database Backup and Recovery

- ✓ Provide the facility of backup and recovery
- ✓ Backup of important data.
- ✓ Data can be recovered from backup file
- ✓ if the original data file is lost or damaged

Advanced Capabilities

- ✓ Provides advance capabilities
- ✓ for online access
- ✓ Reporting of data through Internet.
- ✓ Online database systems.
- ✓ Database technology + Internet technology to access data on the web servers.

Disadvantage of DBMS:

Following are some disadvantages of DBMS

Cost of Hardware & Software

- ✓ Requirement to run the DBMS software
- ✓ High speed processor
- ✓ Large size memory
- ✓ DBMS software is also very costly.

Cost of Data Conversion

- ✓ Traditional file system - replaced with - database system
- ✓ data stored into data file - converted into - database file
- ✓ Conversion → difficult and costly method

Cost of Staff Training

- ✓ DBMSs, complex systems
- ✓ Training of users required at all levels
- ✓ Programming
- ✓ Application development

- ✓ Database administration.
- ✓ Cost on training.

Appointing Technical Staff

- ✓ Trained technical persons required to manage the DBMS.
- ✓ Database administrator
- ✓ Application programmers etc.
- ✓ Salaries to the technical staff, cost increases.

Database Damage

- ✓ All data integrated into a single database.
- ✓ If database is damaged
- ✓ due to power failure or database is corrupted on the storage media, then valuable data may be lost forever.

Need of Data Dictionary

- ✓ Data dictionary is required, must be installed
- ✓ to share data between application systems and for many other purposes.
- ✓ Internal contents of the company's databases must be documented in a consistent manner.
- ✓ Useful tool but expensive.

Microsoft Access

Microsoft Access is a database management system (DBMS) from Microsoft that combines the relational Microsoft Jet Database Engine with a graphical user interface and software-development tools. It is a member of the Microsoft 365 suite of applications, included in the Professional and higher editions or sold separately

What is MySQL?

MySQL is an open-source relational database management system (RDMS) which was initially designed by MySQL AB in 1995. Later, the system was acquired by the Oracle Corporation.

For those who're unaware, open-source software is free to use, and their code is generally made open so that other developers can modify it.

As mentioned earlier, MySQL is one of the popular RDMS systems which is largely used on the web instead of the offline data management.

MySQL is written using C and C++ languages, and it supports all the major platforms like Windows, Linux, Solaris, macOS, and FreeBSD.

The system has already been implemented in many web apps, which are database-driven such as WordPress, Joomla, and Drupal. Most of the popular websites such as Google, Facebook, and Twitter also use MySQL in one way or another.

Since the service is free and open-source, it is quite popular among the startup communities. Generally, MySQL is used with PHP and APACHE web server on top of Linux distribution, hence the popular acronym LAMP (Linux, Apache, MySQL, PHP).

EXERCISE No. 01

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1. The DBMS acts as an interface between what two components of an enterprise-class database system?
   A. Database application and the database
   B. Data and the database
   C. The user and the database application
   D. Database application and SQL
2. The following are components of a database except _____.
   A. user data B. metadata
   C. reports D. indexes
3. DBMS stands for what
   A. Database Management System
   B. Database Master System
   C. Database Management Structure
   D. None of the above
4. Database is an organized collection of related……
   A. Data B. Modules
   B. Programs D. None of the above
5. Before use of DBMS information was stored using _____.
   A. Cloud Storage C. Data System
   B. File Management System D. None of the above
6. DBMS helps achieve
   A. Data independence
   B. Centralized control of data
   C. Both A and B
   D. None of the above
7. An advantage of the database management approach is
   A. Data is dependent on programs
   B. Data redundancy increases
   C. Data is integrated and can be accessed by multiple programs

D. None of the above

8. A collection of related data.
   A. Information
   B. Valuable information
   C. Database
   D. Metadata
9. DBMS manages the interaction between _____ and database
   A. Users
   B. Clients
   C. End Users
   D. Stake Holders
10. Which of the following is not involved in DBMS?
   A. End Users
   b) Data
   c) Application Request
   d) HTML
11. Database is generally _____
   A. System-centered
   B. User-centered
   C. Company-centered
   D. Data-centered
12. Object =_____+ relationships.
   A. data
   B. attributes
   C. entity
   D. constraints
13. The term _____ is used to refer to a row.
   A. Attribute
   B. Tuple
   C. Field
   D. Instance
14. The term attribute refers to a _____ of a table.

A. Record
B. Column
C. Tuple
D. Key

15. The tuples of the relations can be of _____ order.
    A. Any
    B. Same
    C. Sorted
    D. Constant

ANSWER KEY

1. A	**2.** C	**3.** A
4. A	**5.** B	**6.** C
7. C	**8.** C	**9.** C
10.D	**11.**B	**12.**C
13.B	**14.**B	**15.**A

PART-II SAMPLE SHORT QUESTIONS

1. Define Data?
2. Define Information?
3. Define DBMS?
4. Characteristics of DBMS?
5. Define Filed and Record?
6. Name the components of DMBS?
7. DBMS vs Flat File?
8. Advantage and Disadvantage of DBMS?
9. What is Microsoft Access
10. What is MySQL?
11. What is Tuple?
12. What is file management service?
13. Define Data redundancy?
14. What is meant by efficient database?
15. Define data Integration

PART-III SAMPLE LONG QUESTIONS

1. Define Data and types of data also describe information?
2. What is File also describe types of file System.
3. Explain about components of DBMS.

2 Chapter No. 2 (Database System)

Objectives

After completion of this chapter students will be able to understand:

- ✓ **Legacy DB Systems**
- ✓ **File Processing Systems**
- ✓ **Hierarchical Model**
- ✓ **Network Mode**

Legacy Data Base Systems

A legacy database is a group of different databases that combines different kinds of data systems, such as relational or object-oriented databases, hierarchical databases, network databases, spreadsheets, multimedia databases, or filesystems.

File Processing System

File Processing System (FPS) is a way of storing, retrieving and manipulating data which is present in various files.

Files are used to store various documents. All files are grouped based on their categories. The file names are much related to each other and arranged properly to easily access the files. In file processing system, if one needs to insert, delete, modify, store or update data, one must know the entire hierarchy of the files.

Advantages of File Processing System:

1) **Cost friendly –**
   There is a very minimal to no set up and usage fee for File Processing System. (In most cases, free tools are inbuilt in computers.)

2) **Easy to use –**
File systems require very basic learning and understanding, hence, can be easily used.

3) **High scalability –**
One can very easily switch from smaller to larger files as per his needs.

Disadvantage OF File Processing System

1. Data Redundancy and Inconsistency
2. Difficulty in Accessing the Data
3. Integrity Problems:
4. Atomicity problem
5. Concurrent-access anomalies
6. Security Problems

1. Data Redundancy and Inconsistency:

Since the data files and application programs are created by different programmers over a long period.

- ✓ The data files are likely to have different formats.
- ✓ Program maybe written in several programming languages.
- ✓ Same information may be duplicated in several places.

2. Difficulty in Accessing the Data:

Conventional file processing system doesn't allow needed data to be retrieved in a convenient and efficient manner. For Example, consider a data file, saving account data file with fields

{acc_no, name, social security, addr, balance}.

Application programs to access the data are written, but if user wants to display only those records for which balance is greater than $10,000. And is that program is not written, then it is difficult to access that data.

3. Integrity Problems:

The data values stored in the database must satisfy certain types of consistency constraints. Application programmers enforce these consistency constraints by adding appropriate code in the various application programs, however, when a new constraint is to be added, it is difficult to change the program to enforce the new constraint.

4. Atomicity problem:

A computer system is subject failure. In many applications, it is crucial to ensure that, once a failure has occurred and has been detected, the data are stored to the consistent state that existed prior to the failure. It is difficult to ensure this property in a conventional file-processing system.

5. Concurrent-access anomalies:

If multiple users are updating the same data simultaneously it will result in inconsistent data state. In file processing system it is very difficult to handle this using program code. This results in concurrent access anomalies.

6. Security Problems

Not every user of the database system should be able to access all the data. For example, in a banking system, payroll personnel need to see only that part of the database that has information about various bank employees. They do not need access to information about customer accounts. Since application programs are added to the system in ad hoc manner, it is difficult to enforce such security constraints.

Data Model and Its Types

A database model refers to the logical structure, representation or layout of a database and how the data will be stored, managed and processed within it. It helps in designing a database and serves as design for application developers and database administrators in creating a database.

Data Model gives us an idea that how the final system will look like after its complete implementation. It defines the data elements and the relationships between the data elements. Data Models are used to show how data is stored,

connected, accessed and updated in the database management system. Here, we use a set of symbols and text to represent the information so that members of the organization can communicate and understand it. Though there are many data models being used nowadays but the Relational model is the most widely used model. Apart from the Relational model, there are many other types of data models about which we will study in details in this blog. Some of the Data Models in DBMS are:

- ✓ Hierarchical Model
- ✓ Network Model
- ✓ Relational Model

2.1.1 Hierarchical Database Model

In hierarchical model, data is organized into a tree like structure with each record is having one parent record and many children. The main drawback of this model is that, it can have only one to many relationships between nodes.

This model structure allows the one-to-one and a one-to-many relationship between two/ various types of data. This structure is very helpful in describing many relationships in the real world; table of contents, any nested and sorted information.

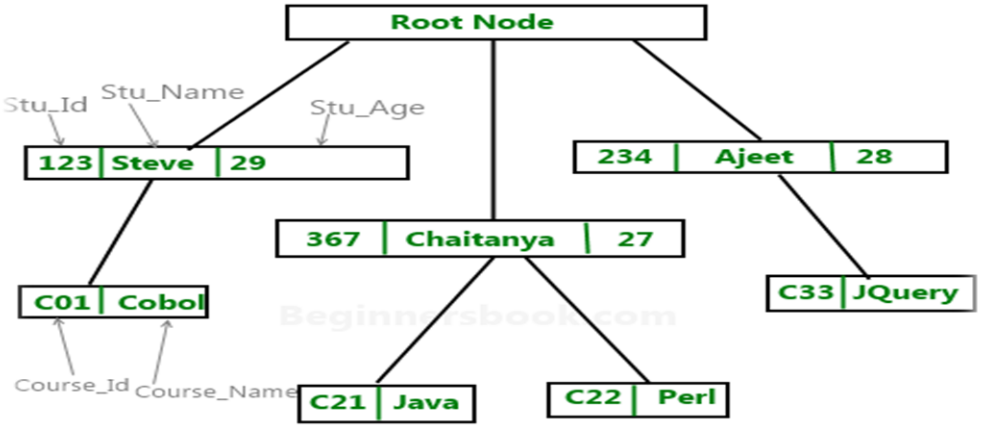
The hierarchical structure is used as the physical order of records in storage. One can access the records by navigating down through the data structure using pointers which are combined with sequential accessing. Therefore, the hierarchical structure is not suitable for certain database operations when a full path is not also included for each record.

Data in this type of database is structured hierarchically and is typically developed as an inverted tree. The "root" in the structure is a single table in the database and other tables act as the branches flowing from the root. The diagram below shows a typical hierarchical database structure

Note: Hierarchical models are rarely used now.

Sample Hierarchical Model Diagram:

Let's say we have few students and few courses and a course can be assigned to a single student only, however a student take any number of courses so this relationship becomes one to many.



Example of hierarchical data represented as relational tables:

The above hierarchical model can be represented as relational tables like this:

Stu_Id	Stu_Name	Stu_Age
123	Steve	29

367	Chaitanya	27
234	Ajeet	28

Course Table

Course_Id	Course_Name	Stu_Id
C01	Cobol	123
C21	Java	367
C22	Perl	367
C33	JQuery	234

What are the characteristics of the hierarchical model?

Does not support many to many relationships:

- ✓ **Deletion problem:**

  If a parent is deleted, the child has also deleted automatically.

- ✓ **Data hierarchy:**

  Data can be represented as a hierarchical tree as can be seen in the figure.

Each child record can have only one parent record:

- ✓ **Hierarchy through pointer:**

  Pointers are used to link the records. Pointer determines that which record is parent record and which one is child record.

- ✓ **Minimize disk input and output:**

  Parent and child records are stored close to each other on the storage device. It helps to minimize the hard disk input and disk output.

- ✓ **Fast navigation:**

  Due to the short distance between parent to child, database access time and performance is improved. Navigation through the database is very fast in a hierarchical model.

- ✓ **Predefined relationships between records:**

  All relationships are predefined. Root nodes, parents, and the child are predefined in the database schema.

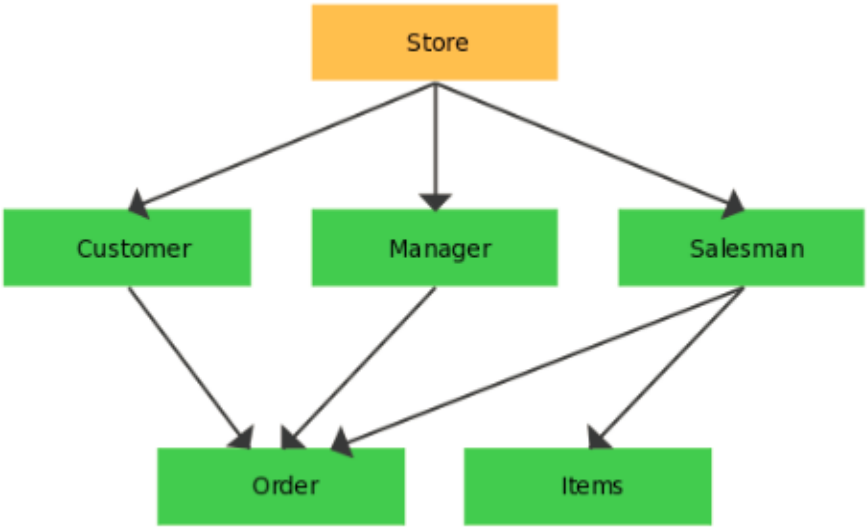
- ✓ **Difficult to re-organize:**

  It is difficult to re-organize the database due to hierarchy. It is difficult to re-organize because parent to child relationships can be disturbed

2.1.2 Network Model

A network database model is a database model that allows multiple records to be linked to the same owner file. The model can be seen as an upside-down tree

where the branches are the member information linked to the owner, which is the bottom of the tree. The multiple linkages which this information allows the network database model to be very flexible. In addition, the relationship that the information has in the network database model is defined as many-to-many relationship because one owner file can be linked to many member files and vice versa. The network database model was invented by Charles Bachman in 1969.



Network Model

A network model is a database model that is designed as a flexible approach to representing objects and their relationships. A unique feature of the network model is its schema, which is viewed as a graph where relationship types are arcs and object types are nodes.

Unlike other database models, the network model's schema is not confined to be a lattice or hierarchy; the hierarchical tree is replaced by a graph, which allows for more basic connections with the nodes.

Charles Bachman was the original inventor of the network model. In 1969, the **Conference on Data Systems Languages (CODASYL)** Consortium developed the

network model into a standard specification. A second publication was introduced in **1971,** which later turned into the basis for virtually all implementations. It was widely supplanted by the Relational Model later on because of its higher-level, more declarative interface.

The main advantage of the network model is the ability to address the lack of flexibility of the hierarchical model, of which it is supposed to be a direct evolution. In the network model, each child (called "member") can have more than one parent (called "owner") to generate more complex, many-to-many relationships.

The benefits of the network model include:

- ✓ **Simple Concept:** Similar to the hierarchical model, this model is simple and the implementation is effortless.
- ✓ **Ability to Manage More Relationship Types:** The network model has the ability to manage one-to-one (1:1) as well as many-to-many (N: N) relationships.
- ✓ **Easy Access to Data:** Accessing the data is simpler when compared to the hierarchical model.
- ✓ **Data Integrity:** In a network model, there's always a connection between the parent and the child segments because it depends on the parent-child relationship.
- ✓ **Data Independence:** Data independence is better in network models as opposed to the hierarchical models.

Drawbacks of the network model include:

- ✓ **System Complexity**: Each and every record has to be maintained with the help of pointers, which makes the database structure more complex.
- ✓ **Functional Flaws:** Because a great number of pointers is essential, insertion, updates, and deletion become more complex.
- ✓ **Lack of Structural Independence:** A change in structure demands a change in the application as well, which leads to lack of structural independence.

- ✓ **Incomplete Flexibility:** Albeit more flexible than the hierarchical model, the network one still cannot satisfy all relations by assigning another owner.

Advantages of a Network Database Model

- ✓ Because it has the many-many relationships, network database model can easily be accessed in any table record in the database
- ✓ For more complex data, it is easier to use because of the multiple relationship founded among its data
- ✓ Easier to navigate and search for information because of its flexibility

Disadvantage of a Network Database Model

- ✓ Difficult for first time users
- ✓ Difficulties with alterations of the database because when information entered can alter the entire database

Network Database Vs Hierarchical Database Model

Network Database Model	Hierarchical Database Model
Many-to-many relationship	One-to-many relationship
Easily accessed because of the linkage between the information	Difficult to navigate because of its strict owner to member connection
Great flexibility among the information files because the multiple relationships among the files	Less flexibility with the collection of information because of the hierarchical position of the files

EXERCISE No. 02

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1. File-processing systems have important limitations:

a) Data is separated and isolated.
b) Data is often duplicated.
c) Application programs are dependent on file formats.
d) All of the above.

2. Database-processing programs:
a) Call the DBMS to access the stored data.
b) Cannot be used by more than one person.
c) Require at least one dedicated workstation.
d) Present problems with storage space.

3. In a database system, all the application data is stored in a single facility called_____.
a) DBMS
b) CPU
c) Hard drive
d) Database

4. The features and functions of a DBMS can be divided into three parts:
a) Fields, records, and files.
b) RAM, ROM, and floppy diskettes.
c) The design tools subsystem, the run-time subsystem, and the DBMS engine.
d) The file-processing subsystem, the transaction-processing subsystem, and the LAN.

5. In a Hierarchical database model, records are organized as _____ structure.
a) Graph
b) List
c) Links
d) Tree

6. What is the basic relationship in a hierarchical database?

a) Parent-child
b) Sibling
c) Cousin
d) All of the data is related in the same way

7. SET concept is used in

a) Network Model
b) Hierarchical Model
c) Relation Model
d) None of these

8. Which of the following is record based logical model?

a) Network Model
b) Hierarchical Model
c) E-R Model
d) All of these

9. Which of the following is handled by DBMS?

a) Data security
b) Data integrity
c) Data independence
d) All

10. Data that causes inconsistency lacks_____.

a) Data integrity
b) Data redundancy
c) Data anomaly
d) Good data

ANSWERKEY

1. d	2. a	3. d	4. c	5. d
6. a	7. a	8. a	9. d	10. b

PART-II SAMPLE SHORT QUESTIONS

1. What Is Legacy Database?
2. Define File Processing System.
3. Write Down Three Disadvantage OF File Processing System.

4. Define Hierarchical Database Model.
5. Write Two Advantage OF Hierarchical Database Model.
6. What Is Data Redundancy in File Processing System?
7. Define Network Model.
8. Write The Name of First Database Model.
9. What are Disadvantage of Network Model?

PART-III SAMPLE LONG QUESTIONS

1. Describe File Processing System in detail.
2. Describe hierarchical database model.
3. What is Difference between Network Data Model and hierarchical database Model?

3 Chapter No. 3 (DATABASE MODELS)

Objectives

After completion of this chapter students will be able to understand:

- ✓ Learn about database Models
- ✓ Semantic Data Model
- ✓ Relational Model
- ✓ Database Models and the Internet

What Does Database Model Mean?

A database model refers to the logical structure, representation or layout of a database and how the data will be stored, managed and processed within it. It helps in designing a database and serves as design for application developers and database administrators in creating a database.

A database model is primarily a type of data model. Depending on the model in use, a database model can include entities, their relationships, data flow, tables and more. For example, within a hierarchal database mode, the data model organizes data in the form of a tree-like structure having parent and child segments.

Some of the popular database models include relational models, hierarchical models, flat file models, object-oriented models, entity relationship models and network models

3.1.1 Data Model

Data Model gives us an idea that how the final system will look like after its complete implementation. It defines the data elements and the relationships between the data elements. Data Models are used to show how data is stored, connected, accessed and updated in the database management system. Here, we use a set of symbols and text to represent the information so that members of the

organization can communicate and understand it. Though there are many data models being used nowadays but the Relational model is the most widely used model. Apart from the Relational model, there are many other types of data models about which we will study in details in this blog. Some of the Data Models in DBMS are:

- ✓ Semantic Data Model
- ✓ Relational Model
- ✓ Database Models and the Internet

Semantic Data Model

The semantic data model is a method of structuring data in order to represent it in a specific logical way. It is a conceptual data model that includes semantic information that adds a basic meaning to the data and the relationships that lie between them. This approach to data modeling and data organization allows for the easy development of application programs and also for the easy maintenance of data consistency when data is updated.

The semantic data model is a relatively new approach that is based on semantic principles that result in a data set with inherently specified data structures. Usually, singular data or a word does not convey any meaning to humans, but paired with a context this word inherits more meaning.

A semantic data model represents data in terms of named sets of objects, named sets of values, named sets of relationships, and constraints over these object, value, and relationship sets. The semantics of a semantic data model are the intentional declarations: the names for object, value, and relationship sets that indicate intended membership in the various sets and the declared constraints that the data should satisfy. The data of a semantic data model is extensional and consists of instances of object identifiers and values for object and value sets and of m-tuples of instances for m-ary relationship sets. The model of a semantic-data-model instance describes intensionally a real-world domain of interest. The

modeling components of the semantic data model specify the modeling elements from which a real-world model instances can be built.

In a database environment, the context of data is often defined mainly by its structure, such as its properties and relationships with other objects. So, in a relational approach, the vertical structure of the data is defined by explicit referential constraints, but in semantic modeling this structure is defined in an inherent way, which is to say that a property of the data itself may coincide with a reference to another object.

A semantic data model may be illustrated graphically through an abstraction hierarchy diagram, which shows data types as boxes and their relationships as lines. This is done hierarchically so that types that reference other types are always listed above the types that they are referencing, which makes it easier to read and understand.

Abstractions used in a semantic data model:

- ✓ Classification - "instance_of" relations
- ✓ Aggregation - "has_a" relations
- ✓ Generalization - "is_a" relations

A semantic data model in software engineering has various meanings:

It is a conceptual data model in which semantic information is included. This means that the model describes the meaning of its instances. Such a semantic data model is an abstraction that defines how the stored symbols (the instance data) relate to the real world.[1]
It is a conceptual data model that includes the capability to express and exchange information which enables parties to interpret meaning (semantics) from the instances, without the need to know the meta-model. Such semantic models are fact-oriented (as opposed to object-oriented). Facts are typically expressed by binary relations between data elements, whereas higher order relations are expressed as collections of binary relations. Typically binary relations have the form of triples: Object-Relation Type-Object. For example: the Eiffel Tower <is located in> Paris.

Data Semantics

Static Information

- ✓ Data -- Entities
- ✓ Associations -- Relationships among entities

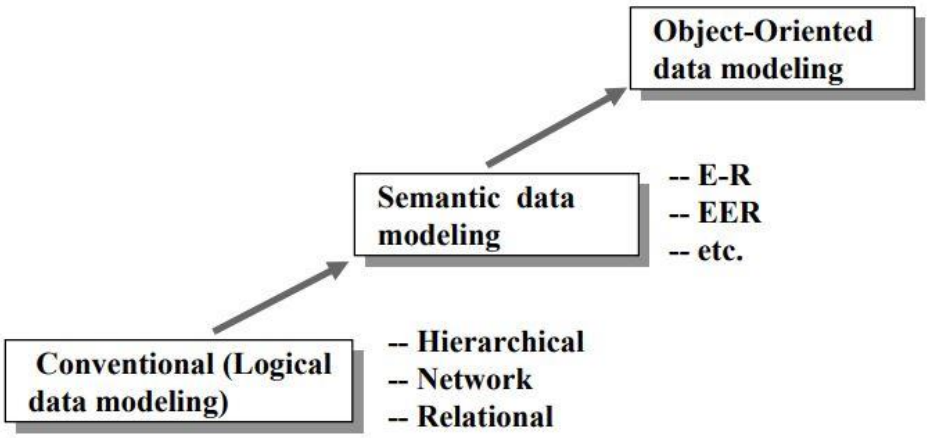
Dynamic Information

- ✓ Activities -- Operations/transactions
- ✓ Integrity constraints -- Business rules/regulations and data meanings

Conceptual Data Model Revisited

A conceptual data model consists of:

- ✓ A collection of formal concepts
- ✓ A set of usage rules

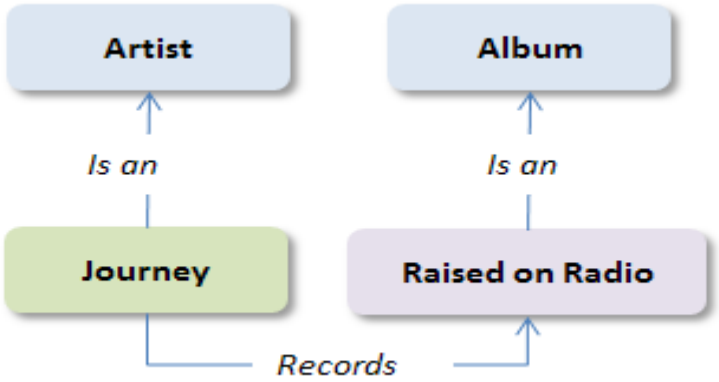


Semantic Data Model

Imagine that you are developing the next-generation music app, and need to create a robust database and application to store and work with data about topics such as artists, albums, and songs. Before you can ever start plugging data into a database, you'll need a model that both you and your non-techie business partners can understand. If the non-technical folks can understand the model, they may even have ideas of how best to implement the data. They can ask

questions and provide feedback. They may even notice connections or relationships that are redundant or unnecessary

In order to show the relationships between all parts of the music database, we can create a **semantic data model**, which is a conceptual diagram of the data as it relates to the real world. Before we get into the model, let's look at a simple relationship between an artist and an album. A given artist has a relationship to an album because they record the album. This can be expressed as follows:

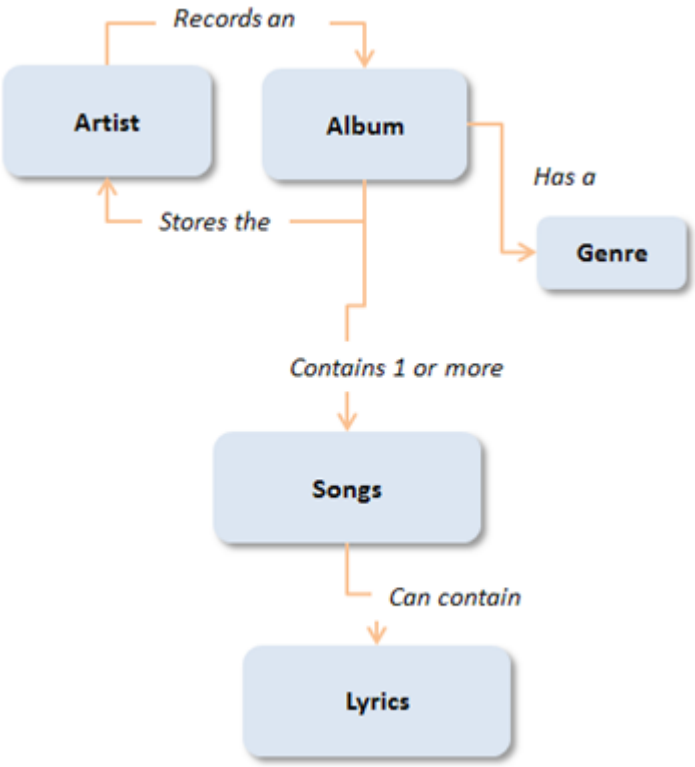


There is a relationship between the artist and the album. Journey is an artist; an artist records an album; *Raised on Radio* is an album. As we will see in the semantic, we will use these simple terms to clearly state what is being defined in the model. Here are some of the key phrases you will see in a semantic model (key terms are capitalized)

- ✓ Journey IS AN Artist
- ✓ An Artist RECORDS an Album
- ✓ An Album CONTAINS songs
- ✓ A song MAY CONTAIN lyrics

✓ Raised *on Radio* is an INSTANCE OF an Album (or a TYPE OF)

Now let's look at a more robust model. This type of model can be shown to non-technical leaders.



In the model, we see that an artist RECORDS an album, an album STORES the artist info, an album HAS A genre, an album CONTAINS one or more songs, and a song CAN CONTAIN lyrics.

Relational Model

The relational model is the conceptual basis of relational databases. Proposed by E.F. Codd in 1969, it is a method of structuring data using relations, which are grid-like mathematical structures consisting of columns and rows. Codd proposed the relational model for IBM, but he had no idea how extremely vital and influential his work would become as the basis of relational databases.

Most of us are very familiar with the physical manifestation of a relation in a database it is called table. Although the relational model borrows heavily from mathematics and uses mathematical terms such as domains, unions and ranges, the features and conditions it describes are easy to define using simple English.

In the relational model, all data must be stored in relations (tables), and each relation consists of rows and columns. Each relation must have a header and body. The header is simply the list of columns in the relation. The body is the set of data that actually populates the relation, organized into rows. You can extrapolate that the junction of one column and one row will result in a unique value - this value is called a tuple.

The second major characteristic of the relational model is the usage of keys. These are specially designated columns within a relation, used to order data or relate data to other relations. One of the most important keys is the primary key, which is used to uniquely identify each row of data. To make querying for data easier, most relational databases go further and physically order the data by the primary key. Foreign keys relate data in one relation to the primary key of another relation.

Besides defining how the data are to be structured as discussed above, the relational model also lays down a set of rules to enforce data integrity, known as integrity constraints. It also defines how the data are to be manipulated (relational calculus). In addition, the model defines a special feature termed normalization to ensure efficient data storage.

A relational database examples

Here's a simple example of two tables a small business might use to process orders for its products. The first table is a customer info table, so each record includes a customer's name, address, shipping and billing information, phone number, and other contact information. Each bit of information (each attribute) is in its own column, and the database assigns a unique ID (a key) to each row. In the second table—a customer order table—each record includes the ID of the customer that placed the order, the product ordered, the quantity, the selected size and color, and so on—but not the customer's name or contact information.

These two tables have only one thing in common: the ID column (the key). But because of that common column, the relational database can create a relationship between the two tables. Then, when the company's order processing application submits an order to the database, the database can go to the customer order table, pull the correct information about the product order, and use the customer ID from that table to look up the customer's billing and shipping information in the customer info table. The warehouse can then pull the correct product, the customer can receive timely delivery of the order, and the company can get paid.

Benefits of relational database management system

The simple yet powerful relational model is used by organizations of all types and sizes for a broad variety of information needs. Relational databases are used to track inventories, process ecommerce transactions, manage huge amounts of mission-critical customer information, and much more. A relational database can be considered for any information need in which data points relate to each other and must be managed in a secure, rules-based, consistent way.

Relational databases have been around since the 1970s. Today, the advantages of the relational model continue to make it the most widely accepted model for databases.

Relational model and data consistency

The relational model is the best at maintaining data consistency across applications and database copies (called instances). For example, when a customer deposits money at an ATM and then looks at the account balance on a mobile phone, the customer expects to see that deposit reflected immediately in an updated account balance. Relational databases excel at this kind of data consistency, ensuring that multiple instances of a database have the same data all the time.

It's difficult for other types of databases to maintain this level of timely consistency with large amounts of data. Some recent databases, such as NoSQL, can supply only "eventual consistency." Under this principle, when the database is scaled or when multiple users access the same data at the same time, the data needs some time to "catch up." Eventual consistency is acceptable for some uses, such as to maintain listings in a product catalog, but for critical business operations such as shopping cart transactions, the relational database is still the gold standard.

Commitment and atomicity

Relational databases handle business rules and policies at a very granular level, with strict policies about commitment (that is, making a change to the database permanent). For example, consider an inventory database that tracks three parts that are always used together. When one part is pulled from inventory, the other two must also be pulled. If one of the three parts isn't available, none of the parts should be pulled—all three parts must be available before the database makes any commitment. A relational database won't commit for one part until it knows it can commit for all three. This multifaceted commitment capability is called atomicity. Atomicity is the key to keeping data accurate in the database and ensuring that it is compliant with the rules, regulations, and policies of the business.

ACID properties and RDBMS

Four crucial properties define relational database transactions: atomicity, consistency, isolation, and durability—typically referred to as ACID.

Atomicity defines all the elements that make up a complete database transaction.

Consistency defines the rules for maintaining data points in a correct state after a transaction.

Isolation keeps the effect of a transaction invisible to others until it is committed, to avoid confusion.

Durability ensures that data changes become permanent once the transaction is committed.

Stored procedures and relational databases

Data access involves many repetitive actions. For example, a simple query to get information from a data table may need to be repeated hundreds or thousands of times to produce the desired result. These data access functions require some type of code to access the database. Application developers don't want to write new code for these functions in each new application. Luckily, relational databases allow stored procedures, which are blocks of code that can be accessed with a simple application call. For example, a single stored procedure can provide consistent record tagging for users of multiple applications. Stored procedures can also help developers ensure that certain data functions in the application are implemented in a specific way.

Database locking and concurrency

Conflicts can arise in a database when multiple users or applications attempt to change the same data at the same time. Locking and concurrency techniques reduce the potential for conflicts while maintaining the integrity of the data.

Locking prevents other users and applications from accessing data while it is being updated. In some databases, locking applies to the entire table, which creates a negative impact on application performance. Other databases, such as Oracle relational databases, apply locks at the record level, leaving the other records within the table available, helping ensure better application performance.

Concurrency manages the activity when multiple users or applications invoke queries at the same time on the same database. This capability provides the right access to users and applications according to policies defined for data control.

What to look for when selecting a relational database

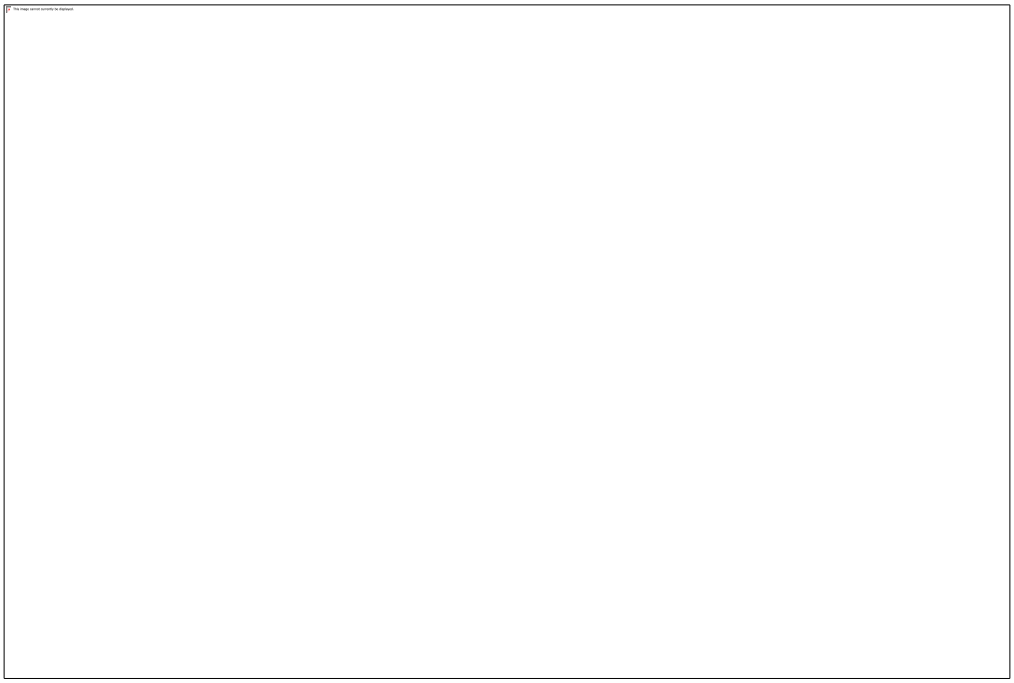
The software used to store, manage, query, and retrieve data stored in a relational database is called a relational database management system (RDBMS). The RDBMS provides an interface between users and applications and the database, as well as administrative functions for managing data storage, access, and performance.

Several factors can guide your decision when choosing among database types and relational database products. The RDBMS you choose will depend on your business needs. Ask yourself the following questions:

- ✓ What are our data accuracy requirements? Will data storage and accuracy rely on business logic? Does our data have stringent requirements for accuracy (for example, financial data and government reports)?
- ✓ Do we need scalability? What is the scale of the data to be managed, and what is its anticipated growth? Will the database model need to support mirrored database copies (as separate instances) for scalability? If so, can it maintain data consistency across those instances?
- ✓ How important is concurrency? Will multiple users and applications need simultaneous data access? Does the database software support concurrency while protecting the data?
- ✓ What are our performance and reliability needs? Do we need a high-performance, high-reliability product? What are the requirements for query-response performance? What are the vendor's commitments for service level agreements (SLAs) or unplanned downtime?

A Relational Database system has multiple other advantages over any other type of database.

Below are the few significant advantages,



1. Simple Model

A Relational Database system is the simplest model, as it does not require any complex structuring or querying processes. It doesn't involve tedious architectural processes like hierarchical database structuring or definition. As the structure is simple, it is sufficient to be handled with simple SQL queries and does not require complex queries to be designed.

2. Data Accuracy

In the relational database system, there can be multiple tables related to one another with the use of a primary key and foreign key concepts. This makes the data to be non-repetitive. There is no chance for duplication of data. Hence the accuracy of data in the relational database is more than any other database system.

3. Easy Access to Data

In the Relational Database System, there is no pattern or pathway for accessing the data, as to another type of databases can be accessed only by navigating through a tree or a hierarchical model. Anyone who accesses the data can query any table in the relational database. Using join queries and conditional statements one can combine all or any number of related tables in order to fetch the required data. Resulting data can be modified based on the values from any column, on any number of columns, which permits the user to effortlessly recover the relevant data as the result. It allows one to pick on the desired columns to be incorporated in the outcome so that only appropriate data will be displayed.

4. Data Integrity

Data integrity is a crucial characteristic of the Relational Database system. Sturdy Data entries and legitimacy validations ensure that all the Data in the database confines within suitable arrangements and the data necessary for creating the relationships are present. This relational reliability amongst the tables in the database helps in avoiding the records from being imperfect, isolated or unrelated. Data integrity aids in making sure of the relational database's other significant characteristics like Ease of use, precision, and stability of the data.

5. Flexibility

A Relational Database system by itself possesses qualities for leveling up, expanding for bigger lengths, as it is endowed with a bendable structure to accommodate the constantly shifting requirements. This facilitates the increasing incoming amount of data, as well as the update and deletes wherever required. This model consents to the changes made to a database configuration as well, which can be applied without difficulty devoid of crashing the data or the other parts of the database.

A Data Analyst can insert, update or delete tables, columns or individual data in the given database system promptly and easily, in order to meet the business needs. There is supposedly no boundary on the number of rows, columns or tables a relational database can hold. In any practical application, development and

transformation are restricted by the Relational Database Management System and the hardware contained by the servers. So, these changes can create an alteration in other peripheral functional devices connected to the particular relational database system.

6. Normalization

The methodical style is maintained for making sure of a relational database structure is liberated of any variances that can make a difference in the integrity and accuracy of the tables in the database. A normalization process provides a set of regulations, characteristics, and purposes for the database structure and evaluation of a relational database model.

Normalization aims at illustrating multiple levels of breaking down the data. Any level of normalization is expected to be accomplished on the same level, that is, before moving ahead to the next levels. A relational database model is usually confirmed to be normalized, only when it satisfies the necessary conditions of the third normalization form. Normalization offers an impression of reassurance on the database plan, to be extra strong and reliable.

7. High Security

As the data is divided amongst the tables of the relational database system, it is possible to make a few tables to be tagged as confidential and others not. This segregation is easily implemented with a relational database management system, unlike other databases. When a data analyst tries to login with a username and password, the database can set boundaries for their level of access, by providing admission only to the tables that they are allowed to work on, depending on their access level.

8. Feasible for Future Modifications

As the relational database system holds records in separate tables based on their categories, it is straightforward to insert, delete or update records that are subjected to the latest requirements. This feature of the relational database model tolerates the newest requirements that are presented by the business. Any number of new or existing tables or columns of data can be inserted or modified

depending on the conditions provided, by keeping up with the basic qualities of the relational database management system.

EXERCISE No. 03

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1. The term 'semantic' means?
A). Data
B). Meaning
C). Attribute
D). Detailed

2. **Which type of attribute is composed of other attributes?**
A). Simple
B). Meta
C). Group
D). Compound

3. **Which of the following is not true of semantic objects?**
A). Always represent physical entities
B). Always provide a sufficient description
C). Are always named
D). Always describe a distinct identity

4. **A semantic object?**
A). Is a representation of some identifiable thing in the users' work environment
B). Is a characteristic of an attribute?
C). Is one or more object attributes that the users employ to identify object instances
D). Is description of an attribute's possible values?

5. **A semantic object that contains one or more multi-value, simple or group attributes but no other attributes is known as?**
A). Simple objects
B). Composite objects
C). Compound objects
D). Hybrid objects

6. **Combinations of composite and compound objects are known as?**
A). Hybrid objects

B). Association objects
C). Subtype objects
D). Arch type objects

7. **A semantic object that contains only single-value, simple or group attributes is called?**
A). Simple objects
B). Composite objects
C). Compound objects
D). Hybrid objects

8. **Which of the following are used to represent the specialization of objects?**
A). Hybrid objects
B). Association objects
C). Subtype objects
D). Archtype objects

9. **The columns of a table correspond to _____?**
A). Table
B). Record
C). Field
D). Cell

10. **A relation is also known as _____?**
A). Table
B). Tuple
C). Relationship
D). Attribute

11. **An attribute is also known as a _____?**
A). Table
B). Relation
C). Row
D). Field

12. **Which of the following is NOT a general characteristic of relations _____?**

A). Each row is unique
B). The order of columns is significant
C). The order of rows is insignificant
D). Columns are all elemental or atomic

13. Which of the following are properties of relations _____?

A). Each attribute has a unique name
B). No two rows in a relation are identical
C). There are no multivalued attributes in a relation
D). All of the above

14. How are the E-R model and the semantic object model similar?

A). Neither strives to model the structure of the things in the users' world
B). Both see the concept of entity as basic
C). They both see the semantic object as basic
D). Both are tools for understanding and documenting the structure of - the users' data

15. How many categories of data models in DBMS?

A. 3
B. 5
C. 4
D. none of these

ANSWER KEY

1. B	2. B	3. A	4. A	5. B
6. A	7. A	8. C	9. C	10. A
11. D	12. B	13. D	14. D	15. A

PART-II SAMPLE SHORT QUESTIONS

1. What is logical view of data?
2. What is database Model?
3. Semantic Data Model?
4. Relational Model?
5. Data base Model & internet?
6. Write the name of advantages of Database Model.
7. What are our data accuracy requirements?
8. Do we need scalability?
9. How important is concurrency? Will multiple users and applications need simultaneous data access?
10. What are our performance and reliability needs?

PART-III SAMPLE LONG QUESTIONS

1. Write the detail Note on Semantic Data Model
2. Write the detail note on Relational Model
3. Write the detail Note database Models

4 Chapter No. 4 (Relational Database Management)

Objectives

After completion of this chapter students will be able to understand:

- ✓ A logical view of Data; Entities and Attributes
- ✓ Tables and their Characteristics, Keys
- ✓ Integrity rules
- ✓ Entity and referential integrity
- ✓ Relational Database operators

Relational Database Management

A LOGICAL VIEW OF DATA • The logical database view **is** how the data appear to the user to be stored. This view represents the structure that the user must interface with in order to extract data from the database. The relational model enables you to view data logically rather than physically. Logical simplicity tends to yield simple and effective database design methodologies. Because the table plays such a prominent role in the relational model.

4.1.1 Entity

An entity is a person, place, event, or thing for which we intend to collect data. University -- Students, Faculty Members, Courses. An entity can be a real-world object, either animate or inanimate, that can be easily identifiable. For example, in a school database, students, teachers, classes, and courses offered can be considered as entities. All these entities have some attributes or properties that give them their identity.

An entity set is a collection of similar types of entities. An entity set may contain entities with attribute sharing similar values. For example, a student's set may contain all the students of a school; likewise, a teachers set may contain all the teachers of a school from all faculties. Entity sets need not be disjoint.

Entity Set: A grouping of related entities becomes an entity set. The STUDENT entity set contains all student entities. The FACULTY entity set contains all faculty entities.

4.1.2 Attributes

Each entity has certain characteristics known as attributes. Student, Number, Name, GPA, Date of Enrollment, Data of Birth, Home Address, Phone Number, Major. Entities are represented by means of their properties, called attributes. All attributes have values. For example, a student entity may have name, class, and age as attributes. There exists a domain or range of values that can be assigned to attributes. For example, a student's name cannot be a numeric value. It has to be alphabetic. A student's age cannot be negative, etc.

4.1.3 Types of Attributes

- ✓ **Simple attribute** − Simple attributes are atomic values, which cannot be divided further. For example, a student's phone number is an atomic value of 10 digits.
- ✓ **Composite attribute** − Composite attributes are made of more than one simple attribute. For example, a student's complete name may have first_name and last_name.
- ✓ Derived attribute − Derived attributes are the attributes that do not exist in the physical database, but their values are derived from other attributes present in the database. For example, average_salary in a department should not be saved directly in the database, instead it can be derived. For another example, age can be derived from data_of_birth.
- ✓ **Single-value attribute** − Single-value attributes contain single value. For example − Social_Security_Number.
- ✓ **Multi-value attribute** − multi-value attributes may contain more than one values. For example, a person can have more than one phone number, email_address, etc.

These attribute types can come together in a way like −

- ✓ simple single-valued attributes
- ✓ simple multi-valued attributes
- ✓ composite single-valued attributes
- ✓ composite multi-valued attributes

What is a Database Table?

A table is a collection of related data entries, and it consists of columns and rows. A column holds specific information about every record in the table. A record (or row) is each individual entry that exists in a table.

4.1.4 What is Relational database Management System?

A relational database management system (RDBMS or just RDB) is a common type of database that stores data in tables, so it can be used in relation to other stored datasets. Most databases used by businesses these days are relational databases, as opposed to a flat file or hierarchical database. The majority of current IT systems and applications are based on a relational DBMS.

A relational database management system (RDBMS) is a program that allows you to create, update, and administer a relational database. Most relational database management systems use the SQL language to access the database.

What is a Relational Database?

A relational database defines database relationships in the form of tables. The tables are related to each other - based on data common to each. Look at the following three tables' "Customers", "Orders", and "Shippers" from the Northwind database:

4.1.5 Customers Table

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

The relationship between the "Customers" table and the "Orders" table is the Customer column:

4.1.6 Orders Table

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10278	5	8	1996-08-12	2
10280	5	2	1996-08-14	1
10308	2	7	1996-09-18	3
10355	4	6	1996-11-15	1
10365	3	3	1996-11-27	2
10383	4	8	1996-12-16	3
10384	5	3	1996-12-16	3

The relationship between the "Orders" table and the "Shippers" table is the ShipperID column:

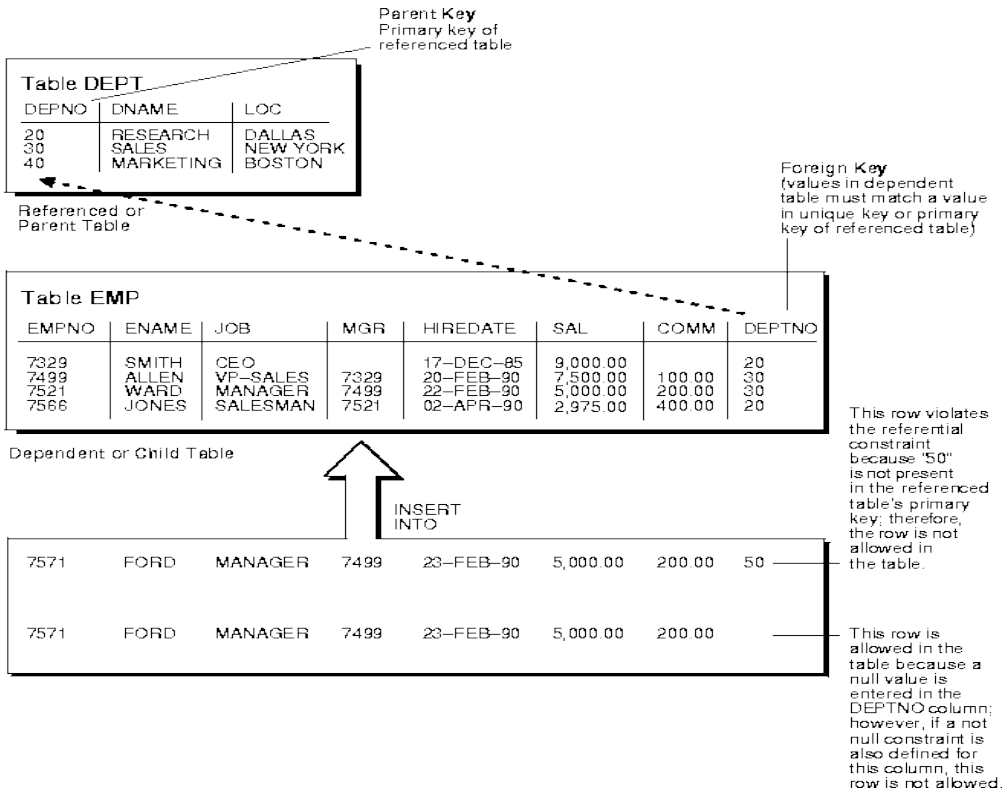
4.1.7 Shippers Table

ShipperID	ShipperName	Phone
1	Speedy Express	(503) 555-9831
2	United Package	(503) 555-3199
3	Federal Shipping	(503) 555-9931

DATA INTEGRITY FOR DATABASES

In the broad sense, data integrity is a term to understand the health and maintenance of any digital information. For many, the term is related to database management. For databases, there are four types of data integrity.

- **Entity Integrity:** In a database, there are columns, rows, and tables. In a primary key, these elements are to be as numerous as needed for the data to be accurate, yet no more than necessary. None of these elements should be the same and none of these elements should be null. For example, a database of employees should have primary key data of their name and a specific "employee number."
- **Referential Integrity:** A foreign key in a database is a second table that can refer to a primary key table within the database. Foreign keys relate data that could be shared or null. For instance, employees could share the same role or work in the same department.
- **Domain Integrity:** All categories and values in a database are set, including nulls (e.g., N/A). The domain integrity of a database refers to the common ways to input and read this data. For instance, if a database uses monetary values to include dollars and cents, three decimal places will not be allowed.
- **User-Defined Integrity**: There are sets of data, created by users, outside of entity, **referential** and domain integrity. If an employer creates a column to input corrective action of employees, this data would be classified as "user-defined."



DATA INTEGRITY VS. DATA SECURITY

Data integrity and data security are related terms, each playing an important role in the successful achievement of the other. Data security refers to the protection of data against unauthorized access or corruption and is necessary to ensure data integrity.

That said, data integrity is a desired result of data security, but the term data integrity refers only to the validity and accuracy of data rather than the act of protecting data. Data security, in other words, is one of several measures which can be employed to maintain data integrity. Whether it's a case of malicious intent or accidental compromise, data security plays an important role in maintaining data integrity.

For modern enterprises, data integrity is essential for the accuracy and efficiency of business processes as well as decision making. It’s also a central focus of many

data security programs. Achieved through a variety of data protection methods, including backup and replication, database integrity constraints, validation processes, and other systems and protocols, data integrity is critical yet manageable for organizations today.

What is Data Redundancy?

Data redundancy occurs when the same piece of data is stored in two or more separate places and is a common occurrence in many businesses. As more companies are moving away from siloes data to using a central repository to store information, they are finding that their database is filled with inconsistent duplicates of the same entry. Although it can be challenging to reconcile — or even benefit from — duplicate data entries, understanding how to reduce and track data redundancy efficiently can help mitigate long-term inconsistency issues for your business.

4.1.8 How does data redundancy occur?

Sometimes data redundancy happens by accident while other times it is intentional. Accidental data redundancy can be the result of a complex process or inefficient coding while intentional data redundancy can be used to protect data and ensure redundancy simply by leveraging the multiple occurrences of data for disaster recovery and quality checks.

If data redundancy is intentional, it's important to have a central field or space for the data. This allows you to easily update all records of redundant data when necessary. When data redundancy isn't purposeful, it can lead to a variety of issues which we'll discuss below.

4.1.9 Understanding database versus file-based data redundancy

Data redundancy can be found in a database, which is an organized collection of structured data that's stored by a computer system or the cloud. A retailer may have a database to track the products they stock. If the same product gets entered twice by mistake, data redundancy takes place.

The same retailer may keep customer files in a file storage system. If a customer purchases from the company more than once, their name may be entered multiple times. Duplicate entries of the customer's name are considered redundant data.

Regardless of whether data redundancy occurs in a database or in a file storage system, it can be problematic. Fortunately, data replication can help prevent data redundancy by storing the same data in multiple locations. With data replication, companies can ensure consistency and receive the information they need at any time.

4.1.10 Top 4 advantages of data redundancy

Although data redundancy sounds like a negative event, there are many organizations that can benefit from this process when it's intentionally built into daily operations.

1. Alternative data backup method

Backing up data involves creating compressed and encrypted versions of data and storing it in a computer system or the cloud. Data redundancy offers an extra layer of protection and reinforces the backup by replicating data to an additional system. It's often an advantage when companies incorporate data redundancy into their disaster recovery plans.

2. Better data security

Data security relates to protecting data, in a database or a file storage system, from unwanted activities such as cyber-attacks or data breaches. Having the same data stored in two or more separate places can protect an organization in the event of a cyberattack or breach — an event which can result in lost time and money, as well as a damaged reputation.

3. Faster data access and updates

When data is redundant, employees enjoy fast access and quick updates because the necessary information is available on multiple systems. This is particularly important for customer service-based organizations whose customers expect promptness and efficiency.

4. Improved data reliability

Data that is reliable is complete and accurate. Organizations can use data redundancy to double check data and confirm it's correct and completed in full — a necessity when interacting with customers, vendors, internal staff, and others. Watch out for data redundancy disadvantages

Although there are noteworthy advantages of intentional data redundancy, there are also several significant drawbacks when organizations are unaware of its presence.

Possible data inconsistency

Data redundancy occurs when the same piece of data exists in multiple places, whereas data inconsistency is when the same data exists in different formats in multiple tables. Unfortunately, data redundancy can cause data inconsistency, which can provide a company with unreliable and/or meaningless information.

Increase in data corruption

Data corruption is when data becomes damaged as a result of errors in writing, reading, storage, or processing. When the same data fields are repeated in a database or file storage system, data corruption arises. If a file gets corrupted, for example, and an employee tries to open it, they may get an error message and not be able to complete their task.

Increase in database size

Data redundancy may increase the size and complexity of a database — making it more of a challenge to maintain. A larger database can also lead to longer load times and a great deal of headaches and frustrations for employees as they'll need to spend more time completing daily tasks.

Increase in cost

When more data is created due to data redundancy, storage costs suddenly increase. This can be a serious issue for organizations who are trying to keep costs low in order to increase profits and meet their goals. In addition, implementing a database system can become more expensive.

How to reduce data redundancy?

Fortunately, it is possible to reduce unintentional cases of data redundancy that often lead to operational and financial problems.

Master data

Master data is a single source of common business data that is shared across several applications or systems. Although master data does not reduce the occurrences of data redundancy, it allows companies to work around and accept a certain level of data redundancy. This is because the use of master data ensures that in the event a data piece changes, an organization only needs to update one piece of data. In this case, redundant data is consistently updated and provides the same information.

In a relational model, data is stored in relations. Relation is another term used for table

Table\Relation:

A table in a database has a unique name that identifies its contents. Each table can be called an intersection of rows and columns. An important property of a table is that the rows are unordered. A row cannot be identified by its position in

the table. Every table must have a column that uniquely identifies each row in the table.

Advantages of Relational Database

A Relational Database system has multiple other advantages over any other type of database. Below are the few significant advantages.

1. Simple Model

A Relational Database system is the simplest model, as it does not require any complex structuring or querying processes. It doesn't involve tedious architectural processes like hierarchical database structuring or definition. As the structure is simple, it is sufficient to be handled with simple SQL queries and does not require complex queries to be designed.

2. Data Accuracy

In the relational database system, there can be multiple tables related to one another with the use of a primary key and foreign key concepts. This makes the data to be non-repetitive. There is no chance for duplication of data. Hence the accuracy of data in the relational database is more than any other database system.

3. Easy Access to Data

In the Relational Database System, there is no pattern or pathway for accessing the data, as to another type of databases can be accessed only by navigating through a tree or a hierarchical model. Anyone who accesses the data can query any table in the relational database. Using join queries and conditional statements one can combine all or any number of related tables in order to fetch the required data. Resulting data can be modified based on the values from any column, on any number of columns, which permits the user to effortlessly recover the

relevant data as the result. It allows one to pick on the desired columns to be incorporated in the outcome so that only appropriate data will be displayed.

4. Data Integrity

Data integrity is a crucial characteristic of the Relational Database system. Sturdy Data entries and legitimacy validations ensure that all the Data in the database confines within suitable arrangements and the data necessary for creating the relationships are present. This relational reliability amongst the tables in the database helps in avoiding the records from being imperfect, isolated or unrelated. Data integrity aids in making sure of the relational database's other significant characteristics like Ease of use, precision, and stability of the data.

5. Flexibility

A Relational Database system by itself possesses qualities for leveling up, expanding for bigger lengths, as it is endowed with a bendable structure to accommodate the constantly shifting requirements. This facilitates the increasing incoming amount of data, as well as the update and deletes wherever required. This model consents to the changes made to a database configuration as well, which can be applied without difficulty devoid of crashing the data or the other parts of the database.

A data analyst can insert, update or delete tables, columns or individual data in the given database system promptly and easily, in order to meet the business needs. There is supposedly no boundary on the number of rows, columns or tables a relational database can hold. In any practical application, development and transformation are restricted by the Relational Database Management System and the hardware contained by the servers. So these changes can create an alteration in other peripheral functional devices connected to the particular relational database system.

6. Normalization

The methodical style is maintained for making sure of a relational database structure is liberated of any variances that can make a difference in the integrity and accuracy of the tables in the database. A normalization process provides a set of regulations, characteristics, and purposes for the database structure and evaluation of a relational database model.

Normalization aims at illustrating multiple levels of breaking down the data. Any level of normalization is expected to be accomplished on the same level, that is, before moving ahead to the next levels. A relational database model is usually confirmed to be normalized, only when it satisfies the necessary conditions of the third normalization form. Normalization offers an impression of reassurance on the database plan, to be extra strong and reliable.

7. High Security

As the data is divided amongst the tables of the relational database system, it is possible to make a few tables to be tagged as confidential and others not. This segregation is easily implemented with a relational database management system, unlike other databases. When a data analyst tries to login with a username and password, the database can set boundaries for their level of access, by providing admission only to the tables that they are allowed to work on, depending on their access level.

8. Feasible for Future Modifications

As the relational database system holds records in separate tables based on their categories, it is straightforward to insert, delete or update records that are subjected to the latest requirements. This feature of the relational database model tolerates the newest requirements that are presented by the business. Any number of new or existing tables or columns of data can be inserted or modified depending on the conditions provided, by keeping up with the basic qualities of the relational database management system.

Conclusion

To sum up all the advantages of using the relational database over any other type of database, a relational database helps in maintaining the data integrity, data accuracy, reduces data redundancy to minimum or zero, data scalability, data flexibility and facilitates makes it easy to implement security methods. Above all, a Relational Database Management system is a simpler database model, both to design and implement.

Relational Database Management Terminologies

Relation

Ina relational model, data is stored in relations. Relation is another term used for table.

Properties of a Table

- ✓ A table has a name that is distinct from all other tables in the database.
- ✓ There are no duplicate rows; each row is distinct.
- ✓ Entries in columns are atomic. The table does not contain repeating groups or multivalued attributes.
- ✓ Entries from columns are from the same domain based on their data type including:
- ✓ number (numeric, integer, float, small-int,…)
- ✓ character (string)
- ✓ date(date)
- ✓ logical (true or false)
- ✓ Operations combining different data types are disallowed.
- ✓ Each attribute has a distinct name.

Following is an example of a relation.

Registration_no	Name	Class
2k19-101	Nadeem Khalil	DAE-CIT
2k19-102	Ejjaz Saeed	DAE-CIT
2k19-103	Adnan Ali	DAE-CIT

Figure: Table of student

4.1.11 Tuples

In a relational model, every relation or table consists of many tuples. Tuples are also called records or rows.

2k19-101	Nadeem Khalil	DAE-CIT

2k19-103	Adnan Ali	DAE-CIT

Records contain fields that are related, such as a customer or an employee. As noted earlier, a tuple is another term used for record.

Records and fields form the basis of all databases. A simple table gives us the clearest picture of how records and fields work together in a database storage project.

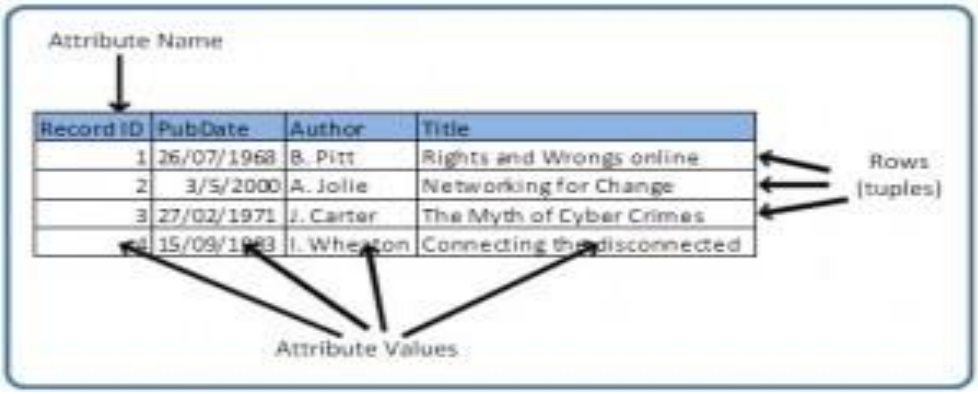


Figure. Example of a simple table by A. Watt.

The simple table example in Figure shows us how fields can hold a range of different sorts of data. This one has:

4.1.12 Attributes

An attribute is a named column of a relation. Attributes are also called characteristics. The characteristics of the tuple are represented by attributes or fields.

DAE-CIT
Ejjaz Saeed
Adnan Ali

4.1.13 Domain

A domain is a collection of all possible Values of one or more attributes. For example, the value in the field "Class can be the name of any taught classes. It is known as class Domain. Similarly, Registration domain is a collection of all possible Registration numbers. A *domain* is the original sets of atomic values used to model data. By *atomic value*, we mean that each value in the domain is indivisible as far as the relational model is concerned. For example:

- ✓ The domain of Marital Status has a set of possibilities: Married, Single, Divorced.
- ✓ The domain of Shift has the set of all possible days: {Mon, Tue, Wed…}.
- ✓ The domain of Salary is the set of all floating-point numbers greater than 0 and less than 200,000.
- ✓ The domain of First Name is the set of character strings that represents names of people.

In summary, a domain is a set of acceptable values that a column is allowed to contain. This is based on various properties and the data type for the column.

4.1.14Degree

The degree of a relation is the number of attributes (columns) in the given table.

STUDENT			
RegNo	**SName**	**Gen**	**Phone**
R1	Adnan	M	9898786756
R3	Imran	M	8798987867
R4	usman	M	7898886756
R2	Rehan	M	9897786776

For the STUDENT table given above, the degree is 4. That is there are 4 attributes in the STUDENT table.

Advantages of a Relational Database Model

Some important advantages of a relational database model are as follows:

Data Integrity

Relational model allows data integrity from field level to table level to avoid Duplication of records. It detects records with missing primary key values at the relationship level to ensure valid relationships between relations.

Data Independence

The implementation of database will not be affected by changes made in the logical design of the database or changes made in the database software.

Structural Independence

Structural independence exists when the structure of database can be changed without Affecting DBMSS ability to access the data. The relational database model does not use a Navigational data access system. The data access paths are irrelevant to relational database designer programmer and the end user. Any change in relational database structure does not affect data access in any way.it make relational database structure independence.

Keys

A key is an attribute or set of attributes that uniquely identifies a tuple in a relation. The keys are defined in tables to access or sequence the stored data quickly and smoothly

They are also used to create relationship between different tables.

4.1.15 Introduction to Database Keys

Keys are very important part of Relational database model. They are used to establish and identify relationships between tables and also to uniquely identify any record or row of data inside a table. A Key can be a single attribute or a group of attributes, where the combination may act as a key.

4.1.16Super Key

A super key is an attribute or combination of attributes in a relation that identifies a tuple uniquely within the relation. A super key is the most general type of key. For example, a relation STUDENT consists of different attributes like RegistrationNo, Name, FatherName, Class and Address. The only attribute that can uniquely identify a tuple in the relation is RegistrationNo. The Name attribute cannot identify a tuple because two or more students may have the same name. Similarly, Father Name, Class and Address cannot be used to identify a tuple. It means that RegistrationNo is the super key for the relation

Any combination of attributes with the super key is also a super key. It means attributes or set of attributes combined with the super key RegistrationNo will also become Super key. A combination of two attributes (RegistrationNo, Name) is also a super key combination can also be used to identify a tuple in the relation. Similarly, [Registratio Class) or (RegistrationNo, Name, Class) are also super keys

4.1.17Candidate Key

A candidate key is a super key that contains no extra attribute. It consists of minimum possible attributes. A super key like RegistrationNo, Name contains an extra field Name. I can be used to identify a tuple uniquely in the relation. But it does not consist of minimum possible attribute as only Registration No can be used to identify a tuple in relation. It means that (RegistrationNo, Name) is a super key but it is not a candidate key because it contains an extra field. On the other hand, RegistrationNo is a super key as well as a candidate key.

4.1.18Primary Key

A primary key is a candidate key that is selected by the database designer to identify tuples uniquely in a relation. A relation may contain many candidate keys, When the designer selects one of them to identify a tuple in the relation, it becomes a primary key. It means that if there is only one Candidate key, it will be automatically selected as primary key. some most important points about a primary key are:

- ✓ A relation can have only one primary key.
- ✓ Each value in primary key attribute must be unique.
- ✓ Primary key cannot contain null values..

Suppose a relation Student contains different attributes such as RegNo, Name and Class. The attribute RegNo uniquely identifies each student in the table. It can be used as Primary key for this table. The attribute Name cannot uniquely identify each row because Two students can have same name. It cannot be used as primary key.

Registration_no	Name	Class
2k19-101	Nadeem Khalil	DAE-CIT
2k19-102	Ejjaz Saeed	DAE-CIT
2k19-103	Adnan Ali	DAE-CIT

Figure: Primary Key of student table

In above table Registration_no is the primary key that is uniquely identifies each record.

4.1.19Alternate Key

The candidate keys that are not selected as primary key are known as alternate keys. Suppose Student relation contains different attributes such as RegNo, RollNo, Name and Class. The attributes RegNo and RollNo can be used to identify each student in the table. If RegNo is selected as primary key then RollNo attribute is known as alternate key.

RegNo	RollNo	Name	Class
2k19-101	101	Nadeem Khalil	DAE-CIT

2k19-102	102	Ejjaz Saeed	DAE-CIT
2k19-103	103	Adnan Ali	DAE-CIT

Figure: Alternate Key

4.1.20 Composite Key

A primary key that consists of two or more attributes is known as composite key. For Example, the following relation uses two fields RolNo and Subject to identify each tuple. This is an example of composite key.

RolNo	**Subject**	**Marks**
101	Computer	72
101	English	68
101	Math	52
102	Computer	75
102	English	60
102	Math	50

Figure: Marks Table with composite key

4.1.21Foreign Key

A foreign key is an attribute or set of attributes in a relation whose values match a primary key in another relation. The relation in which foreign key is created is

known as Dependent table or child table. The relation to which the foreign key refers is known as Parent table, the key connects to another relation when a relationship is established between two relations. A relation may contain many foreign keys.

The following figure shows two relations. The RolNo attribute in Parent relation is used as primary key. The RolNo attribute in Child relation is used as foreign key. It refers to RolNo attribute in Parent relation.

RolNo	Subject	Marks
101	Computer	72
101	English	68
101	Math	52
102	Computer	75
102	English	60
102	Math	50

Properties of Relations

Relations have several properties. These properties are as follows:

1. Atomic Values in Fields

An entry at the intersection of each row and column is atomic. There can be no multi-Valued attributes or repeating groups in a relation.

2. Entries from Same Domain

A domain is the type and range of values of attributes. In a relation, all entries in a given column belongs to the same domain, all entries in RegistrationNo attributes of relation must be from RegistrationNo domain.

3. Unique Tuples

Each tuple in a relation must be unique. For example, the RegistrationNo in each tuple of the table must be different from all other tuples. Uniqueness in a relation is guaranteed assigning a primary key for each relation.

4. Unique Attribute Name

The name of each attribute of relation should be unique. A relation cannot have two identical attributes.

5. Insignificant Attribute Sequence

The sequence of attributes in a elation is insignificant. This sequence can be charged Without changing the meaning or use of the relation.

6. Insignificant Tuple Sequence

The sequence of tuples in a relation is also insignificant. The sequence may be changed. If a new tuple is inserted in a relation, it is immaterial whether it is inserted at the beginning at the end or in the middle of the relation.

Relational Data Integrity

Data integrity means reliability and accuracy of data. Integrity rules are designed to keep the data consistent and correct. These rules act like a check on the incoming data. It is very important that a database maintains the quality of the data stored in it. DBMS provides several mechanisms to enforce integrity of the data in a column. Enforcing data integrity ensures the quality of data in the database. For example, if employee id is entered as 123". this value should not be entered again The ID should not assigned to two or more employees Similarly il grades of students can be from A to F, the database should not accept any other value.

Data integrity falls into following categories:

- ✓ Entity Integrity

- ✓ Domain Integrity
- ✓ Referential Integrity

4.1.22Entity Integrity

The entity integrity rule ensures that the primary key cannot contain null data. It is also called row integrity. If primary key is allowed to have null value, it is not possible to uniquely identify a tuple in relation. Entity integrity means that it should be easy to identify each entity in the database. An entity is anything like an object, subject or event represented in the database. For example, a database for patient tracking in a hospital may have the may have following entities:

- ✓ Patients
- ✓ Prescriptions
- ✓ Physicians
- ✓ Drugs
- ✓ Appointments

4.1.23 Domain Integrity

A set of values that can be stored in a column is called a domain. For example, the Marks of a student in a subject can be from 0 to 100. Domain integrity enforces restrictions on the values entered in a column. It specifies the validity of a specific data entry in a column. The data type of a column enforces domain integrity. For example, if the data type of Experience column is numeric; it cannot store a value like two".

4.1.24 Referential Integrity

Referential integrity preserves the defined relationship between tables when records are added or deleted. It ensures that key values are consistent across the tables.

Registration_no	Name	Class
2k19-101	Nadeem Khalil	DAE-CIT
2k19-102	Ejjaz Saeed	DAE-CIT
2k19-103	Adnan Ali	DAE-CIT

Table: Master Table

Registration_No	**Subject**	**Marks**
2k19-101	Computer	72
2k19-101	English	68
2k19-101	Math	52
2k19-102	Computer	75
2k19-102	English	60
2k19-102	Math	50

Table 1 child table

The above example has two tables Master and Child. A value cannot be entered in Child table before entering the corresponding value in Master table. I the user want to enter the result of a student with Registration_No "2k19-101, he has to enter the record in the Master table first. Then he can enter the details to that student in the Child table. Similarly, if a record is to be deleted from Master table, it is access to delete the corresponding records in Child table first.

Relational Set Operators

A. **Relational Set Operators:** uses relational algebra to manipulate contents in a database. All together there are eight different types of operators. These operators are SQL commands.

B. **SELECT** is the command to show all rows in a table. It can be used to select only specific data from the table that meets certain criteria. This command is also referred to as the Restrict command.

RESTRICTS the rows chosen from a table to those entries with specified attribute values.

```
SELECT item
FROM stock level
WHERE quantity > 100
```

constructs a new, logical table - an unnamed relation - with one column per row (i.e., item) containing all rows from stock level that satisfy the WHERE clause.

C. **UNION**. It combines all of the rows in one table with all of the rows in another table except for the duplicate tuples. The tables are required to have the same attribute characteristics for the Union command to work. The tables must be union-compatible which means that two tables being used have the same amount of columns and the columns have the same names, and also need to share the same domain.

D. **INTERSECT** is the second SQL command that takes two tables and combines only the rows that appear in both tables. The tables must be union-compatible to be able to use the Intersect command or else it won't work.

E. **DIFFERENCE** in another SQL command that gets all rows in one table that are not found in the other table. Basically, it subtracts one table from the other table to leave only the attributes that are not the same in both tables. For this command to work both tables must be union-compatible.

F. **PRODUCT** command would show all possible pairs of rows from both tables being used. This command can also be referred to as the Cartesian Product.

G. **PROJECT** is the command that gives all values for certain attributes specified after the command. It shows a vertical view of the given table.

Selects rows made up of a sub-set of columns from a table.

PROJECT stock_item

OVER item AND description

produces a new logical table where each row contains only two columns item and description. The new table will only contain distinct rows from stock item; i.e., any duplicate rows so formed will be eliminated.

H. **JOIN** takes two or more tables and combines them into one table. This can be used in combination with other commands to get specific information. There are several types of the Join command. The Natural Join, Equi jion, Theta Join etc. Associates' entries from two tables on the basis of matching column values.

JOIN stock_item

WITH stock_level

OVER item

It is not necessary for there to be a one-to-one relationship between entries in two tables to be joined - entries which do not match anything will be eliminated from the result, and entries from one table which match several entries in the other will be duplicated the required number of times.

The above definition is actually that of a NATURAL or EQUI-JOIN - i.e., a join in which the values of the matching columns are equal. It has become normal to extend join to include other comparison operators such as less than, greater than, etc. It is important to be clear about one's intentions here to obtain meaningful results. Join is obviously a very general operation, and the principal source of processing power in relational systems, but it is also costly in time and space. Because no ordering can be guaranteed, a join may require a comparison of every entry in one table with every entry in the other, and create large intermediate results. That is why users of large-scale data bases, while acknowledging the power and flexibility of the relational approach, were slow to adopt it instead of methods based on more efficient file processing techniques.

MULTIPLE CHOISE QUESTIONS

1) A table is a collection of relationships, there is a close correspondence between concept of :
   A. Table and instances.
   B. Table and variables.
   C. Tables and relations.
   D. Tables and entries.
2) Which of the following is generally used for performing tasks like creating the structure of the relations, deleting relation?
   A. DML(Data Manipulation Language)
   B. Query
   C. Relational Schema
   D. DDL(Data Definition Language)
3) For what purpose the DML is provided?
   A. Addition of new structure in the database
   B. Manipulation & processing of the database
   C. Definition of the physical structure of the database system
   D. All of the above
4) In the relation model, the relation are generally termed as _____
   A. Tuples
   B. Attributes
   C. Rows
   D. Tables
5) A column in a database in which customer names are stored would be referred to as a _____.
   A. field
   B. record
   C. table
6) In relational model, the row of table is known to be ?
   A. Relation
   B. Entity Field
   C. Tuple
   D. Attributes

7) A _____ in a table represents a relationship among a set of values.
   A. Column
   B. Key
   C. Row
   D. Entry
8) What is data integrity?
   A. It is the data contained in database that is non redundant.
   B. It is the data contained in database that is accurate and consistent.
   C. It is the data contained in database that is secured.
   D. It is the data contained in database that is shared
9) which one of the following refers to the "data about data"?
   A. Directory
   B. Sub Data
   C. Warehouse
   D. Meta Data.
10) In general, a file is basically a collection of all related_____.
   Rows & Columns
   A. Fields
   B. Database
   C. Records
11) Rows of a relation are known as the _____.
   A. Degree
   B. Tuples
   C. Cardinality
   D. All of the above
12) The term _____ is used to refer to a row.
   A. Attribute
   B. Tuple
   C. Field
   D. Instance
13) For each attribute of a relation, there is a set of permitted values, called the _____ of that attribute.
   A. Domain
   B. Relation
   C. Set
   D. Schema

ANSWER KEY

1.	C	2.	D	3.	B	4.	D	5.	A
6.	C	7.	C	8.	B	9.	D	10.	C
11.	C	12.	B	13	A				

A. PART-II SAMPLE SHORT QUESTIONS

1. Define Tables.?
2. Define Relational model.
3. Define Data Integrity.
4. Define Entity.
5. Define Attributes.
6. What is primary key?
7. What is composite key?
8. What is foreign key?
9. What is meant by atomic values of a field .
10. Define Domain of a relation.

B. PART-III SAMPLE LONG QUESTIONS

1. Define relational database and briefly explain the properties of relational model.
2. Briefly explain the advantages of relational model.
3. Explain any Five keys use in relational database management system.
4. Define 12 rules of Dr.E.F code for relational DBMS.
5. Explain the data integrity of relational model.

5 Chapter No. 5 (Normalization of Database Tables)

Objectives

After completion of this chapter students will be able to understand:

- ✓ Need for Normalization
- ✓ Conversion to First Normal Form
- ✓ Conversion to Second Normal Form
- ✓ Conversion to Third Normal Form
- ✓ Boyce-Codd Normal Form (BCNF)

Description of normalization

Normalization is the process of organizing data in a database. This includes creating tables and establishing relationships between those tables according to rules designed both to protect the data and to make the database more flexible by eliminating redundancy and inconsistent dependency.

Database Anomalies

Database anomalies are the problems in relations that occur due to redundancy in the relations. These anomalies affect the process of inserting deleting and modifying data in the relations. Some important data may be lost if a relation is updated that contains database Anomalies. It is important to remove these anomalies in order to perform different processing on the relations without any problem.

5.1.1 Types of Anomalies.

Different types of database anomalies are as follows.

There are different types of anomalies which can occur in referencing and referenced relation which can be discussed as:

STUDENT

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY	STUD_AGE
1	RAM	9716271721	Haryana	India	20
2	RAM	9898291281	Punjab	India	19
3	SUJIT	7898291981	Rajsthan	India	18
4	SURESH		Punjab	India	21

Table 1

STUDENT_COURSE

STUD_NO	COURSE_NO	COURSE_NAME
1	C1	DBMS
2	C2	Computer Networks
1	C2	Computer Networks

Table 2

5.1.1.1 Insertion Anomaly

The insertion anomaly occurs when new record is inserted in the relation. In this anomaly, the user cannot insert a fact about entity until he has an additional fact about another entity.

If a tuple is inserted in referencing relation and referencing attribute value is not present in referenced attribute, it will not allow inserting in referencing relation. For Example, if we try to insert a record in STUDENT_COURSE with STUD_NO =7, it will not allow

5.1.1.2 Deletion Anomaly

The deletion anomaly occurs when a record is deleted from the relation. In this Anomaly, the deletion of facts about an entity automatically deletes the fact of another entity. If a tuple is deleted from referenced relation and referenced attribute value is used by referencing attribute in referencing relation, it will not allow deleting the tuple from referenced relation. For Example, if we try to delete a record from STUDENT with STUD_NO =1, it will not allow.

5.1.1.3 Modification Anomaly

The modification anomaly occurs when the record is updated in the relation. In this anomaly, the modification in the value of specific attribute requires modification records in which that value occurs.

If a tuple is Modified from referenced relation and referenced attribute value is used by referencing attribute in referencing relation, it will not allow Modified the tuple from referenced relation. For Example, if we try to modify a record from STUDENT with STUD_NO =1, it will not allow.

Normalization

The process of producing a simpler and more reliable database structure is called normalization. It is used to create a suitable set of relations for storing data. This process Works through different stages known as normal forms. These stages are 1NF, 2NF, 3NF and so on. Each normal form has certain requirements or condition. These conditions have to fulfilled to bring the database in that particular normal form. 1f a relation satisfies the conditions of a normal form, it said to be in that normal form.

The task of database design starts with un-normalized set of relations. The process of normalization identifies and corrects the problems and complexities of database design. It Produces a new set of relations. The new design is as free of processing problems as possible.

5.1.2 Purposes of Normalization

The purposes of normalization are as follows:

- ✓ It makes the database design efficient in performance
- ✓ It makes the database design efficient in performance.
- ✓ It reduces the amount of data il possible.
- ✓ It makes the database design free of update, insertion and deletion anomalies.
- ✓ It makes the design according to the rules of relational databases
- ✓ It identifies relationship between entities

- ✓ It makes a design that allows simple retrieval of data.
- ✓ It simplifies data maintenance and reduces the need to restructure data.

5.1.3 Characteristics of Normalized Database

Normalized database should have the following characteristics

- ✓ Each relation must have a key field.
- ✓ All fields must contain atomic data.
- ✓ There must be no repeating fields.
- ✓ Each table must contain information about a single entity.
- ✓ Each field in a relation munt depends on key fields.
- ✓ All non-key fields must be mutually independent.

Functional Dependency

Functional dependency is a relationship between attributes. It means that if the value of one attribute is known, it is possible to obtain the value of another attribute. Suppose there is a relation STUDENT with following fields:

STUDENT (RegistrationNo, StudentNome, Class, Email)
If value of RegistrationNo is known, it is possible to obtain the value of StudentName. It means that StudentNameis functionally depends on RegistrationNo. A particular value of RegistrationNo is related to only one value of StudentName StudentName may be related with multiple values of RegistrationNo. For example
RegistrationNo 10 is related with only one value of StudentName. But Student "Usman" may be related with two or more RegistrationNo values because two or Students may have the name Usman".

First Normal Form

As per the rule of first normal form, an attribute (column) of a table cannot hold multiple values. It should hold only atomic values. A relation is in first normal form (1N) if it does not contain a repeating group. A repeating group is a set of one or

more data items that may occur a variable number of times in a tuple. The value in each attribute value should be atomic and every tuple should be unique. Each cell in a relation should contain only one value. An example of un-normalized relation is as follows:

Accountant No	Skill No.	Skill category	Proficiency No.	Accountant Name	Age	Group No	Group city	Group Supervisor
21	113	System	3	Babar	35	52	Multan	Adnan
35	113 204 275	System Tax Audit	5 1 6	Ghafoor	40	44	Bwp	Imran
50	179	Consulting	2	Usman	30	52	Bwp	Gulman
55	148 179	Tax Audit	6 6	Zahid	30	47	Lhr	Adnan

Table: Un-Normalized Relation

The above relation is un-normalized because it contains repeating groups of three attributes Skill Number. Skill Category and Proficiency Number. All three fields contain more than one value.

In order to convert this relation in first normal form these repeating groups should be removed. The following relation is in first normal form:

First Normal Form (1NF)

Accountant No	Skill No.	Skill category	Proficiency No.	Accountant Name	Age	Group No	Group city	Group Supervisor
21	113	System	3	Babar	35	52	Multan	Adnan
35	113	System	5	Ghafoor	40	44	Bwp	Imran
35	204	Tax	1	Ghafoor	40	44	Bwp	Imran
35	275	Audit	6	Ghafoor	40	44	Bwp	Imran
50	179	Consulting	2	Usman	30	52	Bwp	Gulman
55	148	Audit	6	Zahid	30	47	Lhr	Adnan
55	179	Tax	6	Zahid	30	47	Lhr	Adnan

5.1.4 Problems in INF

The relation in 1NF has certain problems which are as follows

5.1.4.1 Updating Problem

Suppose the user wants to change the name of Accountant Number 35 to "M. Ghafoor he has to the name in all records in which Accountant number 35 appears. This process of updating can be very lengthy.

5.1.4.2 Inconsistent Data

The above table may contain inconsistent data. There are three records of Accountant Number 35. It is possible that there are two different names with Account Number 35 in two different records. The user can make this error during updating

5.1.4.3 Addition Problem

Suppose the user wants to add another skill number in the table. It is not possible until an Accountant with that skill exists because both Skill Number and Accountant Number used as primary key in the above table.

5.1.4.4 Deletion Problem

Suppose the user wants to delete the record of supervisor Ghafoor. If he' deletes whole record in which Ghafoor appears, the information about Accountants will also be lost.

Full Functional Dependency

A full functional dependency is a state of database normalization that equates to the normalization standard of Second Normal Form (2NF). In brief, this means that it meets the requirements of First Normal Form (1NF), and all non-key attributes are fully functionally dependent on the primary key. Full dependency between database attributes helps ensure data integrity and avoid data anomalies.

Suppose there is a relation MARKS as follows:

MARKS (RegistratioNo, Subject, Marks)

Assume that one student is studying many subjects. Both Registration No and Subject are required to determine a particular marks. It can be written as follows:

RegistrationNo, SubjectMarks

Here, Marks is fully functionally dependent on both fields because it is not functionally dependent on either RegistrationNo or Subjeet alone.

Second Normal Form

A relation is in Second Normal Form (2N) It is in 1NF and if all of its non-key Attributes are fully functionally dependent on the whole key, It means that none of non-key attributes are related to a part of key.

A table is said to be in 2NF if both the following conditions hold:

- ✓ Table is in 1NF (First normal form)
- ✓ No non-prime attribute is dependent on the proper subset of any candidate key of table.

An attribute that is not part of any candidate key is known as non-prime attribute. The above relation in INF has some attributes which are not depending on the whole Primary key. For example, Accountant Name, Accountant Age and group information is determined by Accountant Number and is not dependent on Skill. The following relation can be created in which all attributes are fully dependent on primary key Account Number.

Accountant No	Proficiency No.	Accountant Name	Age	Group No	Group city	Group Supervisor
21	3	Babar	35	52	Multan	Adnan
35	5	Ghafoor	40	44	Bwp	Imran
50	2	Usman	30	52	Bwp	Gulman
55	6	Zahid	30	47	Lhr	Adnan

Table : Acountant Table in 2NF

Similarly, another relation Skill can be created in which all fields are fully depend on the primary key(Skill No.)

Skill No.	Skill category
113	System
204	Tax
275	Audit

Table: Skill Table in 2NF

The

148	Audit
179	Tax

attribute Proficiency in INF relation was fully dependent on the whole primary key. The Proficiency requires to know the accountant number and skill umber. The third relation will be created as follows:

Accountant No	**Skill No.**	**Proficiency No.**
21	113	3
35	204	5
50	275	2
55	148	6

Figure: Proficiency No in 2NF

There are three relations in Second Normal Form (2NF). the attributes are fully functional dependent on primary keys.

5.1.5 Analysis of Second Normal Form (2NF)

The following analysis indicates whether the problems are eliminated in 2NF or not. Updating Problem: 1f the user needs to change the name of Accountant Number 35 to "M. Ghafoor" in INF, he must change the name in every record in which Accountant number 35 appears. But in 2NE. the record of one accountant appears only once. The updating problem is eliminated in 2NE Inconsistent Data: The record of one accountant appears only once in the database, the possibility of inconsistent data is automatically eliminated. Addition Problem: In 1NF, it was not possible to enter a new Skill Number until an Accountant with that skill existed. In 2NF, any number of skills can be added in Skill relation without an accountant with that skill. It eliminates the addition problem. Deletion problem: In 2NE, if the record of Ghafoor is deleted, it does not delete any other record. The analysis shows that the second normal form has solved all problems of INE.

Transitive Dependency

Transitive dependency is a condition in which an attribute is dependent on an attribute that is not part of the primary key.

Suppose A, B, and C is attributes of a relation. If A →B and B→C then C

Transitive dependent on A via B provided that A is not functionally dependent on B or C. Suppose there is a relation BOOK as follows:

BOOK (BookID, BookDescription, CategorylD, CategoryDescription)

We can write:

BookID→ CatogorylD, CategorylD→CategoryDescription

A transitive dependency occurs when one non-key attribute determines another non-key attribute. In above relation, BooklD determines CategoryID and CategaryID determines CategoryDescriplion. Category description is transitively dependent on BookID attribute.

Third Normal Form

A relation is in third normal form if it is in 2NF and if no non-key attribute is dependent on another non-key attribute. It means that all non-key attributes are functionally dependent only on primary key. There should be no transitive dependency in a relation.

A table design is said to be in 3NF if both the following conditions hold:

- ✓ Table must be in 2NF
- ✓ Transitive functional dependency of non-prime attribute on any super key should be removed.

An attribute that is not part of any candidate key is known as non-prime attribute. In order to convert a relation to 3NF, remove all attributes from the 2NF record that depend on another non-key field Place them into a new relation with the

other attributes as the primary key. The accountant table in 2NF contains some attributes which are depending on non-key attributes. For example, Group City and Group Supervisor are depending on a non-key field

Group Number. A new relation can be created as follows:

Accountant Number	**Accountant Name**	**Accountant Age**	**Group number**
21	Ali	55	52
35	Daud	32	44
50	Chohan	40	44
77	Zahid	52	52

Figure: Accountant Table in 3NF

The 2nd table is created from the accountant table in 1NF is as follows.

Group Number	**Group city**	**Group Supervisor**
52	ISD	Baber
44	Lhr	Ghafoor

Figure: Group Table in Third Normal Form(3NF)

Both Accountant table and Group table contain the attribute Group Number. This attribute is used to join both tables. The Skill table in 2NF contains no attribute, which is depending on a non-key attribute. It is already in third normal form and will be used without any further change.

Primary Key

Skill Number	Skill Category
113	Systems
179	Tax
204	Audit
148	Consulting

Figure: Skill Table in 3NF

The Proficiency table in 2NF also contains no attribute which is depending on non-key attribute. It is already in third normal form and will be used without any further change.

Accountant Number	Skill Number	Proficiency
21	113	3
35	113	5
35	179	1
35	204	6
50	179	2
77	148	6
77	179	6

Figure: Proficiency Table in 3NF

Boyce-Codd Normal Form (BCNF)

A relation is in Boyce-Codd normal form and only if every determinant is candidate key. It can be checked by identifying all determinants and then making sure that all these determinants are candidate keys. BCNF is a stronger form of third normal form. A relation in BCNF is also in third normal form. But a relation in 3NF may not be in BCNE.

ProjectID	PartID	QuantityUSed	PartName
101	P01	20	CD-R
112	P05	6	Zip Disk
194	P01	12	CD-R
194	P02	1	Floppy Disk
194	P05	3	Zip Disk

Figure: Parts And Project Table

Assume that PartName is unique. It means that no two parts can have the same name. There are now two candidate keys (ProjectiD, PartiD) and (ProjectiD, PaitName)
There are following dependencies
(ProjectID, PartID)→QlyUsed
PartID→PartName
PartName→PartiD
This relation satisfies 2NF because here are no non-key attributes that are dependent on a subset of the primary key. PartName is not a non-key attribute, it is part of a candidate key. Therefore this relation is in 2NF.

There are no transitive dependencies so the relation is in 3NF. PartID and PartName are ignored by 3NF rule because they are both key attributes.

In order to convert this relation to BCNE, all functional dependencies must be removed which have a determinant that is not a candidate key. The result is as follows:

PartID	PartName
P01	CD-R
P02	Floppy Disk
P05	Zip Disk

ProjectID	PartID	QuantityUSed
101	P01	20
112	P05	6
194	P01	12
194	P02	1
194	P05	3

Figure: Tables in BCNF

EXERCISE No. 05

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1. A functional dependency is a relationship between or among

A. Entities
B. Rows
C. Attributes
D. Tables

2. Which functional dependency types is/are not present in the following dependencies?

Empno -> EName, Salary, Deptno, DName
DeptNo -> DName
EmpNo -> DName

A. Full functional dependency
B. Partial functional dependency
C. Transitive functional dependency
D. Both B and C

3. The database design prevents some data from being stored due to _____.

A. Deletion anomalies
B. Insertion anomalies
C. Update anomalies
D. Selection anomalies

4. **If one attribute is determinant of second, which in turn is determinant of third, then the relation cannot be:**

   A. Well-structured
   B. 1NF
   C. 2NF
   D. 3NF

5. **A relation is in 2NF if:**

   A. All the values of non-key attributes are dependent fully on the candidate key.
   B. Any non-key attribute that are dependent on only part of the candidate key should be moved to another relation where the partial key is the actual full key.
   C. It must be already in the 1NF.
   D. All of the above.

6. **In which form of function there is no partial functional dependencies.**
   A. BCNF
   B.2NF
   C.3NF
   D.4NF

7. **In which normal foam Boyce-code can operate?**
   A. First normal foam
   B. Second normal foam
   C. Third normal foam
   D. all of above

8. **In relational databases, the table is also called _____?**
   A. Tuple
   B. Relation
   C. File
   D.None

9. In 3NF a non-key attribute must not depend on a _____?

A. Non key attribute
B. Key attribute
C. Composite key
D. Sort key

10. Different attributes in two different tables having same name are referred to as _____?

A. Synonym
B. Homonym
C. Acronym
D. Mutually exclusive

11. Every relation must have _____?

A. Primary key
B. Candidate key
C. Secondary key
D. Mutually exclusiveness

12. The goal of normalization is to ____?

A). Get stable data structure
B). Increase number of relations
C). Increase redundancy
D). None of these

13. A rule that states that each foreign key value must match a primary key value in the other relation is called _____?

A. Referential integrity constraint
B. Key match rule
C. Entity key group rule
D. Foreign / primary match rule

14. Two or more attributes having different names but same meaning are called _____?

A. Homonym
B. Aliases
C. Synonym
D. Alternate attributes

15. A constraint between two attributes is called _____?

A. Functional relation

B. Attribute dependency

C. Functional dependency

D. Functional relation constraint

ANSWER KEY

1.	C	2.	B	3.	B	4.	D	5.	D
6.	B	7.	C	8.	B	9.	A	10.	B
11.	A	12.	A	13	A	14.	C	15.	C

PART-II SAMPLE SHORT QUESTIONS

1. Define Normalization.
2. Write any to benefit of normalized data.
3. Define partial dependencies.
4. Define functional dependencies.
5. What is deletion anomaly?
6. What is transitive dependency?
7. What is meant by anomaly?
8. What is the purpose of 3NF?
9. Why we do 2NF?
10. Define BCNF?

PART-III SAMPLE LONG QUESTIONS

1. What is normalization and write advantages of normalization.?

2.Write about 3 types of Anomalies occurs during database management.

3.Briefly explain any 3 types of dependencies remove during database normalization.

4.Explain difference between 1NF and 2NF.

5.Explain difference Between Functional and fully functional dependencies.

6 Chapter No. 6 (Relational Algebra and SQL)

Objectives

After completion of this chapter students will be able to understand:

- ✓ Unary and Binary operations
- ✓ Cartesian Product
- ✓ Set Operations
- ✓ SQL Operators
- ✓ Relational Algebra and SQL
- ✓ Introduction to DDL and DML
- ✓ Data Control Language
- ✓ Aggregate Function in SQL, Grouping Data

Introduction of Relational Algebra and SQL in DBMS

Relational Algebra is procedural query language, which takes Relation as input and generates relation as output. Relational algebra mainly provides theoretical foundation for relational databases and SQL.

Relational Algebra is a widely used procedural query language. It collects instances of relations as input and gives occurrences of relations as output. It uses various operations to perform this action. SQL Relational algebra query operations are performed recursively on a relation. The output of these operations is a new relation, which might be formed from one or more input relations.

Every database management system must define a query language to allow users to access the data stored in the database. Relational Algebra is a procedural query language used to query the database tables to access data in different ways.

In relational algebra, input is a relation (table from which data has to be accessed) and output is also a relation (a temporary table holding the data asked for by the user).

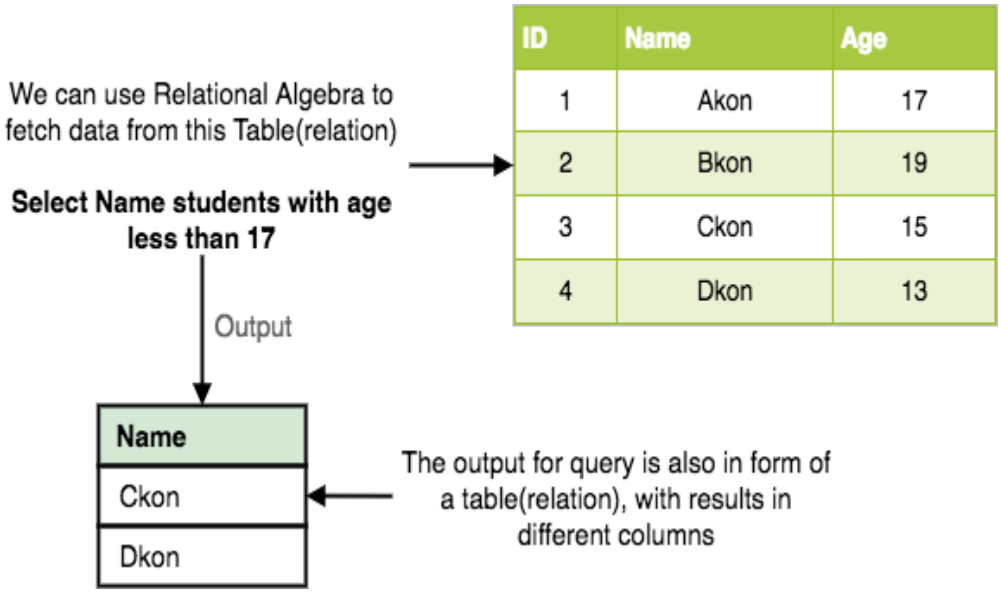


Figure 2Relational Algebra

Relational Algebra works on the whole table at once, so we do not have to use loops etc. to iterate over all the rows(tuples) of data one by one. All we have to do is specify the table name from which we need the data, and in a single line of command, relational algebra will traverse the entire given table to fetch data for you.

Relational algebra is a query language that processes one or more relations to define another-relation.
The basic operation of relational algebra are as follows;

6.1.1 Unary operations

Operations which involve only one relation are called unary operations.

Examples
Select operation.
project Operation.
Binary operations:
Operations which involve pairs of relations are called binary operations.
Examples:
Union
Difference.
Cartesian product

6.1.2 Select Operation (σ)

This is used to fetch rows(tuples) from table(relation) which satisfies a given condition.

Syntax: $\sigma_p(r)$

Where, σ represents the Select Predicate, r is the name of relation (table name in which you want to look for data), and p is the prepositional logic, where we specify the conditions that must be satisfied by the data. In prepositional logic, one can use **unary** and **binary** operators like =, <, > etc., to specify the conditions.

Let's take an example of the student table we specified above in the Introduction of relational algebra, and fetch data for **students** with **age** more than 17.

σage > 17 (Student)

This will fetch the tuples(rows) from table **Student**, for which **age** will be greater than **17**. You can also use, and, or etc operators, to specify two conditions, for example,

σage > 17 and gender = 'Male' (Student)

This will return tuples(rows) from table **Student** with information of male students, of age more than 17. (Consider the Student table has an attribute Gender too.)

Selection is used to select required tuples of the relations.

```
   R
 (A  B  C)
 ----------
  1  2  4
  2  2  3
  3  2  3
  4  3  4
```

for the above relation

σ (c>3) R

will select the tuples which have c more than 3.

Note: selection operator only selects the required tuples but does not display them. For displaying, data projection operator is used.
For the above selected tuples, to display we need to use projection also.

π (σ (c>3)R) will show following tuples.

```
A  B  C
-------
1  2  4
4  3  4
```

More examples −

$\sigma_{subject = "database"}$(Books)

Output − Selects tuples from books where subject is 'database'.

$\sigma_{subject = "database" \text{ and } price = "450"}$(Books)

Output − Selects tuples from books where subject is 'database' and 'price' is 450.

$\sigma_{subject = "database" \text{ and } price = "450" \text{ or } year > "2010"}$(Books)

Output − Selects tuples from books where subject is 'database' and 'price' is 450 or those books published after 2010

Project Operation (∏)

Project operation is used to project only a certain set of attributes of a relation. In simple words, If you want to see only the **names** all of the students in the **Student** table, then you can use Project Operation.

It will only project or show the columns or attributes asked for, and will also remove duplicate data from the columns.

Syntax: $\prod_{A1, A2...}(r)$

where A1, A2 etc. are attribute names (column names).
For example,
$\prod_{Name, Age}$ (Student)

Above statement will show us only the Name and Age columns for all the rows of data in Student table.

Projection is used to project required column data from a relation.

Example:

R
(A B C)
---
1 2 4
2 2 3
3 2 3
4 3 4
π (BC)
B C
---
2 4

2 3
3 4

By Default, projection removes duplicate data.

Union Operation (∪)

This operation is used to fetch data from two relations(tables) or temporary relation (result of another operation).

For this operation to work, the relations(tables) specified should have same number of attributes(columns) and same attribute domain. Also, the duplicate tuples are automatically eliminated from the result.

Syntax: A ∪ B

where A and B are relations.

For example, if we have two tables **RegularClass** and **ExtraClass**, both have a column **student** to save name of student, then,

$\prod_{Student}$ (RegularClass) ∪ $\prod_{Student}$ (ExtraClass)

Above operation will give us name of **Students** who are attending both regular classes and extra classes, eliminating repetition.

Set Difference (-)

This operation is used to find data present in one relation and not present in the second relation. This operation is also applicable on two relations, just like Union operation.

Syntax: A - B

where A and B are relations.

For example, if we want to find name of students who attend the regular class but not the extra class, then, we can use the below operation:

$\prod_{Student}$ (RegularClass) - $\prod_{Student}$ (ExtraClass)

Cartesian Product (X)

This is used to combine data from two different relations (tables) into one and fetch data from the combined relation.

Syntax: A X B

For example, if we want to find the information for Regular Class and Extra Class which are conducted during morning, then, we can use the following operation:

$\sigma_{time = 'morning'}$ (RegularClass X ExtraClass)

For the above query to work, both **RegularClass** and **ExtraClass** should have the attribute **time**.

Cross product between two relations let say A and B, so cross product between A X B will results all the attributes of A followed by each attribute of B. Each record of A will pair with every record of B.

Below is the example

A			B	
(Name	Age	Sex)	(Id	Course)
Ram	14	M	1	DS
Sona	15	F	2	DBMS
kim	20	M		

A X B

Name	Age	Sex	Id	Course
Ram	14	M	1	DS
Ram	14	M	2	DBMS
Sona	15	F	1	DS
Sona	15	F	2	DBMS
Kim	20	M	1	DS
Kim	20	M	2	DBMS

Note: if A has 'n' tuples and B has 'm' tuples then A X B will have 'n*m' tuples.

Rename Operation (ρ)

This operation is used to rename the output relation for any query operation which returns result like Select, Project etc. Or to simply rename a relation(table)

Syntax: ρ (Relation New, Relation Old)

Join

DBMS, a joint statement is **mainly used to combine two tables based on a specified common field between them**. Thus, we can execute the product and selection process on two tables using a single join statement.

Apart from these common operations Relational Algebra is also used for **Join** operations like

1. Natural Join (⋈)

Natural join is a binary operator. Natural join between two or more relations will result set of all combination of tuples where they have equal common attribute.

Let us see this example

Emp **Dep**

(Name Id Dept_name) (Dept_name Manager)

---------- ----------

A 120 IT Sale Y

B 125 HR Prod Z

C 110 Sale IT A

D 111 IT

Emp ⋈ Dep

Name	Id	Dept_name	Manager
A	120	IT	A
C	110	Sale	Y
D	111	IT	A

1. Conditional Join

Conditional join works similar to natural join. In natural join, by default condition is equal between common attribute while in conditional join we can specify the any condition such as greater than, less than, not equal

Let us see below example

R S

(ID Sex Marks) (ID Sex Marks)

---------- ----------

1 F 45 10 M 20

2 F 55 11 M 22

3 F 60 12 M 59

Join between R And S with condition **R.marks >= S.marks**

R.ID	R.Sex	R.Marks	S.ID	S.Sex	S.Marks
1	F	45	10	M	20
1	F	45	11	M	22
2	F	55	10	M	20
2	F	55	11	M	22
3	F	60	10	M	20
3	F	60	11	M	22
3	F	60	12	M	59

2. Equi Join

An **equi join** is a type of join that combines tables based on matching values in specified columns.

Please remember that:

- ✓ The column names do not need to be the same.
- ✓ The resultant table contains repeated columns.
- ✓ It is possible to perform an equi join on more than two tables.

Syntax

There are two ways to use equi join in SQL:

```
SELECT *
FROM TableName1, TableName2
WHERE TableName1.ColumnName = TableName2.ColumnName;
-- OR
SELECT *
FROM TableName1
```

```
JOIN TableName2
ON TableName1.ColumnName = TableName2.ColumnName;
```

In the first method, after the SELECT keyword, the names of the columns that are to be included in the result of the query are specified. The * operator is used if all the columns need to be selected. After the FROM keyword, the tables which need to be joined are specified. In the WHERE clause, the table and column names are specified along with an = operator.

In the second method, the JOIN keyword is used to join the tables based on the condition provided after the ON keyword.

Example

The following tables have been created

1. **Product_List**

ID	**Pname**
1	Apples
2	Oranges

2. **Product_details**

ID	**Brand**	**Origin**
1	Fresh Foods	USA
2	Angro Ltd	Pakistan

3. **brand details**

Brand	Office Address
Fresh Foods	123 Seattle USA
Angro Ltd	124 Lahore

```
/*Performing the equi join with two tables*/
SELECT *
FROM product_list
JOIN product_details
ON product_list.ID = product_details.ID;
/*Performing the equi join with three tables*/
SELECT product_list.ID, product_list. Pname,
product_details. Brand, product_details. Origin,
brand details. Office Address
FROM product_list, product_details, brand details
WHERE product_list.ID = product_details.ID
and product_details. Brand = brand_details. Brand
```

Output of above statement

ID	Pname	ID	Brand	Origin
1	Apples	1	Fresh Foods	USA
2	Oranges	2	Angro Ltd	Pakistan

ID	Pname	Brand	Origin	OfficeAddress
1	Apples	Fresh Foods	USA	123 Seattle USA
2	Oranges	Angro Ltd	Pakistan	124 Lahore

3.INNER JOIN

A INNER JOIN clause is used to combine rows from two or more tables, based on a related column between them.Let's look at a selection from the "Orders" table:

OrderID	CustomerID	OrderDate
10308	2	1996-09-18
10309	37	1996-09-19
10310	77	1996-09-20

CustomerID	CustomerName	ContactName	Country
1	Alfreds Futterkiste	Maria Anders	Germany

2	Ana Trujillo Emparedados y helados	Ana Trujillo	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mexico

Then, look at a selection from the "Customers" table: Notice that the "CustomerID" column in the "Orders" table refers to the "CustomerID" in the "Customers" table. The relationship between the two tables above is the "CustomerID" column. Then, we can create the following SQL statement (that contains an INNER JOIN), that selects records that have matching values in both tables:

Example

```
SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
INNER JOIN Customers ON Orders.CustomerID=Customers.CustomerID;
```

and it will produce something like this:

OrderID	CustomerName	OrderDate

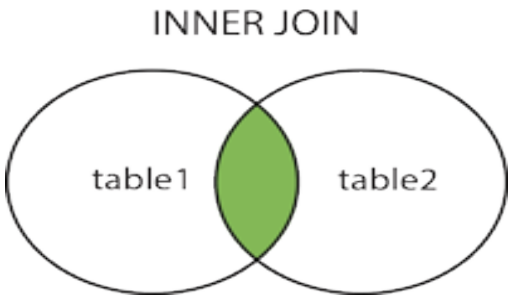
10308	Ana Trujillo Emparedados y helados	9/18/1996
10365	Antonio Moreno Taquería	11/27/1996
10383	Around the Horn	12/16/1996
10355	Around the Horn	11/15/1996
10278	Berglunds snabbköp	8/12/1996

SQL INNER JOIN Keyword

The INNER JOIN keyword selects records that have matching values in both tables.

6.1.3 INNER JOIN Syntax

SELECT *column_name(s)*
FROM *table1*
INNER JOIN *table2*
ON *table1.column_name* = *table2.column_name*;



Demo Database

In this tutorial we will use the well-known Northwind sample database.

Below is a selection from the "Orders" table:

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10308	2	7	1996-09-18	3
10309	37	3	1996-09-19	1
10310	77	8	1996-09-20	2

And a selection from the "Customers" table:

CustomerID	CustomerName	Contact Name	Address	City	Postal Code	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany

2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

SQL INNER JOIN Example

The following SQL statement selects all orders with customer information:

Example

```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID;
```

Note: The INNER JOIN keyword selects all rows from both tables as long as there is a match between the columns. If there are records in the "Orders" table that do not have matches in "Customers", these orders will not be shown!

JOIN Three Tables

The following SQL statement selects all orders with customer and shipper information:

Example

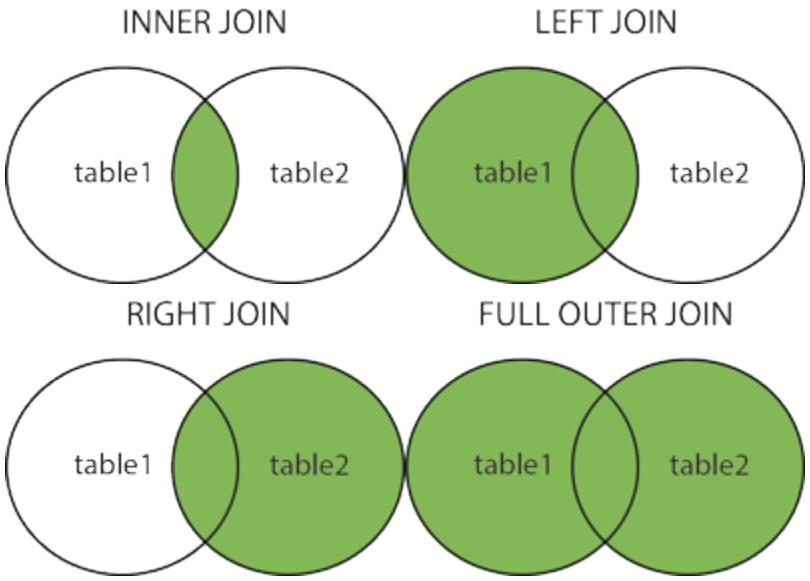
```
SELECT Orders.OrderID, Customers.CustomerName, Shippers.ShipperName
FROM ((Orders
```

```
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID)
INNER JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID);
```

Different Types of SQL JOINs

Here are the different types of the JOINs in SQL:

- ✓ (INNER) JOIN : Returns records that have matching values in both tables
- ✓ LEFT (OUTER) JOIN: Returns all records from the left table, and the matched records from the right table
- ✓ RIGHT (OUTER) JOIN: Returns all records from the right table, and the matched records from the left table
- ✓ FULL (OUTER) JOIN: Returns all records when there is a match in either left or right table



What is SQL?

SQL (**S**tructured **Q**uery **L**anguage) is a programming language used to communicate with data stored in a relational database management system. SQL syntax is similar to the English language, which makes it relatively easy to write, read, and interpret.

6.1.4 What is an Operator in SQL?

An operator is a reserved word or a character used primarily in an SQL statement's WHERE clause to perform operation(s), such as comparisons and arithmetic operations. These Operators are used to specify conditions in an SQL statement and to serve as conjunctions for multiple conditions in a statement.

- ✓ Arithmetic operators
- ✓ Comparison operators
- ✓ Logical operators
- ✓ Operators used to negate conditions

6.1.5 SQL Arithmetic Operators

Assume **'variable a'** holds 10 and **'variable b'** holds 20, then –

Show Examples

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator.	a + b will give 30
- (Subtraction)	Subtracts right hand operand from left hand operand.	a - b will give -10
* (Multiplication)	Multiplies values on either side of the operator.	a * b will give 200

/ (Division)	Divides left hand operand by right hand operand.	b / a will give 2
% (Modulus)	Divides left hand operand by right hand operand and returns remainder.	b % a will give 0

6.1.6 SQL Comparison Operators

Assume **'variable a'** holds 10 and **'variable b'** holds 20, then −

Show Examples

Operator	Description	Example
=	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(a = b) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(a != b) is true.
<>	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(a <> b) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(a > b) is not true.

<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(a < b) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(a >= b) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(a <= b) is true.
!<	Checks if the value of left operand is not less than the value of right operand, if yes then condition becomes true.	(a !< b) is false.
!>	Checks if the value of left operand is not greater than the value of right operand, if yes then condition becomes true.	(a !> b) is true.

6.1.7 SQL Logical Operators

Here is a list of all the logical operators available in SQL.

Show Examples

Sr.No.	Operator & Description
1	**ALL**

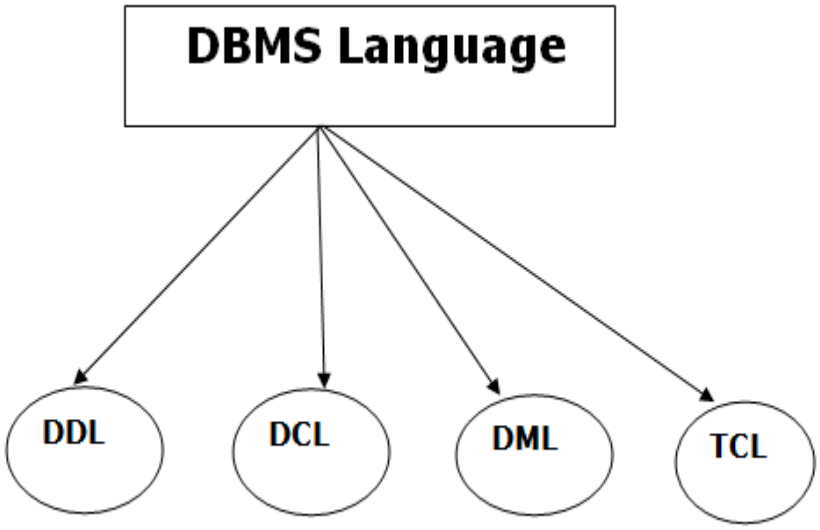
	The ALL operator is used to compare a value to all values in another value set.
2	**AND** The AND operator allows the existence of multiple conditions in an SQL statement's WHERE clause.
3	**ANY** The ANY operator is used to compare a value to any applicable value in the list as per the condition.
4	**BETWEEN** The BETWEEN operator is used to search for values that are within a set of values, given the minimum value and the maximum value.
5	**EXISTS** The EXISTS operator is used to search for the presence of a row in a specified table that meets a certain criterion.
6	**IN** The IN operator is used to compare a value to a list of literal values that have been specified.
7	**LIKE** The LIKE operator is used to compare a value to similar values using wildcard operators.

8	**NOT** The NOT operator reverses the meaning of the logical operator with which it is used. Eg: NOT EXISTS, NOT BETWEEN, NOT IN, etc. **This is a negate operator.**
9	**OR** The OR operator is used to combine multiple conditions in an SQL statement's WHERE clause.
10	**IS NULL** The NULL operator is used to compare a value with a NULL value.
11	**UNIQUE** The UNIQUE operator searches every row of a specified table for uniqueness (no duplicates).

Database Language

- ✓ A DBMS has appropriate languages and interfaces to express database queries and updates.
- ✓ Database languages can be used to read, store and update the data in the database.

6.1.8 Types of Database Language



DDL is used to specify database schema. DML is used to, read and update the database.

These languages are called data sublanguages because they do not provide construct

Computing needs like conditional or iterative statements. Many DBMSs provide the facility to embed the sublanguage in a high-level programming language like COBOL, Fortran Java and Visual Basic etc. In this situation, the high-level language is liked host language.

6.1.9 Data Definition Language (DDL)

A language that is used to describe and name the entities attributes. Relationships, Associated integrity and security constraint is called data definition language. DD1 is used to express a set of definitions for specifying database schema. It is used to define or modify a schema. It is not used to manipulate data,

- ✓ It is used to create schema, tables, indexes, constraints, etc. in the database.
- ✓ Using the DDL statements, you can create the skeleton of the database.
- ✓ Data definition language is used to store the information of metadata like the number of tables and schemas, their names, indexes, columns in each table, constraints, etc.

Here are some tasks that come under DDL:

- ✓ **Create:** It is used to create objects in the database.
- ✓ **Alter:** It is used to alter the structure of the database.
- ✓ **Drop:** It is used to delete objects from the database.
- ✓ **Truncate:** It is used to remove all records from a table.
- ✓ **Rename:** It is used to rename an object.
- ✓ **Comment:** It is used to comment on the data dictionary.

These commands are used to update the database schema that's why they come under Data definition language.

6.1.10 Data Manipulation Language

A language that supports the basic data manipulation operations on data in databases is called data manipulation language. Data manipulation operations include the following

- ✓ Insertion of new data in database
- ✓ Modification of data in database
- ✓ Retrieval of data from database
- ✓ Deletion of data from database

DML applies to external, conceptual and internal levels. It is necessary to define efficient low level procedural to access data efficiently.

DML stands for **D**ata **M**anipulation **L**anguage. It is used for accessing and manipulating data in a database. It handles user requests.

Here are some tasks that come under DML:

- ✓ **Select:** It is used to retrieve data from a database.
- ✓ **Insert:** It is used to insert data into a table.
- ✓ **Update:** It is used to update existing data within a table.
- ✓ **Delete:** It is used to delete all records from a table.
- ✓ **Merge:** It performs UPSERT operation, i.e., insert or update operations.
- ✓ **Call:** It is used to call a structured query language or a Java subprogram.
- ✓ **Explain Plan:** It has the parameter of explaining data.
- ✓ **Lock Table:** It controls concurrency.

There are basically two types of DML

- ✓ **Procedural DML:** It requires a user to specify the required data and how to get the required data.
- ✓ **Nonprocedural DML**: It requires a user to specify the required without specifying

How to get the data?

Nonprocedural DML is usually easier to learn and use than procedural DML. The user does not specify how to get data. These languages may generate code that is not as efficient as produced by procedural languages.

Difference between DDL and DML:

DDL	DML
It stands for Data Definition Language.	It stands for Data Manipulation Language.

It is used to create database schema and can be used to define some constraints as well.	It is used to add, retrieve or update the data.
It basically defines the column (Attributes) of the table.	It add or update the row of the table. These rows are called as tuple.
It doesn't have any further classification.	It is further classified into Procedural and Non-Procedural DML.
Basic command present in DDL are CREATE, DROP, RENAME, ALTER etc.	BASIC command present in DML are UPDATE, INSERT, MERGE etc.
DDL does not use WHERE clause in its statement.	While DML uses WHERE clause in its statement.

A data control language (DCL) is a syntax similar to a computer programming language used to control access to data stored in a database (Authorization). In particular, it is a component of Structured Query Language (SQL). Data Control Language is one of the logical groups in SQL Commands. SQL is the standard language for relational database management systems. SQL statements are used to perform tasks such as insert data to a database, delete or update data in a database, or retrieve data from a database.

6.1.11 Data Control Language

- ✓ **DCL** stands for **D**ata **C**ontrol **L**anguage. It is used to retrieve the stored or saved data.
- ✓ The DCL execution is transactional. It also has rollback parameters. (But in Oracle database, the execution of data control language does not have the feature of rolling back.)

Here are some tasks that come under DCL:

- ✓ **Grant:** It is used to give user access privileges to a database.
- ✓ **Revoke:** It is used to take back permissions from the user.

There are the following operations which have the authorization of Revoke:

CONNECT, INSERT, USAGE, EXECUTE, DELETE, UPDATE and SELECT.

6.1.12 Transaction Control Language

TCL is used to run the changes made by the DML statement. TCL can be grouped into a logical transaction.

Here are some tasks that come under TCL:

- ✓ **Commit:** It is used to save the transaction on the database.
- ✓ **Rollback:** It is used to restore the database to original since the last Commit.

Aggregate Functions (Transact-SQL)

An aggregate function performs a calculation on a set of values, and returns a single value. Except for COUNT (*), aggregate functions ignore null values. Aggregate functions are often used with the GROUP BY clause of the SELECT statement. All aggregate functions are deterministic. In other words, aggregate functions return the same value each time that they are called, when called with a specific set of input values. See Deterministic and Nondeterministic Functions for more information about function determinism. The OVER clause may follow all aggregate functions, except the STRING_AGG, GROUPING or GROUPING_ID functions.

Use aggregate functions as expressions only in the following situations:

- ✓ The select list of a SELECT statement (either a subquery or an outer query).
- ✓ A HAVING clause.

As the Basic SQL Tutorial points out, SQL is excellent at aggregating data the way you might in a pivot table in Excel. You will use aggregate functions all the time, so it's important to get comfortable with them. The functions themselves are the same ones you will find in Excel or any other analytics program. We'll cover them individually in the next few lessons. Here's a quick preview:

- ✓ COUNT counts how many rows are in a particular column.
- ✓ SUM adds together all the values in a particular column.
- ✓ MIN and MAX return the lowest and highest values in a particular column, respectively.
- ✓ AVG calculates the average of a group of selected values.

Introduction to SQL COUNT function

The SQL COUNT function is an aggregate function that returns the number of rows returned by a query. You can use the COUNT function in the SELECT statement to get the number of employees, the number of employees in each department, the number of employees who hold a specific job, etc.

The following illustrates the syntax of the SQL COUNT function:

```
COUNT([ALL | DISTINCT] expression);
```
Code language: SQL (Structured Query Language) (sql)

The result of the COUNT function depends on the argument that you pass to it.

✓ By default, the COUNT function uses the ALL keyword whether you specify it or not. The ALL keyword means that all items in the group are considered including the duplicate values. For example, if you have a group (1, 2, 3, 3, 4, 4) and apply the COUNT function, the result is 6.

✓ If you specify the DISTINCT keyword explicitly, only unique non-null values are considered. The COUNT function returns 4 if you apply it to the group (1,2,3,3,4,4).

Another form of the COUNT function that accepts an asterisk (*) as the argument is as follows:

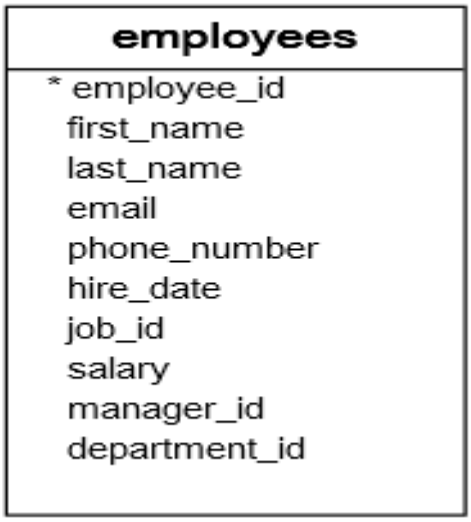
```
COUNT(*)
```

Code language: SQL (Structured Query Language) (sql)

The COUNT(*) function returns the number of rows in a table including the rows that contain the NULL values.

SQL COUNT Function examples

Let's take some examples to see how the COUNT function works. We will use the employees table in the sample database for the demonstration purposes.



SQL COUNT (*) example

To get the number of rows in the employees table, you use the COUNT (*) function table as follows:

```
SELECT
   COUNT(*)
```

```
FROM
   employees;
```

Introduction to SQL SUM function

In this topic you will learn how to use SQL Server SUM() function to calculate the sum of values. The SQL Server SUM() function is an aggregate function that calculates the sum of all or distinct values in an expression.

The syntax of the SUM() function is as follows:

```
SUM([ALL | DISTINCT ] expression)
```

Code language: SQL (Structured Query Language) (sql)

In this syntax:

- ✓ ALL instructs the SUM() function to return the sum of all values including duplicates. ALL is used by default.
- ✓ DISTINCT instructs the SUM() function to calculate the sum of the only distinct values.
- ✓ expression is any valid expression that returns an exact or approximate numeric value. Note that aggregate functions or subqueries are not accepted in the expression.

The SUM() function ignores NULL values.

SQL Server SUM() function examples

Let's take some practical examples of using the SUM() function.

A) Simple SQL Server SUM() function example

The following statement returns the total stocks of all products in all stores:

```
SELECT
   SUM(quantity) total_stocks
FROM
   production.stocks;
```
Code language: SQL (Structured Query Language) (sql)

The following shows the output:

```
total_stocks
------------
13511
```

Introduction to the SQL Server `MAX()` function

SQL Server MAX() function is an aggregate function that returns the maximum value in a set.

The following shows the syntax of the MAX() function:

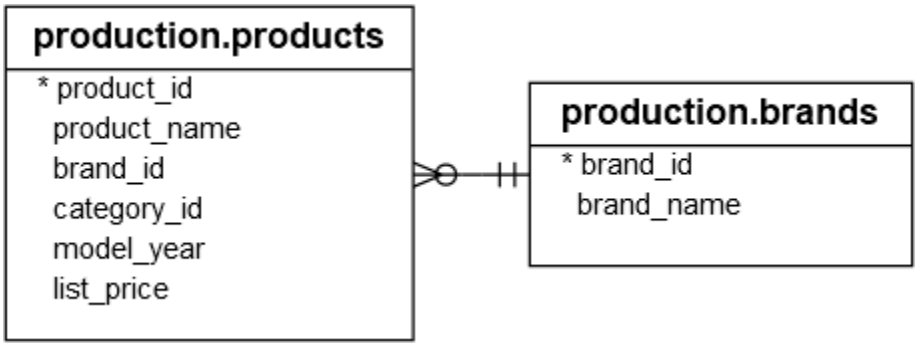
```
MAX(expression)
```
Code language: SQL (Structured Query Language) (sql)

The MAX () function accepts an expression that can be a column or a valid expression.

Similar to the MIN () function, the MAX() function ignores NULL values and considers all values in the calculation.

SQL Server MAX () function examples

We will use the products and brands tables for the demonstration:



SQL Server MAX() – finding the highest list price

The following statement uses the MAX() function to find the highest list price of all products in the products table:

```
SELECT
   MAX(list_price) max_list_price
FROM
   production.products;
```
Code language: SQL (Structured Query Language) (sql)

Here is the output:

max_list_price
11999.99

SQL Server MIN () function

SQL Server MIN () function is an aggregate function that allows you to find the minimum value in a set. The following illustrates the syntax of the MIN () function:

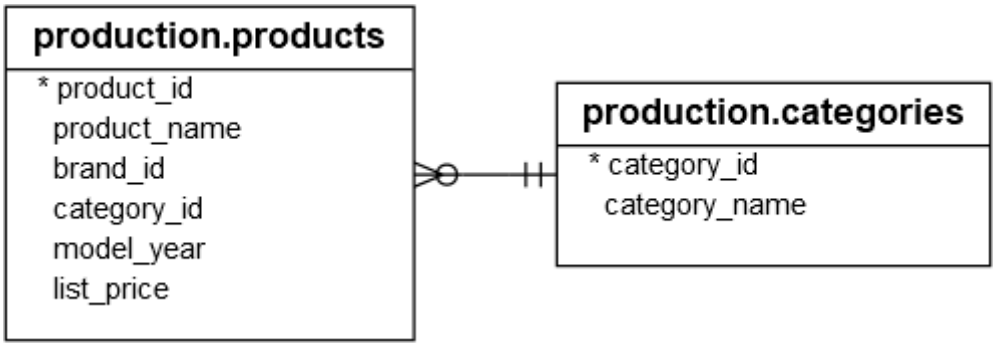
```
MIN(expression)
```
Code language: SQL (Structured Query Language) (SQL)

The MIN() function accepts an expression that can be a column or a valid expression. The MIN () function applies to all values in a set. It means that the DISTINCT modifier has no effect for the MIN () function.

Note that the MIN () function ignores NULL values.

SQL Server MIN () function examples

We will use the products and categories tables from the sample database for the demonstration.



SQL Server MIN () function simple example

The following example finds the lowest list price of all products:

```
SELECT
   MIN(list_price) min_list_price
FROM
   production.products;
```

Code language: SQL (Structured Query Language) (sql)

Here is the output:

min_list_price
89.99

Introduction to SQL Server AVG () function

SQL Server AVG () function is an aggregate function that returns the average value of a group.

The following illustrates the syntax of the AVG () function:

```
AVG([ALL | DISTINCT] expression)
```
Code language: SQL (Structured Query Language) (SQL)

In this syntax:

- ✓ ALL instructs the AVG () function to take all values for calculation. ALL is used by default.
- ✓ DISTINCT instructs the AVG () function to operate only on unique values.
- ✓ expression is a valid expression that returns a numeric value.

The AVG() function ignores NULL values.

SQL Server AVG () function: ALL vs. DISTINCT

The following statements create a new table, insert some values into the table, and query data against it:

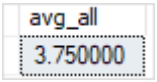
```
CREATE TABLE t(
   val dec(10,2)
);
INSERT INTO t(val)
VALUES(1),(2),(3),(4),(4),(5),(5),(6);

SELECT
   val
FROM
   t;
```
Code language: SQL (Structured Query Language) (SQL)

The following statement uses the AVG() function to calculate the average of all values in the t table:

```
SELECT
   AVG(ALL val)
FROM
   t;
```
Code language: SQL (Structured Query Language) (SQL)

The following picture shows the output:



avg_all
3.750000

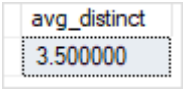
In this example, we used ALL modifier, therefore, the average function considers all eight values in the Val column in the calculation:

```
(1 + 2 + 3 + 4 + 4 + 5 + 5 + 6) /  8 = 3.75
```
Code language: SQL (Structured Query Language) (SQL)

The following statement uses the AVG() function with DISTINCT modifier:

```
SELECT
   AVG(DISTINCT val)
FROM
   t;
```
Code language: SQL (Structured Query Language) (SQL)

Here is the result:



avg_distinct
3.500000

Because of the DISTINCT modifier, the AVG() function performs the calculation on distinct values:

```
(1 + 2 + 3 + 4 + 5 + 6) / 6 = 3.5
```
Code language: SQL (Structured Query Language) (SQL)

SQL Server AVG() function examples

Let's take some examples to see how the AVG() function works.

A) SQL Server AVG() simple example

The following example returns the average list price of all products:

```
SELECT
   AVG(list_price)
FROM
   production.products;
```

Code language: SQL (Structured Query Language) (SQL)

In this example, the AVG() function returned a single value for the whole table.

Here is the output:



Exercise No.04

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1) What is the full form of SQL?
   A. Structured Query List
   B. Structure Query Language
   C. Sample Query Language
   D. None of these.

2) Which of the following is not a built-in aggregate function in SQL?
   A. Avg
   B. Max
   C. Total
   D. Count

3) The _____ aggregation operation adds up all the values of the attribute?
   A. Add
   B. Avg
   C. Max
   D. Sum

4) What values does the count (*) function ignore?
   A. Repetitive values
   B. Null values
   C. Characters
   D. Integers

5) Which of the following is not a valid aggregate function?
   A. COUNT
   B. COMPUTE

C. SUM
D. MAX

6) Which data manipulation command is used to combines the records from one or more tables?

A. SELECT
B. PROJECT
C. JOIN
D. PRODUCT

7) Which type of JOIN is used to returns rows that do not have matching values?

A. Natural JOIN
B. Outer JOIN
C. EQUI JOIN
D. All of the above

8) Which SQL function is used to count the number of rows in a SQL query?

A. COUNT()
B. NUMBER()
C. SUM()
D. COUNT(*)

9) Which of the following is not a DDL command?

A. UPDATE
B. TRUNCATE
C. ALTER
D. None of the Mentioned

10) ___ is not a category of Database language.

A. TCL
B. SCL
C. DCL
D. DDL

11) Which of the following is a legal expression in SQL?

A. SELECT NULL FROM SALES;
B. SELECT NAME FROM SALES;
C. SELECT * FROM SALES WHEN PRICE = NULL;
D. SELECT # FROM SALES;

12) DCL provides commands to perform actions like

A. Change the structure of Tables
B. Insert, Update or Delete Records and Values
C. Authorizing Access and other control over Database
D. None of Above

13) The result of a SQL SELECT statement is a _____.

A. file
B. report
C. table
D. form

14) Which of the following do you need to consider when you make a table in SQL?

A. Data types
B. Primary keys
C. Default values
D. All of the above.

15) Create **table** student (name varchar ,id integer) What type of statement is this?

A. DML
B. DDL
C. DCL
D. TCL

16) Which of the following is not a DDL command?

A. ALTER
B. CREATE

C. UPDATE
D. TRUNCATE

17) Which of the following are TCL commands?
   A. COMMIT and ROLLBACK
   B. UPDATE and TRUNCATE
   C. SELECT and INSERT
   D. GRANT and REVOKE

18) Relational Algebra is a _____ query language that takes two relations as input and produces another relation as an output of the query.
   A. Relational
   B. Structural
   C. Procedural
   D. Fundamental

19) Which of the following is a fundamental operation in relational algebra?
   A. Set intersection
   B. Natural Join
   C. Assignment
   D. Select

20) The _____ operation, denoted by −, allows us to find tuples that are in one relation but are not in another.
   A. Union
   B. Set-difference
   C. Difference
   D. Intersection

ANSWER KEY

1.	B	2.	C	3.	D	4.	B	5.	B
6.	C	7.	B	8.	D	9.	A	10.	B
11.	C	12.	C	13	C	14.	D	15.	A
16	D	17	A	18	C	19	D	20	B

PART-II SAMPLE SHORT QUESTIONS

1. What is Relational Algebra?
2. What is SQL commands?
3. What is DDL?
4. What is DML?
5. What is SQL operator?
6. Define DCL.
7. What is join?
8. How many kinds of join? Write name them
9. What is the syntax of select operator?
10. What do average function do?
11. What is Cartesian product?
12. Define arithmetic operator in SQL
13. Define the projection operation
14. Define union with example.
15. Write any 5 name of logical operators.

PART-III SAMPLE LONG QUESTIONS

1) Explain database language. Also describe difference between DDL and DML.

2) Explain the Following terms with example.
    1. Equi join 2. Count function
3) Explain with example aggregate function.
4) Define SQL and write briefly about SQL operators.
5) Describe briefly about relational algebra and SQL in DBMS.

7 Chapter No. 7 (Database Life Cycle (DBLC))

Objectives

After completion of this chapter students will be able to understand:

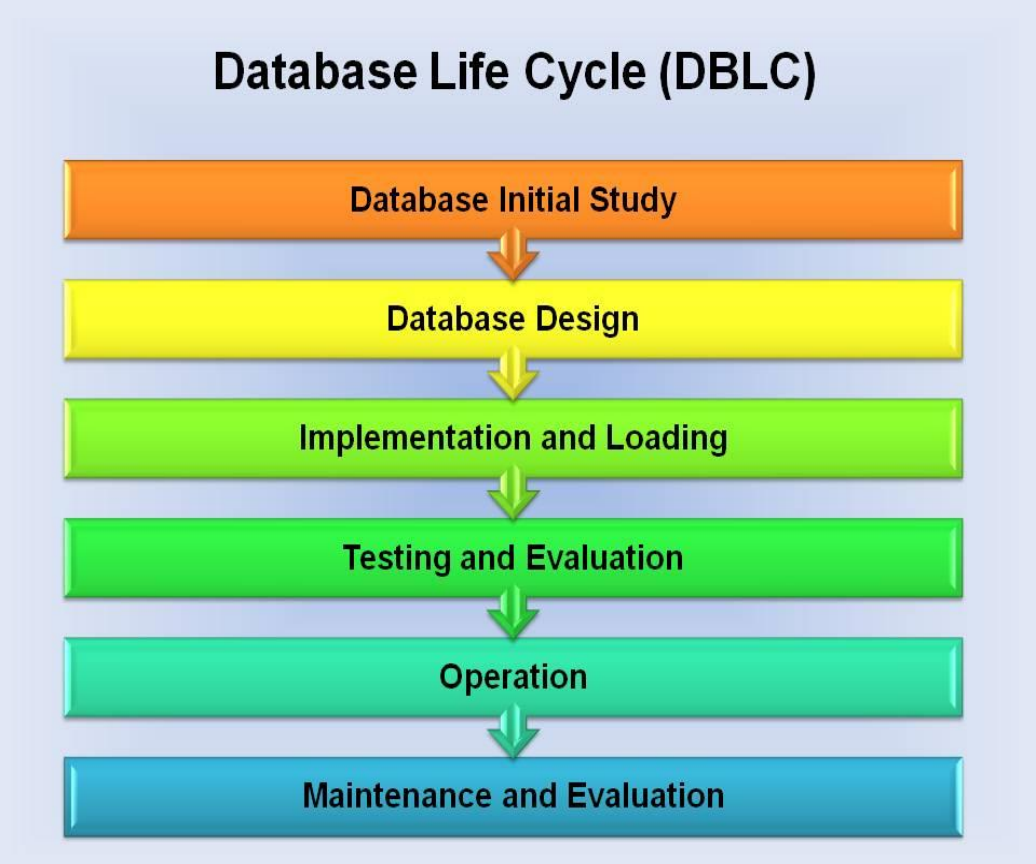
- ✓ Database Life Cycle (DBLC)
- ✓ Database Initial Study
- ✓ Database Design
- ✓ Database Design Strategies
- ✓ Centralized versus Decentralized Design

DATABASE LIFE CYCLE (DBLC)

Introduction:

The database life cycle is a cycle that traces the history of the database in an information system. The database life cycle incorporates the necessary steps involved in database development, starting with requirements analysis and ending with monitoring and modification. The DBLC never ends because database monitoring, improvement, and maintenance are part of the life cycle, and these activities continue as long as the database is alive and in use.

The Database Life Cycle (DBLC) contains six phases, as shown in the following Figure: database initial study, database design, implementation and loading, testing and evaluation, operation, and maintenance and evolution.



Phases of DBLC

The Database Initial Study

In the Database initial study, the designer must examine the current system's operation within the company and determine how and why the current system fails. The overall purpose of the database initial study is to:

1) Analyze the company situation.
2) Define problems and constraints.
3) Define objectives.
4) Define scope and boundaries.

7.1.1 Analyze the Company Situation:

The company situation describes the general conditions in which a company operates, its organizational structure, and its mission. To analyze the company situation, the database designer must discover what the company's operational components are, how they function, and how they interact.

7.1.2 Define Problems and Constraints:

The designer has both formal and informal sources of information. The process of defining problems might initially appear to be unstructured. Company end users are often unable to describe precisely the larger scope of company operations or to identify the real problems encountered during company operations.

7.1.3 Define Objectives:

A proposed database system must be designed to help solve at least the major problems identified during the problem discovery process. In any case, the database designer must begin to address the following questions:

- What is the proposed system's initial objective?
- Will the system interface with other existing or future systems in the company?
- Will the system share the data with other systems or users?

7.1.4 Define Scope and Boundaries:

The designer must recognize the existence of two sets of limits: scope and boundaries. The system's scope defines the extent of the design according to operational requirements. Will the database design encompass the entire organization, one or more departments within the organization, or one or more functions of a single department? Knowing the scope helps in defining the required data structures, the type and number of entities, the physical size of the database, and so on. The proposed system is also subject to limits known as boundaries, which are external to the system. Boundaries are also imposed by existing hardware and software.

Database Design

The second phase focuses on the design of the database model that will support company operations and objectives. This is arguably the most critical DBLC phase: making sure that the final product meets user and system requirements. As you examine the procedures required to complete the design phase in the DBLC, remember these points:

✓ The process of database design is loosely related to the analysis and design of a larger system. The data component is only one element of a larger information-system.

✓ The systems analysts or systems programmers are in charge of designing the other system components. Their activities create the procedures that will help transform the data within the database into useful information.

7.1.5 Implementation and Loading:

The output of the database design phase is a series of instructions detailing the creation of tables, attributes, domains, views, indexes, security constraints, and storage and performance guidelines. In this phase, you actually implement all these design specifications.

1) Install the DBMS:

This step is required only when a new dedicated instance of the DBMS is necessary for the system. The DBMS may be installed on a new server or it may be installed on existing servers. One current trend is called virtualization. Virtualization is a technique that creates logical representations of computing resources that are independent of the underlying physical computing resources.

2) Create the Database(s):

In most modern relational DBMSs, a new database implementation requires the creation of special storage-related constructs to house the end-user tables. The

constructs usually include the storage group (or file groups), the table spaces, and the tables.

3) Load or Convert the Data:

After the database has been created, the data must be loaded into the database tables. Typically, the data will have to be migrated from the prior version of the system. Often, data to be included in the system must be aggregated from multiple sources. Data may have to be imported from other relational databases, non-relational databases, flat files, legacy systems, or even manual paper-and-pencil systems

7.1.6 Testing and Evaluation:

In the design phase, decisions were made to ensure integrity, security, performance, and recoverability of the database. During implementation and loading, these plans were put into place. In testing and evaluation, the DBA tests and fine-tunes the database to ensure that it performs as expected. This phase occurs in conjunction with applications programming.

1) Test the Database:

During this step, the DBA tests the database to ensure that it maintains the integrity and security of the data. Data integrity is enforced by the DBMS through the proper use of primary and foreign key rules. In database testing you must check Physical security allows, Password security, Access rights, Data encryption-etc.

2) Fine-Tune the Database:

Although database performance can be difficult to evaluate because there are no standards for database performance measures, it is typically one of the most important factors in database implementation. Different systems will place different performance requirements on the database. Many factors can impact the database's performance on various tasks. Environmental factors, such as the hardware and software environment in which the database exists, can have a significant impact on database performance.

3) Evaluate the Database and Its Application Programs:

As the database and application programs are created and tested, the system must also be evaluated from a more holistic approach. Testing and evaluation of the individual components should culminate in a variety of broader system tests to ensure that all of the components interact properly to meet the needs of the users. To ensure that the data contained in the database are protected against loss, backup and recovery plans are tested.

7.1.7 Operation

Once the database has passed the evaluation stage, it is considered to be operational. At that point, the database, its management, its users, and its application programs constitute a complete information system. The beginning of the operational phase invariably starts the process of system evolution.

7.1.8 Maintenance and Evolution

The database administrator must be prepared to perform routine maintenance activities within the database. Some of the required periodic maintenance activities include:

- ✓ Preventive maintenance (backup).
- ✓ Corrective maintenance (recovery).
- ✓ Adaptive maintenance (enhancing performance, adding entities and attributes, and so on).
- ✓ Assignment of access permissions and their maintenance for new and old users.

7.3.3 Database Design Approaches

Based on the results of the data requirements analysis, carry out the design process. There are two database design approaches, namely:

The top-down design uses an entity-relationship (ER) model. The design begins with identifying entities, followed by relationships between entities and cardinality

or multiplicity. Each entity consists of attributes, primary keys, and foreign keys (if any).

The bottom-up design uses the process of normalization. Design starts with identifying attributes, then grouping them into data sets to form relations.

The two approaches are complementary. The design process begins with conceptual data modeling and continues to the logical database design and physical database design stages. The ER model is used in the conceptual data modeling stage. In the early ER model, normalization was carried out on entities or *relationships* with multiple data attributes. The normalization results are used to modify the initial ER model to obtain a better final ER model. In the logical database design stage, normalization is carried out on the mapping result relations of ER modeling, which has multiple data.

The two approaches are complementary. The design process begins with conceptual data modeling and continues to the logical database design and physical database design stages. The conceptual data modeling uses the ER model. In the early ER model, carry out normalization on entities or relationships that have data redundancies. Use the normalization results to obtain a better final ER model.

1) Conceptual Data Modeling

Conceptual data modeling is free from implementation, hardware, software, operating systems, DBMS, application programs, programming languages, etc. Later, it can be implemented on any platform. Conceptual data modeling using the entity-relationship (ER) model must represent existing business functions in the organization and describe the users' data requirements entirely and accurately.

2) Logical Database Design

Based on the conceptual data model and mapping rules, every entity and *relationship* with attributes are converted into relations. Relationships that have attribute groups with data redundancies result in anomalies when adding, updating, or deleting data. Such tables need to be normalized, at least up to 3NF (*third normal form*).

Each relation attribute is determined by its data type and domain, including whether the data must be unique or not. The result is a specification for each relation.

3) Physical Database Design

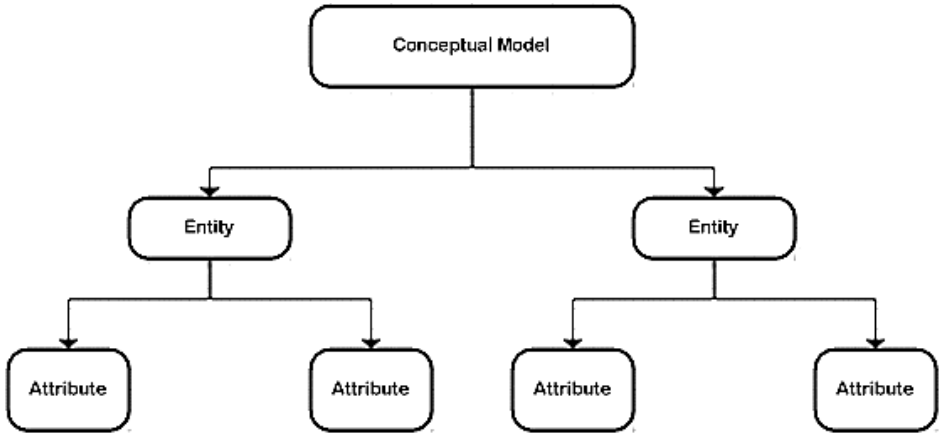
Physical database design requires knowledge of the specific DBMS that will be used to implement the database. In the design and definition of physical databases, *records* organization, *file* organization, and use of indexes are determined. The goal is to design a data store that provides adequate performance and ensures proper database integrity, security, and recovery. For example, a dominant process with high frequency, high volume, or explicit priority can be improved by denormalization. Thus, physical database design is carried out in coordination with other aspects: programs, computer hardware, operating systems, and data communication networks.

Database Design Strategies

There are two approaches for developing any database, the top-down method and the bottom-up method. While these approaches appear radically different, they share the common goal of utilizing a system by describing all of the interaction between the processes.

7.1.9 Top – down design method

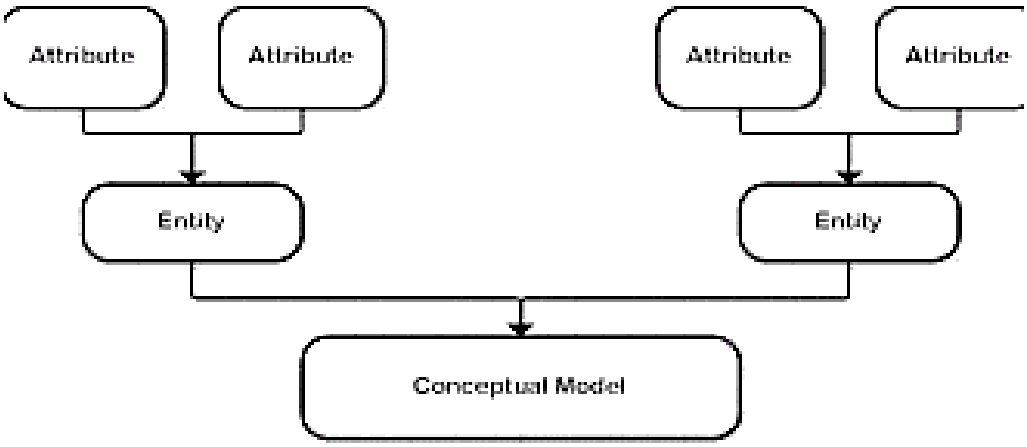
The top-down design method starts from the general and moves to the specific. In other words, you start with a general idea of what is needed for the system and then work your way down to the more specific details of how the system will interact. This process involves the identification of different entity types and the definition of each entity's attributes.



7.1.10 Bottom – up design method

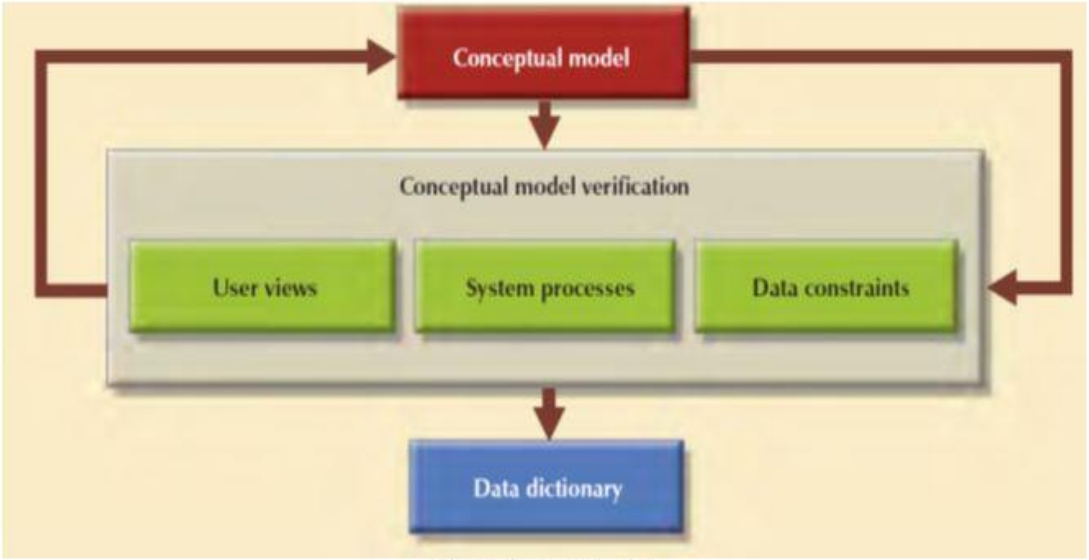
The bottom-up approach begins with the specific details and moves up to the general. This is done by first identifying the data elements (items) and then grouping them together in data sets. In other words, this method first identifies the attributes, and then groups them to form entities.

Two general approaches (top – down and bottom – up) to the design of the databases can be heavily influenced by factors like scope, size of the system, the organizations management style, and the organizations structure. Depending on such factors, the design of the database might use two very different approaches, centralized design and decentralized design.



Centralized design

Centralized design is most productive when the data component is composed of a moderately small number of objects and procedures. The design can be carried out and represented in a somewhat simple database. Centralized design is typical of a simple or small database and can be successfully done by a single database administrator or by a small design team. This person or team will define the problems, create the conceptual design, verify the conceptual design with the user views, and define system processes and data constraints to ensure that the design complies with the organizations goals. That being said, the centralized design is not limited to small companies. Even large companies can operate within the simple database environment.

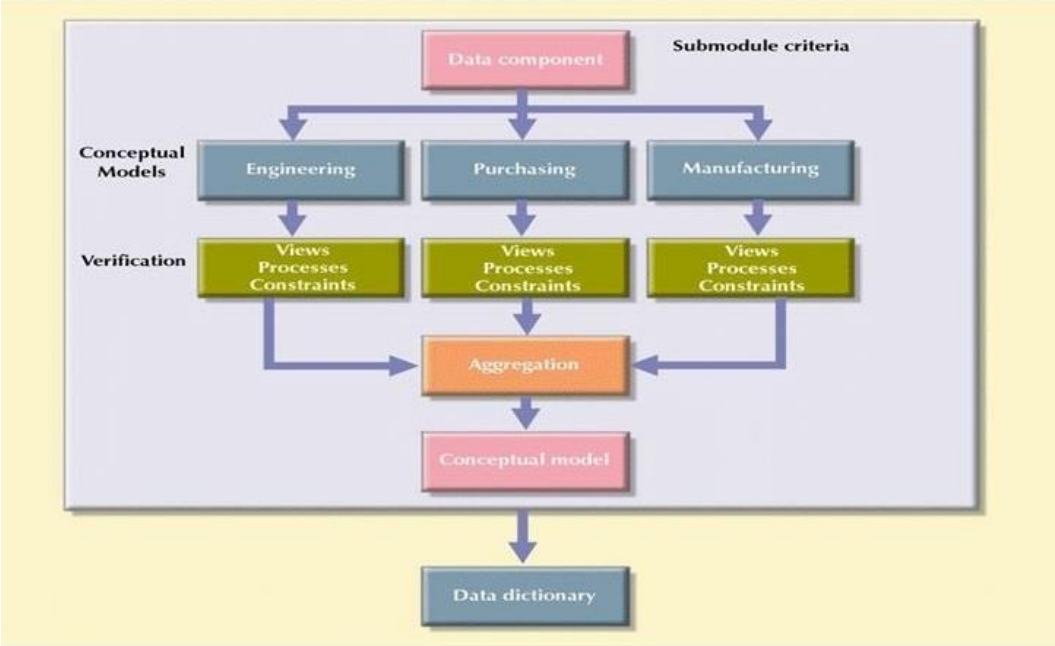


Centralized design

7.1.11 Decentralized design

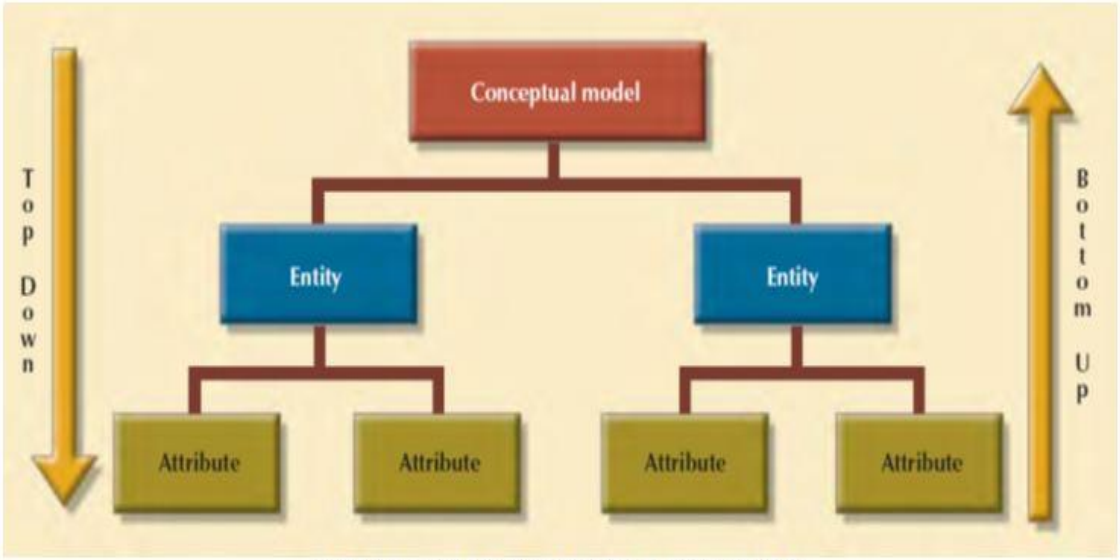
Decentralized design might best be used when the data component of the system has a large number of entities and complex relations upon which complex

operations are performed. This is also likely to be used when the problem itself is spread across many operational sites and the elements are a subset of the entire data set. In large and complex projects, a team of carefully selected designers are employed to get the job done. This is commonly accomplished by several teams that work on different subsets or modules of the system. Conceptual models are created by these teams and compared to the user views, processes, and constraints for each module. Once all the teams have completed their modules, they are all put aggregated into one large conceptual model.



Decentralized design

7.1.12Centralized versus Decentralized Design



Top-down vs. bottom-up design sequencing

CENTRALIZED DATABASE VERSUS DISTRIBUTED DATABASE

CENTRALIZED DATABASE	DISTRIBUTED DATABASE
A type of database that contains a single database located at one location in the network	A type of database that contains two or more database files located at different locations in the network
Managing, updating and taking in backups of data is easier because there is only one database file	As there are multiple database files in a distributed database, it requires time to synchronize data
Requires time for accessing data because multiple users access the database file	Speed in accessing the data is higher because the data is retrieved from the nearest database file
If the database fails, the users do not have access to a database	If one database fails, the users can still access other database files
Has more data consistency and it provides the complete view to the user	Can have data replications, and there can be some data inconsistency

EXERCISE No. 07

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

2. The _____ is a cycle that traces the history of the database in an information system.
   a) Database design
   b) Database architecture
   c) Database life cycle
   d) Database
3. Producing the required information flow is part of the _____ phase of the Database Life Cycle (DBLC).
   a) Operation
   b) Database Design
   c) Implementation and Loading
   d) None of these
4. The _____ design uses an entity-relationship (ER) model
   a) Top-down design
   b) Bottom-up design
   c) Centralized design
   d) Decentralized design
5. The _____ design uses the process of normalization
   a) Top-down design
   b) Bottom-up design
   c) Centralized design
   d) Decentralized design
6. _____ is free from implementation, hardware, software, operating systems, DBMS, application programs, programming languages, etc.
   a) Physical data modeling
   b) Conceptual data modeling
   c) Both a and b
   d) None of these

7. In the design and definition of _____ *records* organization, *file* organization, and use of indexes are determined
   a) Physical database
   b) Logical database
   c) Primary database
   d) Step up database
8. The_____ method starts from the general and moves to the specific.
   a) Top-down design
   b) Bottom-up design
   c) Centralized design
   d) Decentralized design
9. The _____ begins with the specific details and moves up to the general.
   a) Top down design
   b) Bottom up design
   c) Centralized design
   d) Decentralized design
10. _____ is typical of a simple or small database and can be successfully done by a single database administrator or by a small design team
    a) Top down design
    b) Bottom up design
    c) Centralized design
    d) Decentralized design
11. Each relation attribute is determined by its data type and _____.
    a) Range
    b) Entity
    c) Domain
    d) Tuple
12. In DBLC, the phase after the database initial study is _____.
    a) Analysis
    b) Database Design
    c) Implementation and Loading
    d) Operation
13. The implementation and loading phase of the Database Life Cycle (DBLC) involves _____.
    a) Installing the DBMS

b) Problems and constraints
c) Test the Database
d) Database Design

ANSWER KEY

1.	c	2.	A	3.	a	4.	b	5.	b
6.	a	7.	A	8.	b	9.	c	10.	c
11.	b	12.	A						

PART-II SAMPLE SHORT QUESTIONS

1. What is database life cycle?
2. What are the six phases of database life cycle?
3. What are the two basic database designs strategies?
4. What is top-down design?
5. What is bottom-up design?
6. What is centralized design with diagram?
7. What is decentralized design?
8. What is Testing and Evaluation
9. Define Conceptual Data Modeling
10. Define Logical Database Design
11. Define Physical Database Design
12. Define Implementation and Loading.
13. Draw a diagram of Centralized versus Decentralized Design
14. What is Database Design?
15. What is importance of Maintenance and Evolution?

PART-III SAMPLE LONG QUESTIONS

Q1. Write a note on initial study of database life cycle.
Q2. Write a detail note on database life cycle

Q3. Define database design and its stages in detail.
Q4. What is the difference between top-down design and bottom-up design?
Q5. What is the difference between centralized and decentralized deign?

8 Chapter No. 8 (Entity Relationship (E-R) Modeling)

Objectives

After completion of this chapter students will be able to understand:

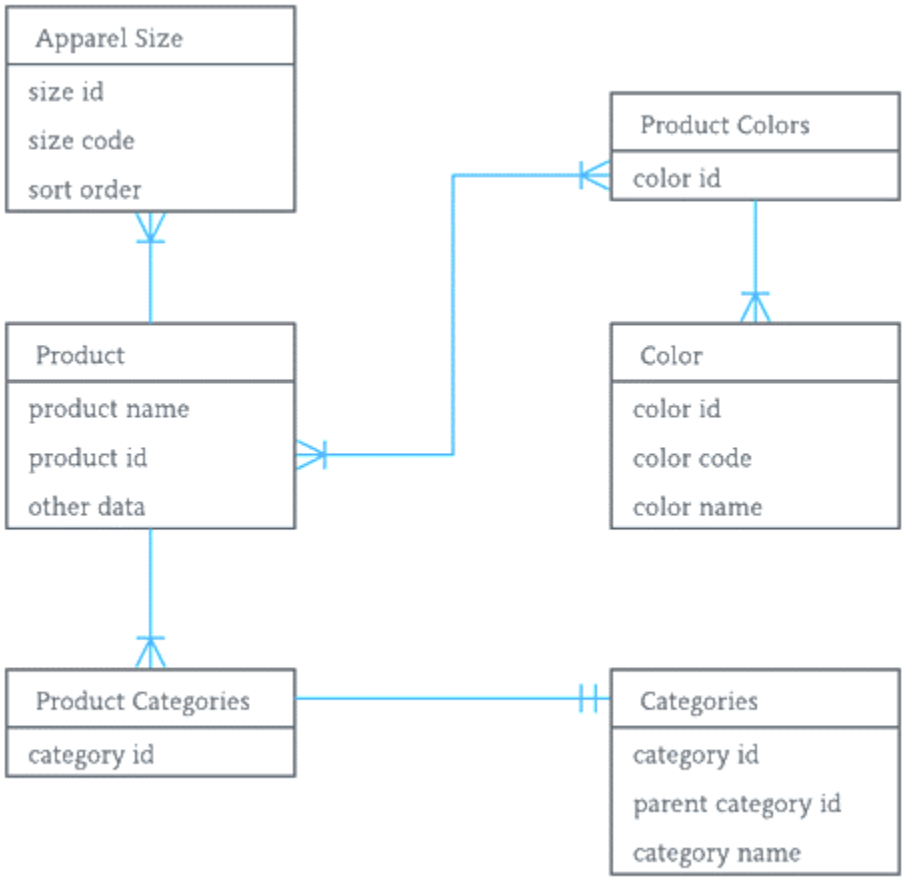
- ✓ Entity Relationship (E-R) Modeling
- ✓ Basic Modeling Concepts
- ✓ Degrees of Data Abstraction
- ✓ Association and Cardinality
- ✓ Relationship Participation
- ✓ Composite Entities, Entity Super types and subtypes
- ✓ Enhanced Entity Relationship Diagram
- ✓ Transform ER/EER to Relational Model

8.1 Entity Relationship (E-R) Modeling

ER Diagram: stands for Entity Relationship Diagram, also known as ERD is a diagram that displays the relationship of entity sets stored in a database. In other words, ER diagrams help to explain the logical structure of databases. ER diagrams are created based on three basic concepts: entities, attributes and relationships.

ER Diagrams contain different symbols that use rectangles to represent entities, ovals to define attributes and diamond shapes to represent relationships.

At first look, an ER diagram looks very similar to the flowchart. However, ER Diagram includes many specialized symbols, and its meanings make this model unique. The purpose of ER Diagram is to represent the entity framework infrastructure.



Entity Relationship Diagram Example

8.2 Basic Modeling Concepts

8.2.1 ER model:

ER model stands for an Entity-Relationship model. It is a high-level data model. This model is used to define the data elements and relationship for a specified system.

It develops a conceptual design for the database. It also develops a very simple and easy to design view of data. ER Modeling helps you to analyze data requirements systematically to produce a well-designed database. In ER modeling, the database structure is portrayed as a diagram called an entity-relationship diagram.

8.2.2 History of ER models

ER diagrams are visual tools that are helpful to represent the ER model. Peter Chen proposed ER Diagram in 1971 to create a uniform convention that can be used for relational databases and networks. He aimed to use an ER model as a conceptual modeling approach.

8.2.3 Why use ER Diagrams?

Here, are prime reasons for using the ER Diagram

1) Helps you to define terms related to entity relationship modeling
2) Provide a preview of how all your tables should connect, what fields are going to be on each table
3) Helps to describe entities, attributes, relationships
4) ER diagrams are translatable into relational tables which allows you to build databases quickly
5) ER diagrams can be used by database designers as a blueprint for implementing data in specific software applications
6) The database designer gains a better understanding of the information to be contained in the database with the help of ERP diagram
7) ERD Diagram allows you to communicate with the logical structure of the database to users

8.2.4 Facts about ER Diagram Model

1) ER model allows you to draw Database Design
2) It is an easy-to-use graphical tool for modeling data
3) Widely used in Database Design
4) It is a GUI representation of the logical structure of a Database

5) It helps you to identify the entities which exist in a system and the relationships between those entities.

8.2.5 ER Diagrams Symbols & Notations

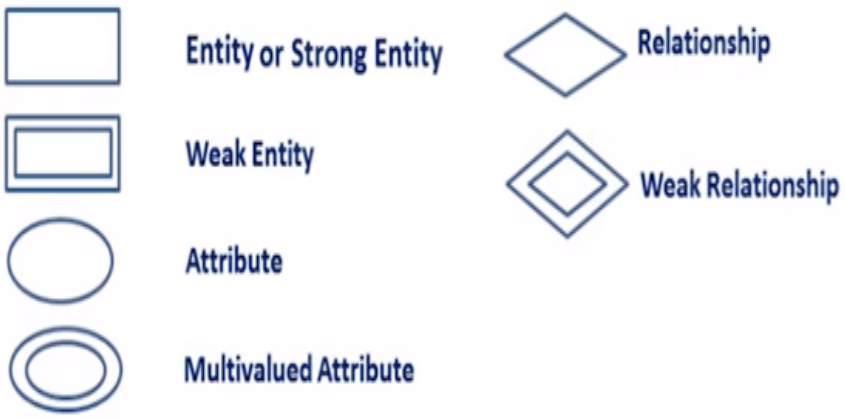
Entity Relationship Diagram Symbols & Notations mainly contains three basic symbols which are rectangle, oval and diamond to represent relationships between elements, entities and attributes. There are some sub-elements which are based on main elements in ERD Diagram. ER Diagram is a visual representation of data that describes how data is related to each other using different ERD Symbols and Notations.

Following are the main components and its symbols in ER Diagrams:

- ✓ **Rectangles:** This Entity Relationship Diagram symbol represents entity types
- ✓ **Ellipses:** Symbol represent attributes
- ✓ **Diamonds:** This symbol represents relationship types
- ✓ **Lines:** It links attributes to entity types and entity types with other relationship types

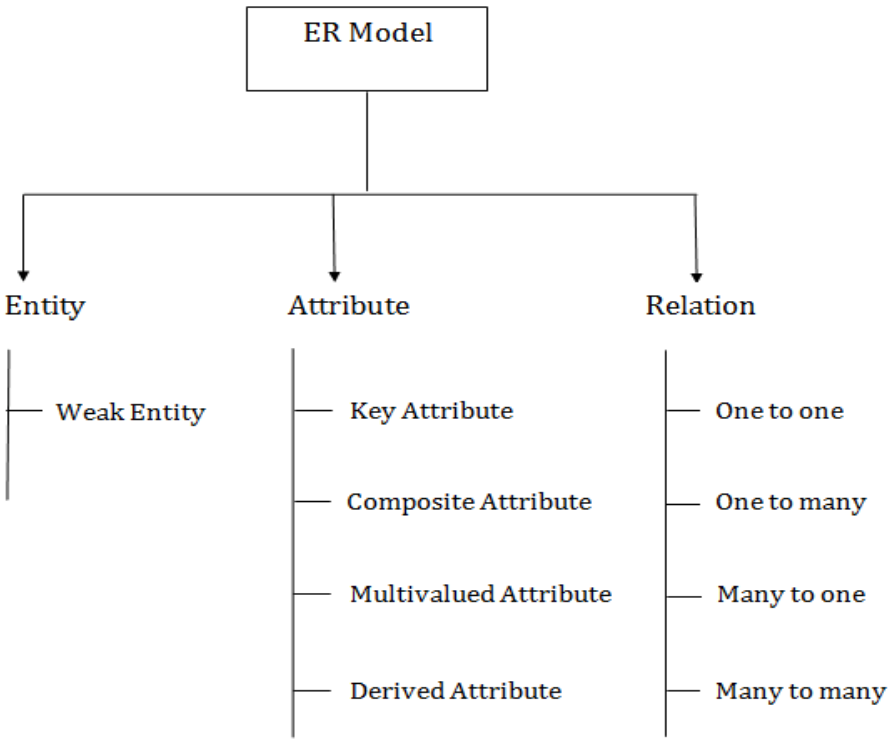
Primary key: attributes are underlined

Double Ellipses: Represent multi-valued attributes



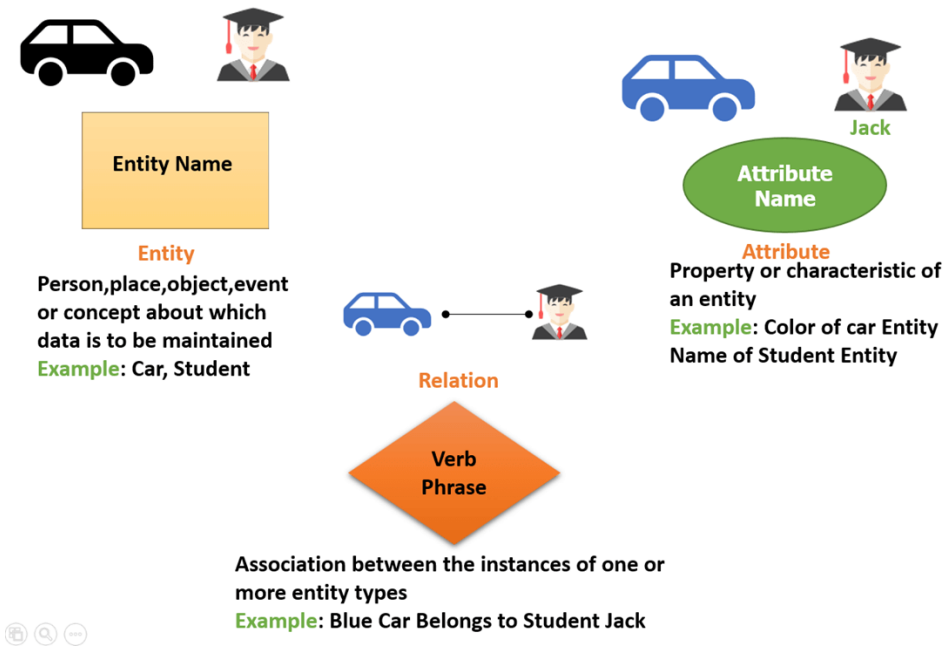
Component of ER Diagram 1

8.2.6 ER Model



- **ER Diagram Examples**

For example, in a university database, we might have entities for Students, Courses, and Lecturers. Student's entity can have attributes like Rollno, Name, and DeptID. They might have relationships with Courses and Lecturers.



8.3 What is Entity?

A real-world thing either living or non-living that is easily recognizable and non-recognizable. It is anything in the enterprise that is to be represented in our database. It may be a physical thing or simply a fact about the enterprise or an event that happens in the real world.

An entity can be place, person, object, event or a concept, which stores data in the database. The characteristics of entities are must have an attribute, and a unique key. Every entity is made up of some 'attributes' which represent that entity.

➢ Examples of entities:

Person: Employee, Student, Patient
Place: Store, Building
Object: Machine, product, and Car
Event: Sale, Registration, Renewal

Concept: Account, Course

8.3 Entity set:

An entity set is a group of similar kind of entities. It may contain entities with attribute sharing similar values. Entities are represented by their properties, which also called attributes. All attributes have their separate values. For example, a student entity may have a name, age, class, as attributes.



Student Entity Example 1

➢ Example of Entities:

A university may have some departments. All these departments employ various lecturers and offer several programs.

Some courses make up each program. Students register in a particular program and enroll in various courses. A lecturer from the specific department takes each course, and each lecturer teaches a various group of students.

8.5 Degrees of Data Abstraction:

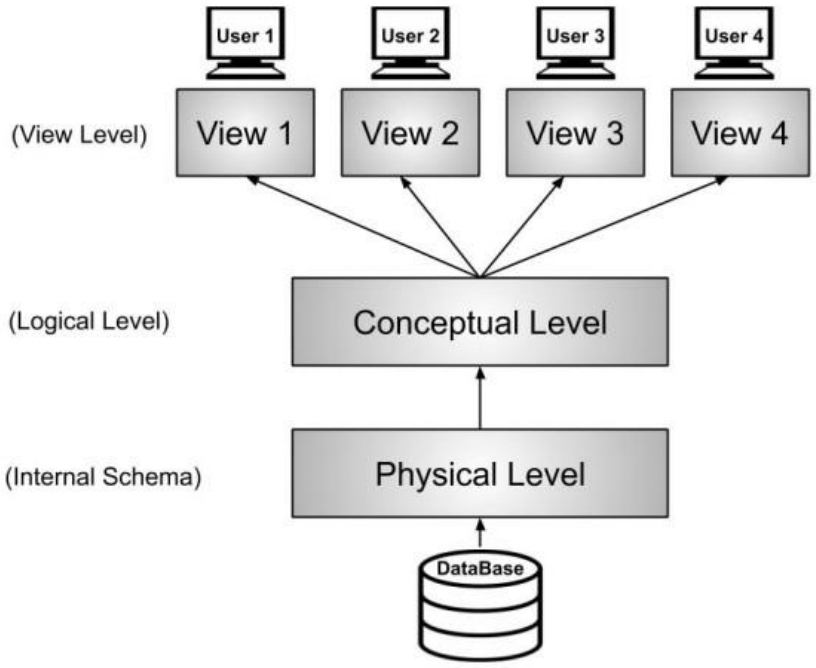
Data Abstraction refers to the process of hiding irrelevant details from the user. The database designer starts with an abstract view of the overall data environment and adds details as the design comes closer to implementation.

Data abstraction in DBMS can also be very helpful in integrating multiple (and sometimes conflicting) views of data as seen at different levels of an organization. In 1970, the American National Standards Institute (ANSI) Standards Planning and Requirements Committee (SPARC) established a framework for database design

based on the degrees of abstraction. The ANSI/SPARC architecture is composed of four levels of data abstraction; these levels are external, conceptual, internal, and physical.

8.5.1 Three levels of data abstraction:

1) View Level
2) Conceptual Level
3) Physical Level



Levels of Data Abstraction

8.5.2 View Level or External Schema

This level tells the application about how the data should be shown to the user.

Example:

If we have a login-id and password in a university system, then as a student, we can view our marks, attendance, fee structure, etc. But the faculty of the university will have a different view. He will have options like salary, edit marks

of a student, enter attendance of the students, etc. So, both the student and the faculty have a different view. By doing so, the security of the system also increases. In this example, the student can't edit his marks but the faculty who is authorized to edit the marks can edit the student's marks. Similarly, the dean of the college or university will have some more authorization and accordingly, he will has his view. So, different users will have a different view according to the authorization they have.

8.5.3 Conceptual Level or Logical Level

This level tells how the data is actually stored and structured. We have different data models by which we can store the data

Example:

Let us take an example where we use the relational model for storing the data. We have to store the data of a student, the columns in the student table will be student_name, age, mail_id, roll_no etc. We have to define all these at this level while we are creating the database. Though the data is stored in the database but the structure of the tables like the student table, teacher table, books table, etc are defined here in the conceptual level or logical level. Also, how the tables are related to each other are defined here. Overall, we can say that we are creating a blueprint of the data at the conceptual level.

8.5.4 Physical Level or Internal Schema

As the name suggests, the Physical level tells us that where the data is actually stored i.e. it tells the actual location of the data that is being stored by the user. The Database Administrators (DBA) decide that which data should be kept at which particular disk drive, how the data has to be fragmented, where it has to be stored etc. They decide if the data has to be centralized or distributed. Though we see the data in the form of tables at view level the data here is actually stored in the form of files only. It totally depends on the DBA, how he/she manages the database at the physical level.

8.6 Association and Cardinality

8.6.1 Association:

An association defines a relationship between two entity objects based on common attributes. Association is a relationship between two objects. In other words, association defines the multiplicity between objects. You may be aware of one-to-one, one-to-many, many-to-one, many-to-many all these words define an association between objects.

- **Relationship:**

Relationship is nothing but an association among two or more entities. E.g., Tom works in the Chemistry department.



Entities take part in relationships. We can often identify relationships with verbs or verb phrases.

For example:

You are attending this lecture
I am giving the lecture
A student attends a lecture
A lecturer is giving a lecture.

8.6.2 Cardinality

Defines the numerical attributes of the relationship between two entities or entity sets.

Different types of cardinal relationships are:

1) One-to-One Relationships

2) One-to-Many Relationships
3) May to One Relationships
4) Many-to-Many Relationships

8.6.2.1 One-to-one:

One entity from entity set X can be associated with at most one entity of entity set Y and vice versa. Example: One student can register for numerous courses. However, all those courses have a single line back to that one student.



8.6.2.2 One-to-many:

One entity from entity set X can be associated with multiple entities of entity set Y, but an entity from entity set Y can be associated with at least one entity.

For example, one class is consisting of multiple students.



8.6.2.3 Many to One

More than one entity from entity set X can be associated with at most one entity of entity set Y. However, an entity from entity set Y may or may not be associated with more than one entity from entity set X.

For example, many students belong to the same class.



8.6.2.4 Many to Many:

One entity from X can be associated with more than one entity from Y and vice versa. For example, Students as a group are associated with multiple faculty members, and faculty members can be associated with multiple students.



8.7 Relationship Participation

In a Relationship, Participation constraint specifies the existence of an entity when it is related to another entity in a relationship type. It is also called minimum cardinality constraint.

This constraint specifies the number of instances of an entity that can participate in a relationship type.

There are two types of Participation constraint

8.7.1 Total Participation

Each entity in the entity set is involved in at least one relationship in a relationship set i.e. the number of relationship in every entity is involved is greater than 0.

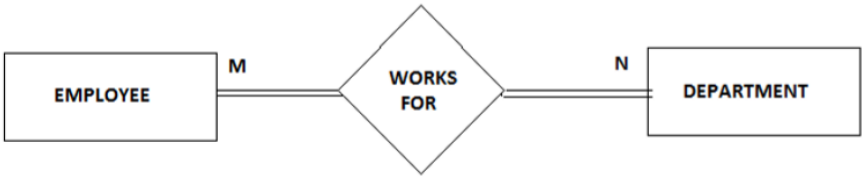


Figure: Total participation

Consider two entities Employee and Department related via Works For relationship. Now, every Employee works in at least one department therefore an employee entity exists if it has at least one Works For relationship with Department entity. Thus, the participation of Employee in Works For is total relationship.

Total Participation is represented by double line in ER diagram.

8.7.2 Partial Participation

Each entity in entity set may or may not occur in at least one relationship in a relationship set.

For example:

Consider two entities Employee and Department and they are related to each other via Manages relationship. An Employee must manage a department, he or she could be the head of the department. But not every Employee in the company manages the department. So, participation of employee in the Manages relationship type is partial i.e., only a particular set of Employees will manage the Department but not all.

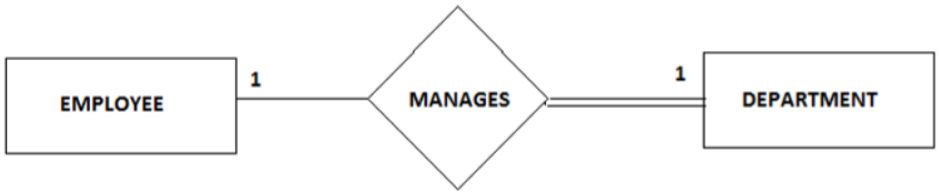


Figure: Partial Participation

8.8 Composite Entities, Entity Super types and subtypes

Database Entities in E/R Modeling

General Definition of Entity

An entity can be a real-world object, either animate or inanimate, that can be easily identifiable. For example, in a school database, students, teachers, classes, and courses offered can be considered as entities. All these entities have some attributes or properties that give them their identity.

An entity in terms of an E/R Model is an entity set, which is a set of entities all which are of the same type. This means that there is not just one particular occurrence of that entity, but a number of occurrences about the same thing of interest. An entity in database is a noun; that is a person, place, thing, or idea. There are different kinds of entities that an E/R Model can hold. They are traditional entities, composite entities, entities of the subtype/supertype, and strong/weak entities.

8.8.1 Traditional Entity:

The traditional entity, also known as the simple entity, is just how it sounds it would be: The typical normal entity. This generally only has one primary key associated with it. This is never associated with many-to-many type relationships.

8.8.2 Composite Entity:

A composite entity is also known as a “bridge” entity. This “bridge” is used to handle the many-to-many relationships that the traditional entity could not handle. This entity lies between the two entities that are of interest and this composite entity shares the primary keys from both the connecting tables. This composite entity is also known as a “gerund” because it has the characteristics of an entity and a relationship.

8.8.3 Subtype/Supertype Entity:

A subtype/supertype is just what you would expect to see. That is the generic parent-child relationship. The supertype (parent) entity is the top most entity that shares the information down to the subtypes (children). The subtypes inherit all the information from the supertype entity. When moving down the hierarchical supertype to the subtype, that is known as specialization. When moving from the subtype to the supertype, that is known as generalization.

8.8.4 Weak Entity:

A weak entity is an entity that both cannot survive without the other entity that it shares a relationship with and also contains a primary key that is either partially of fully derived from the parent entity.

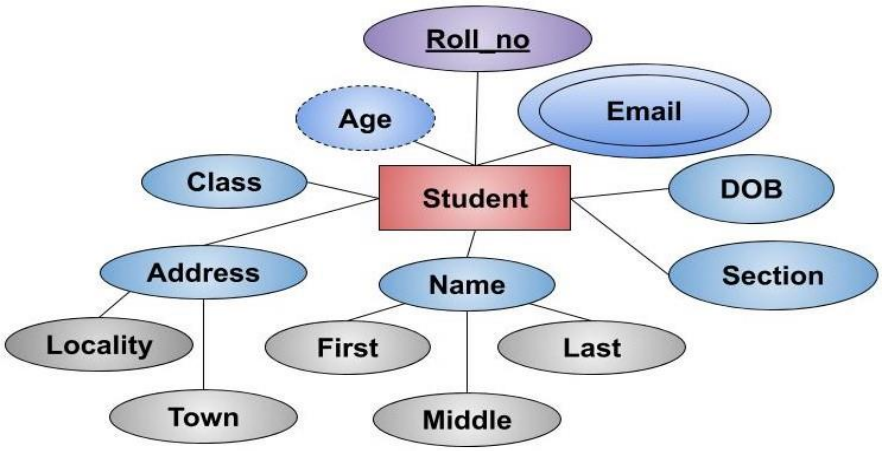
8.8.5 Strong Entity:

A strong entity is the standard database entity that has existence independence, meaning can stand alone whether another entity exists or not.

8.8.6 Attributes:

Entities are represented by means of their properties, called attributes. All attributes have values. For example, a student entity may have name, class, and age as attributes.

There exists a domain or range of values that can be assigned to attributes. For example, a student's name cannot be a numeric value. It has to be alphabetic. A student's age cannot be negative, etc.



8.8.7 Types of Attributes

1) **Simple attribute** − Simple attributes are atomic values, which cannot be divided further. For example, a student's phone number is an atomic value of 10 digits.
2) **Composite attribute** − Composite attributes are made of more than one simple attribute. For example, a student's complete name may have first_name and last_name.
3) **Derived attribute** − Derived attributes are the attributes that do not exist in the physical database, but their values are derived from other attributes present in the database. For example, average salary in a department should not be saved directly in the database, instead it can be derived. For another example, age can be derived from data_of_birth.
4) **Single-value attribute** − Single-value attributes contain single value. For example − Social_Security_Number.
5) **Multi-value attribute −** Multi-value attributes may contain more than one values. For example, a person can have more than one phone number, email address, etc.

8.9 Enhanced Entity Relationship Diagram

8.9.1 Enhanced ER Model

Enhanced entity-relationship diagrams are advanced database diagrams very similar to regular ER diagrams which represent requirements and complexities of complex databases. It is a diagrammatic technique for displaying the Sub Class and Super Class; Specialization and Generalization; Union or Category; Aggregation etc. EER is a high-level data model that incorporates the extensions to the original ER model.
It is a diagrammatic technique for displaying the following concepts

1) Sub Class and Super Class
2) Specialization and Generalization
3) Union or Category
4) Aggregation

These concepts are used when these comes in EER schema and the resulting schema diagrams called as EER Diagrams.

8.9.2 Features of EER Model

1) EER creates a design more accurate to database schemas.
2) It reflects the data properties and constraints more precisely.
3) It includes all modeling concepts of the ER model.
4) Diagrammatic technique helps for displaying the EER schema.
5) It includes the concept of specialization and generalization.
6) It is used to represent a collection of objects that is union of objects of different of different entity types.

8.9.3 Sub Class and Super Class

Sub class and Super class relationship leads the concept of Inheritance.

The relationship between sub class and super class is denoted with  symbol.

1. Super Class

Super class is an entity type that has a relationship with one or more subtypes. An entity cannot exist in database merely by being member of any super class. For example: Shape super class is having sub groups as Square, Circle, and Triangle.

2. Sub Class

Sub class is a group of entities with unique attributes. Sub class inherits properties and attributes from its super class. For example: Square, Circle, Triangle are the sub class of Shape super class.

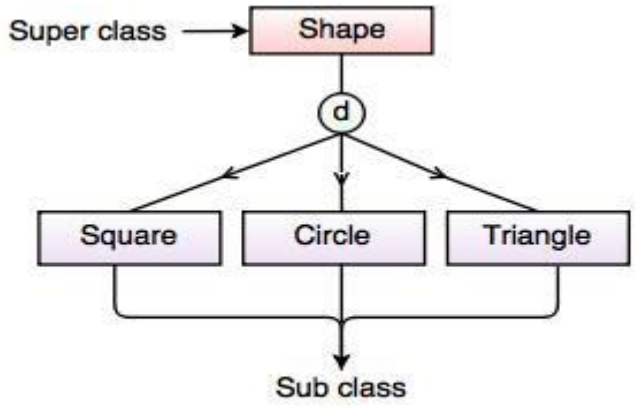


Fig. Super class/Sub class Relationship

1. Generalization

Generalization is the process of generalizing the entities which contain the properties of all the generalized entities. It is a bottom approach, in which two lower-level entities combine to form a higher-level entity. Generalization is the reverse process of Specialization. It defines a general entity type from a set of specialized entity type.

It minimizes the difference between the entities by identifying the common features.
For example:

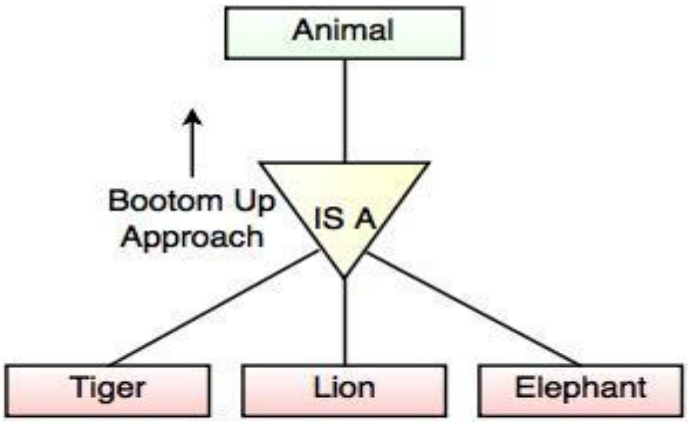


Fig. Generalization

In the above example, Tiger, Lion, Elephant can all be generalized as Animals.

2. Specialization

Specialization is a process that defines a group entity which is divided into sub groups based on their characteristic. It is a top-down approach, in which one higher entity can be broken down into two lower-level entities. It maximizes the difference between the members of an entity by identifying the unique characteristic or attributes of each member. It defines one or more sub class for the super class and also forms the superclass/subclass relationship. For example

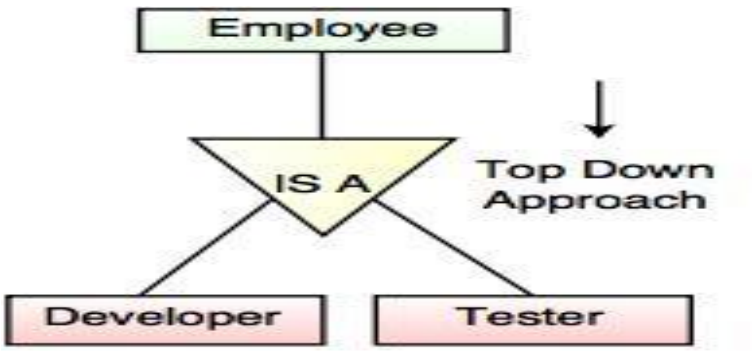


Fig. Specialization

In the above example, Employee can be specialized as Developer or Tester, based on what role they play in an organization.

8.10 Category or Union

Category represents a single super class or sub class relationship with more than one super class. It can be a total or partial participation. For example Car booking, Car owner can be a person, a bank (holds a possession on a Car) or a company. Category (sub class) → Owner is a subset of the union of the three super classes → Company, Bank, and Person. A Category member must exist in at least one of its super classes.

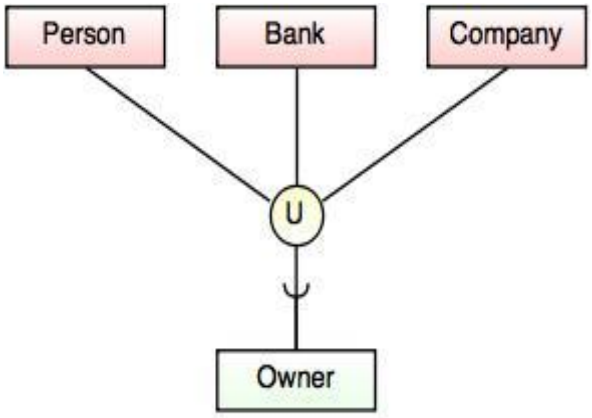


Fig. Categories (Union Type)

8.11 Aggregation

Aggregation is a process that represent a relationship between a whole object and its component parts. It abstracts a relationship between objects and viewing the relationship as an object. It is a process when two entity is treated as a single entity.

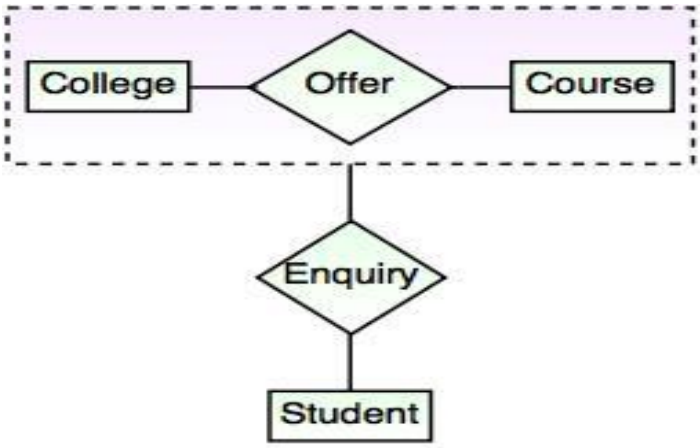


Fig. Aggregation

In the above example, the relation between College and Course is acting as an Entity in Relation with Student.

8.12 Transform ER/EER to Relational Model

1) Relational Model (RM)

In 1970, E.F. Codd developed the relational model. He proposed this model as well as a non-procedural approach for modeling data in the form of relations or tables. In the Relational Model, tables are usually interpreted as relations. If we model the database using ER diagrams, we must convert them into the relational model, which can be implemented by one of the RDBMS languages such as SQL and MySQL.

Relational model represents the database as a collection of relations. A relation is nothing but a table of values. Every row in the table represents a collection of related data values. These rows in the table denote a real-world entity or relationship.

In relational model, the data and relationships are represented by collection of inter-related tables. Each table is a group of column and rows, where column represents attribute of an entity and rows represents records.

Sample relational Model: Student table with 4 columns and 4 records.

Table: Student

Student_Id	Student_Name	Student_Age	Student_Address
111	Aisha	23	Lahore
333	Layba	20	Gujranwala
555	Aleeha	21	Multan

666	Sana	22	Faisalabad

Relational Model table

Here Student_Id, Student_Name, Student_Age & Student_Address are attributes of table Student. The rows with values are the records (commonly known as tuples).

- **Importance of ER Model**

ER model stands for the Entity-Relationship model that Peter Chen developed in 1976.

ER Model, when conceptualized into diagrams, gives a good overview of entity-relationship, which is easier to understand. ER diagrams can be mapped to relational schema, that is, it is possible to create relational schema using ER diagram. We cannot import all the ER constraints into relational model, but an approximate schema can be generated.

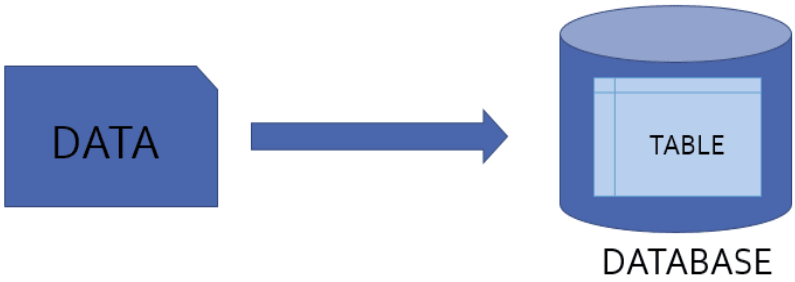
There are several processes and algorithms available to convert ER Diagrams into Relational Schema. Some of them are automated and some of them are manual. We may focus here on the mapping diagram contents to relational basics.

ER diagrams mainly comprise of −

- ✓ Entity and its attributes
- ✓ Relationship, which is association among entities.

2) Transformation of ER/EER into Relational Model

First step of any relational database design is to make ER Diagram for it and then convert it into relational Model. Relational Model represents how data is stored in database in the form of table.

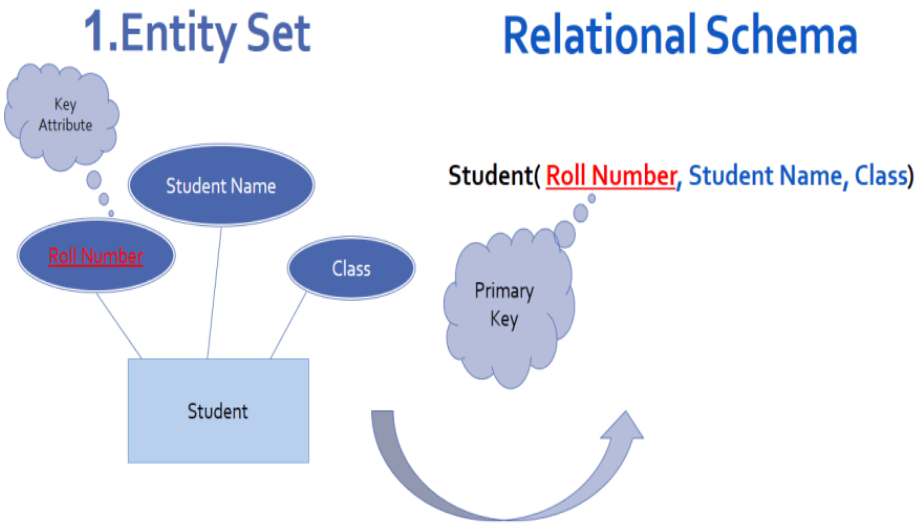


Step by step procedure to convert ER diagram into relational model

1. Entity Set:

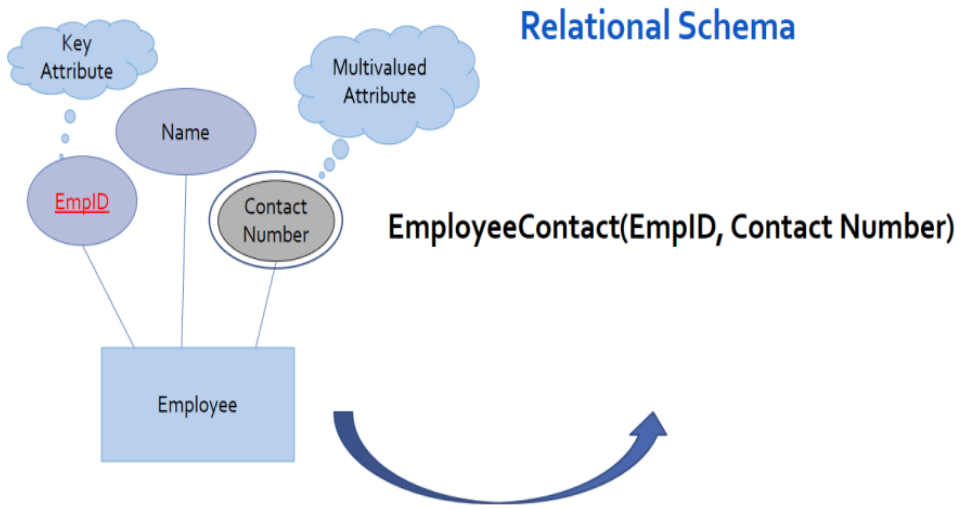
Consider we have entity STUDENT in ER diagram with attributes Roll Number, Student Name and Class.
To convert this entity set into relational schema

1. Entity is mapped as relation in Relational schema.
2. Attributes of Entity set are mapped as attributes for that Relation.
3. Key attribute of Entity becomes Primary key for that Relation.



2. Entity set with multi valued attribute:

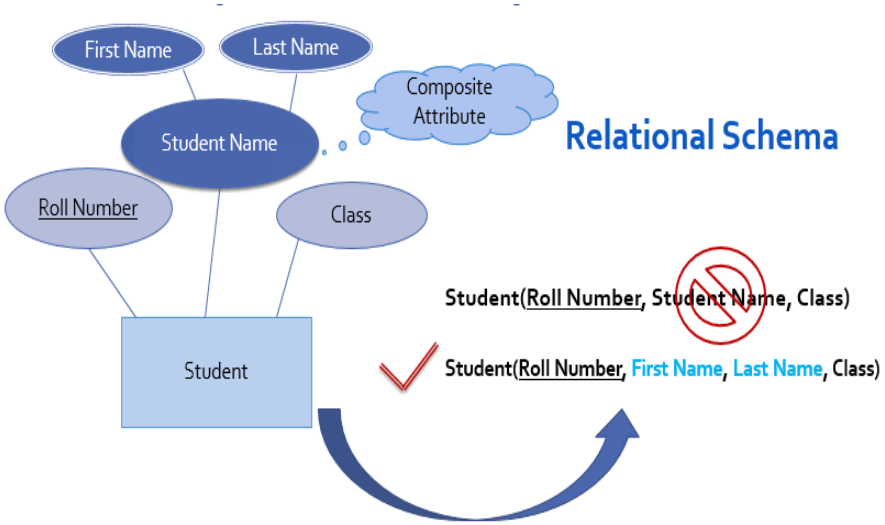
Consider we have entity set Employee with attributes Employee ID, Name and Contact number. Here contact number is multivalued attribute as it has multiple values. as an employee can have more than one contact number for that we have to repeat all attributes for every new contact number. This will lead to data redundancy in table. Hence to convert entity with multivalued attribute into relational schema separate relation is created for multivalued attribute in which -> Key attribute and multivalued attribute of entity set becomes primary key of relation. -> Separate relation employee is created with remaining attributes. Due to this instead of repeating all attributes of entity now only one attribute is need to repeat.



3. Entity set with Composite attribute:

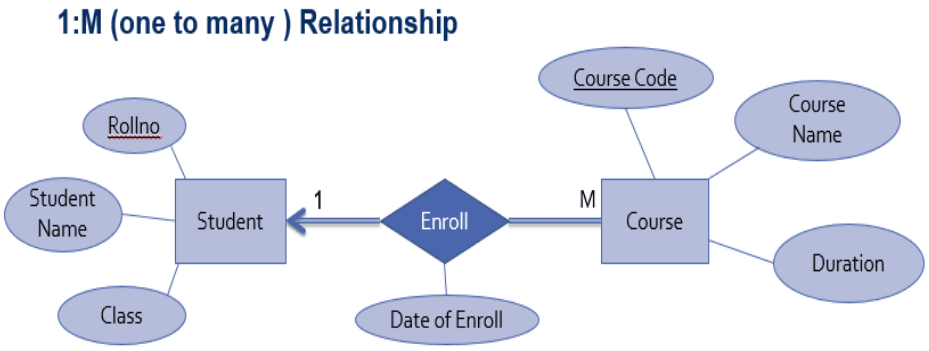
Consider entity set student with attributes Roll Number, Student Name and Class. here student name is composite attribute as it has further divided into First name, last name. In this case to convert entity into relational schema, composite attribute student name should not be including in relation but all

parts of composite attribute are mapped as simple attributes for relation.



4. 1:M (one to many) Relationship:

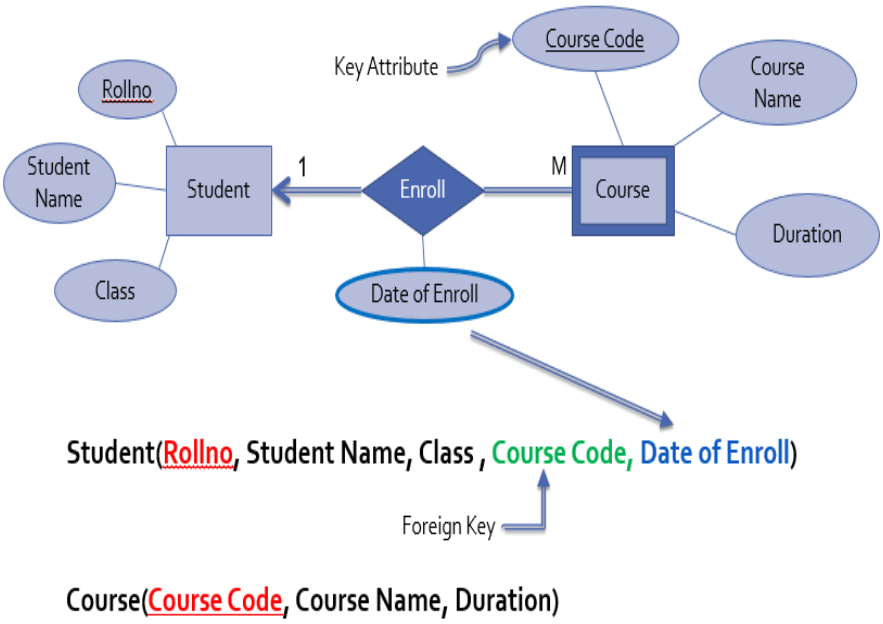
Consider 1:M relationship set enrolled exist between entity sets student and course as follow,



Attributes of entity set student are Roll no which is primary key, student name and class Attributes of entity set course are Course code which is primary key, Course name and duration and date of enroll is attribute of relationship set enroll. Here Enroll is 1:M relationship exist between entity set student and

course which means that one student can enroll in multiple courses
In this case to convert this relationship into relational schema,

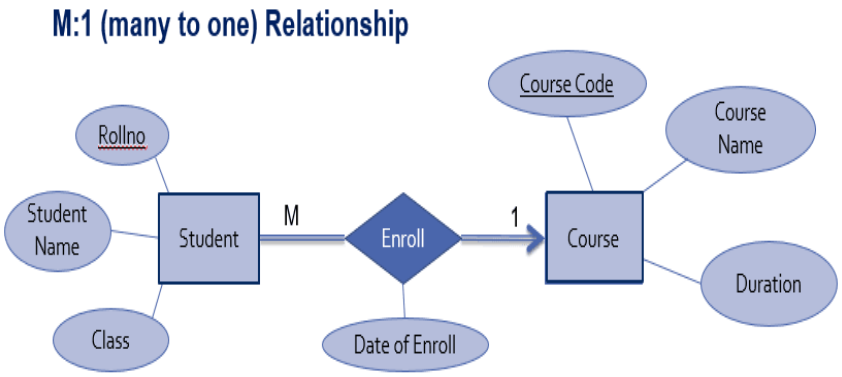
1. Separate relation is created for all participating entity sets (student and course
2. Key attribute of Many's side entity set (course) is mapped as foreign key in one's side relation (Student)
3. All attributes of relationship set are mapped as attributes for relation of one's side entity set (student)



5. M:1 (many to one) Relationship:

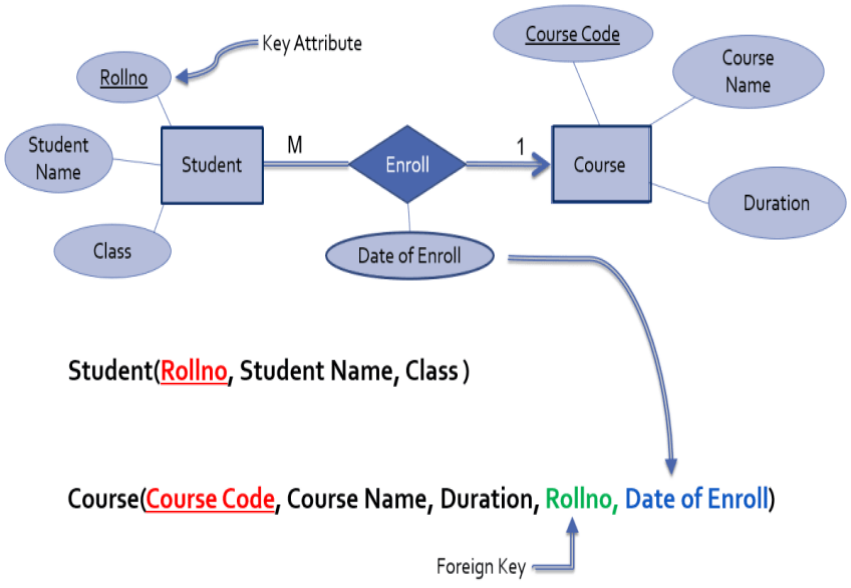
Consider same relationship set enroll exist between entity sets student and course. But here student is many sides entity set while course is one side entity

set. Which means many student can enroll in one course.



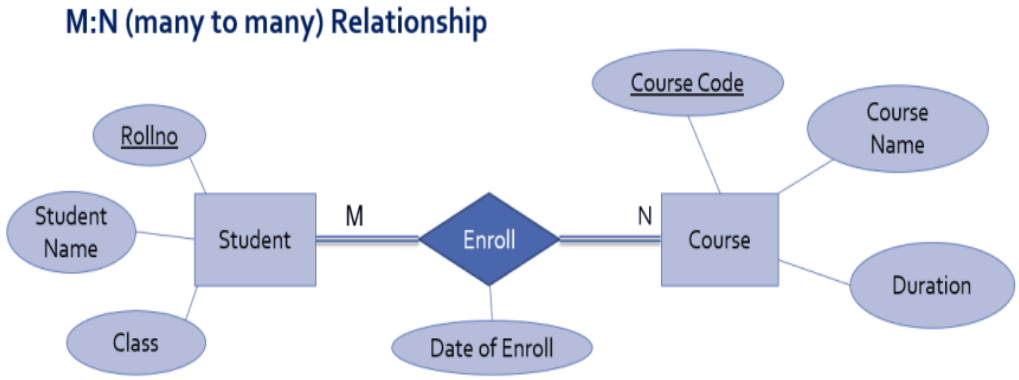
To convert this relationship set into relational schema,

1. Separate relation is created for all participating entity sets.
2. Key attribute of Many's side entity set student is mapped as foreign key in one's side relation
3. All attributes of relationship set are mapped as attributes for one's side relation course.



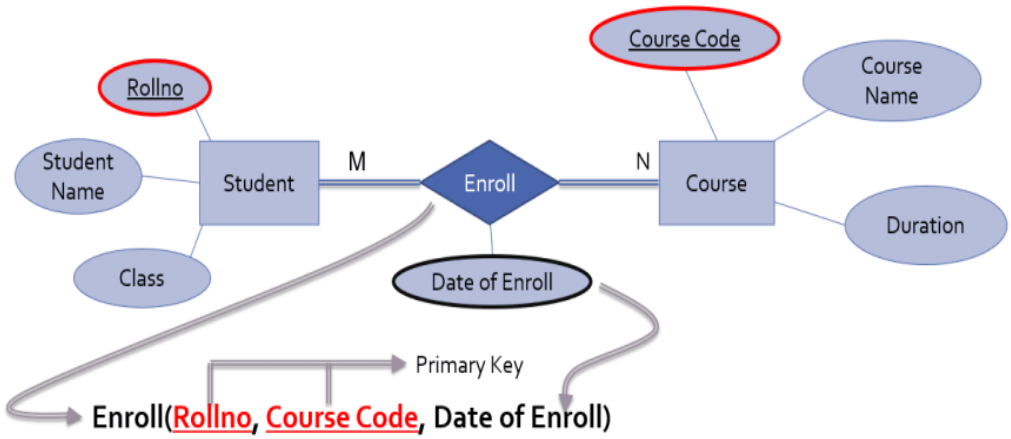
6. M: N (many to many) Relationship:

Consider same relationship set enrolled exist between entity sets student and course, which means multiple students can enroll in multiple courses.



To convert this Relationship set into relational schema, Relationship set is mapped as separate relation Key attributes of participating entity sets are mapped as primary key for that relation Attribute of relationship set becomes simple attributes for that relation

And separate relation is created for other participating entities

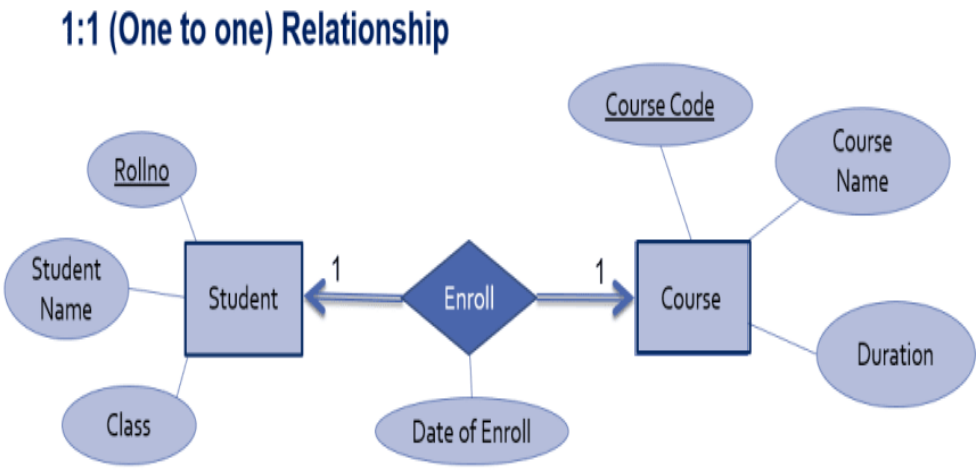


Student(Rollno, Student Name, Class)

Course(Course Code, Course Name, Duration)

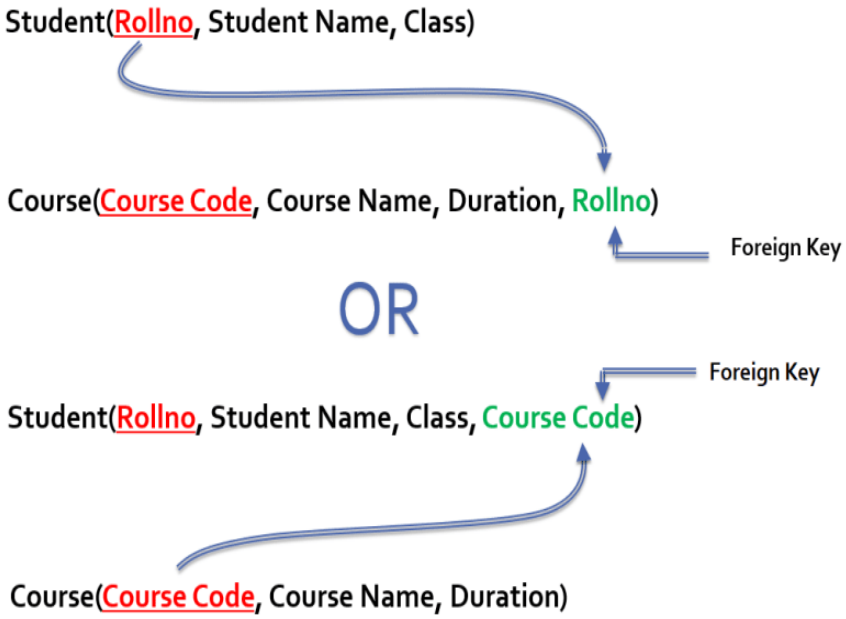
7.(One to one) Relationship:

Consider same relationship set enroll exist between entity sets student and course, which means one student can enroll in only one courses

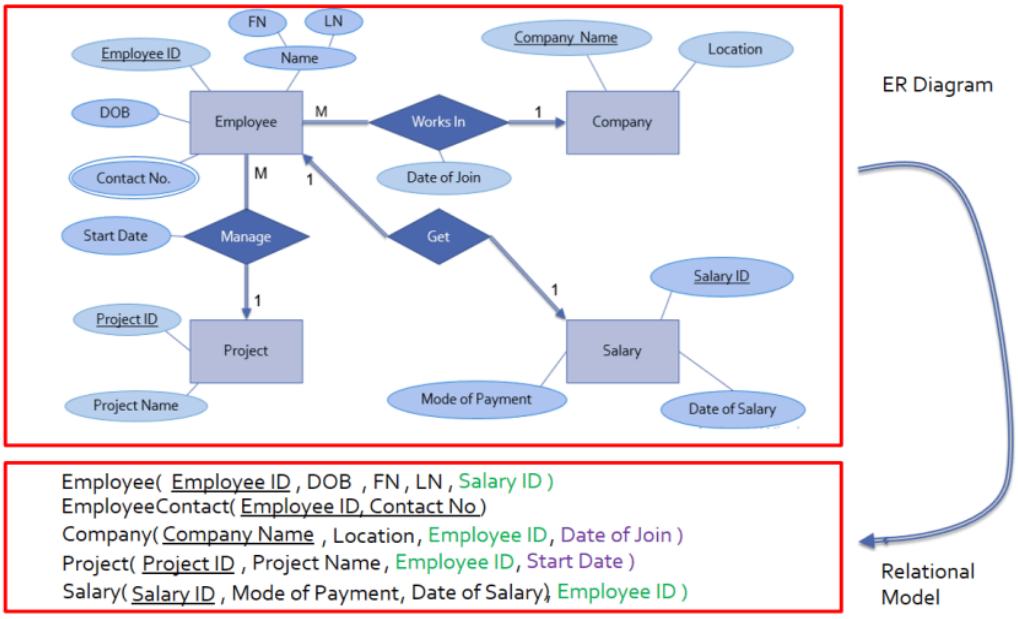


To convert this Relationship set into relational schema, Separate relation is created for all participating entity sets.

Primary Key of Relation Student can be act as foreign key for relation Course OR Primary Key of Relation Course act as foreign key for relation Student.



EXAMPLE:



EXERCISE No. 08

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1. A many to many relationships between two entities usually results in how many tables?
   A. One
   B. Two
   C. Three
   D. Four
2. An _____ is a set of entities of the same type that share the same properties, or attributes.
   A. Entity set
   B. Attribute set
   C. Relation set
   D. Entity model
3. Entity is a _____ .
   A. Object of relation
   B. Present working model
   C. Thing in real world
   D. Model of relation
4. Every weak entity set can be converted into a strong entity set by:
   A. using generalization
   B. adding appropriate attributes
   C. using aggregation
   D. none of the above
5. E-R modeling technique is a _____ .
   A. Top-down approach
   B. Bottom-up approach
   C. Left-right approach
   D. None of the above
6. In ER model the details of the entities are hidden from the user. This process is called :

A. Generalization
B. Specialization
C. Abstraction
D. none of these above

7. The attribute AGE is calculated from DATE_OF_BIRTH. The attribute AGE is
   A. Single valued
   B. Multi valued
   C. Composite
   D. Derived
8. Which of the following can be a multivalued attribute?
   A. Phone_number
   B. Name
   C. Date_of_birth
   D. All of the mentioned
9. Which of the following is a single valued attribute?
   A. Register_number
   B. Address
   C. SUBJECT_TAKEN
   D. Reference
10. The E-R model is expressed in terms of
    A. Entities
    B. Relationships between Entities
    C. Attributes of Entities
    D. All of these
11. The relationship between the entities is represented graphically by using_____.
    A. ER Diagram
    B. Data Flow Diagram
    C. Flowcharts
    D. Decision Tables
12. Which of following represents entity?
    A. Student
    B. Teacher
    C. Train

D. All of these

13. In an ER diagram, entities are represented by _____.
    A. Circles
    B. Rectangles
    C. Diamond shaped box
    D. Ellipse
14. A collection of entities that have common attributes is called
    A. Entity type
    B. Entity Class
    C. Entity Set
    D. Both a and b
15. The characteristics of an entity are called_____.
    A. Entity Set
    B. Entity Class
    C. Entity properties
    D. Entity Attributes
16. In a relational database model, the number of records in a table is called
    A. Modality
    B. Cardinality
    C. Degree of relation
    D. All of these
17. Strong entity is also called
    A. Parent Entity
    B. Subordinate Entity
    C. Dependent Entity
    D. Child Entity
18. The relationship between subtype and supertype is called_____ .
    A. IS-AN relationship
    B. HAS-A relationship
    C. IS-A relationship
    D. Both A and B
19. The process of defining one or more subtype of a supertype is called_____.
    A. Generalization

B. Identifier
C. Specialization
D. Cardinality

20. The entity types Car and Truck can be generalized into the entity type___.
    A. Train
    B. Engine
    C. Vehicle
    D. Bus

ANSWER KEY

1.	C	2.	A	3.	C	4.	B	5.	A
6.	C	7.	D	8.	A	9.	A	10.	A
11.	A	12.	D	13	B	14.	D	15.	D
16	B	17	A	18	D	19	C	20	C

PART-II SAMPLE SHORT QUESTIONS

1. Define ER Modeling.
2. What is Entity?
3. Draw ER Model diagram
4. Explain about history of ER model
5. Why we use ER model?
6. What is difference between Specialization and Generalization
7. Define Sub Class.
8. Define Super Class.
9. What are the main components of ER model?
10. What is Relational model?

11. What is difference between Sub-Class and Super-Class?
12. Define Subtype/Supertype Entity
13. Define Strong Entity
14. Define Weak Entity
15. What is Relationship participation?
16. Define Partial Participation
17. Define composite entity
18. What is Aggregation
19. What is cardinality?
20. What is Association?

PART-III SAMPLE LONG QUESTIONS

Q.1. What is ER Modeling explain with the help of diagram?
Q.2. Explain about Degrees of Data Abstraction in detail.
Q.3. What is EER Model? What are Features of EER MODEL?
Q. 4. What is an entity subtype? What are entity subtypes used for? Give example
Q.5. What is Relational Model? Transform ER/EER to Relational Model.
Q.6. What is Attribute also explain types of Attribute?
Q.7. How to transform ER to Relational model?

9 Chapter No. 9 (Transaction Management)

Objectives
After completion of this chapter students will be able to explain:

- ✓ What is a Transaction?
- ✓ Evaluating Transaction Results
- ✓ Transaction Management with SQL
- ✓ Transaction Log
- ✓ Transaction Types

9.1 What is transaction?

A transaction can be defined as a group of tasks. A single task is the minimum processing unit which cannot be divided further. A transaction, in the context of a database, is a logical unit that is independently executed for data retrieval or updates. Experts talk about a database transaction as a “unit of work” that is achieved within a database design environment. A transaction is a logical unit of work that must be either entirely completed or aborted; no intermediate states are acceptable

Example:

A simple example of a transaction will be dealing with the bank accounts of two users, let say Karlos and Ray. A simple transaction of moving an amount of 5000 from Karlos to Ray engages many low-level jobs. As the amount of Rs. 5000 gets transferred from the Karlos's account to Ray's account, a series of tasks gets performed in the background of the screen. This straightforward and small transaction includes several steps: decrease Karlos's bank account from 5000:

Open_Acc (Karlos)
OldBal = Karlos.bal
NewBal = OldBal - 5000

Ram.bal = NewBal
CloseAccount(Karlos)
You can say, the transaction involves many tasks, such as opening the account of Karlos, reading the old balance, decreasing the specific amount of 5000 from that account, saving new balance to an account of Karlos, and finally closing the transaction session.

For adding amount 5000 in Ray's account, the same sort of tasks needs to be done:

OpenAccount(Ray)
Old_Bal = Ray.bal
NewBal = OldBal + 1000
Ahmed.bal = NewBal
Close Account (B)

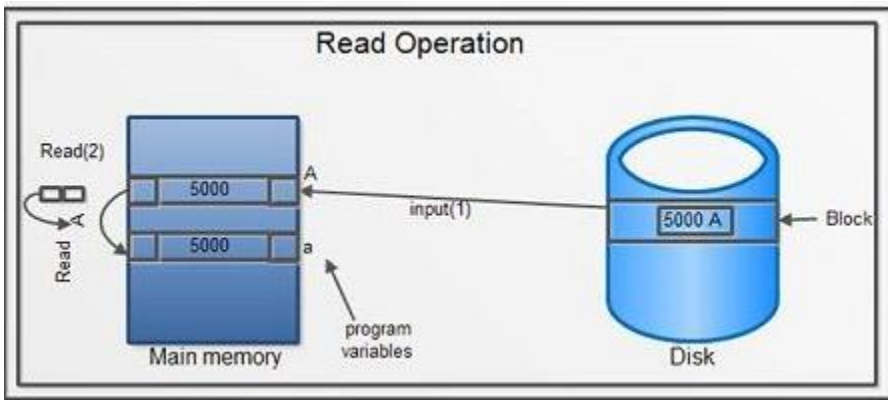
In DBMS any action that reads from and/or writes to a database may consist of
–Simple SELECT statement to generate a list of table contents
–A series of related UPDATE statements to change the values of attributes in various tables
–A series of INSERT statements to add rows to one or more tables
–A combination of SELECT, UPDATE, and INSERT statements

9.1.1 Process of Transaction

The transaction is executed as a series of reads and writes of database objects, which are explained below:

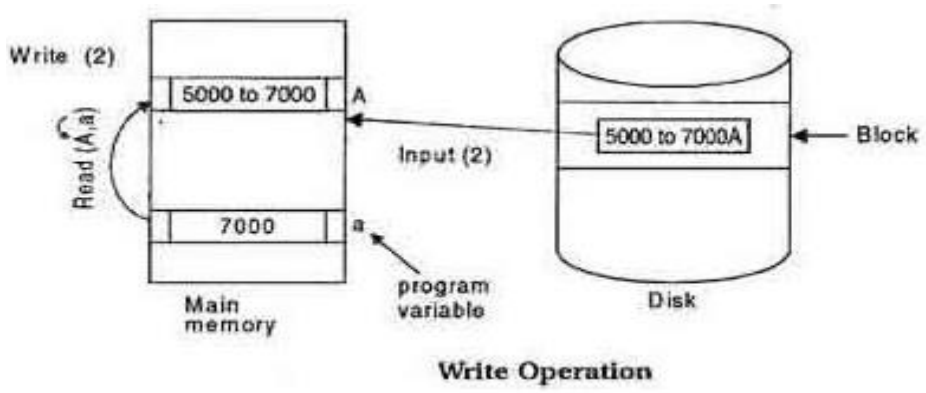
9.1.1.1 Read Operation

To read a database object, it is first brought into main memory from disk, and then its value is copied into a program variable as shown in figure.



9.1.1.2 Write Operation

To write a database object, an in-memory copy of the object is first modified and then written to disk.



Write Operation

9.2 Evaluating Transaction Results

- ✓ Not all transactions update database
- ✓ SQL DML statements represent components of a transaction. Each DML statement requires accessing the database and is considered a database request.
- ✓ Improper or incomplete transactions can have devastating effect on database integrity

- ✓ Some DBMSs provide means by which user can define enforceable constraints (using triggers)
- ✓ Other integrity rules are enforced automatically by the DBMS

9.2.1 Transaction Properties or ACID Properties.

A transaction is a very small unit of a program and it may contain several low-level tasks. A transaction in a database system must maintain Atomicity, Consistency, Isolation, and Durability − commonly known as ACID properties − in order to ensure accuracy, completeness, and data integrity.

There are four important properties of transaction that a DBMS must ensure to maintain data in the case of concurrent access and system failures. These are:

- **Atomicity: (all or nothing)**

A transaction is said to be atomic if a transaction always executes all its actions in one step or not executes any actions at all It means either all or none of the transaction's operations are performed.

- **Consistency: (No violation of integrity constraints)**

A transaction must preserve the *consistency* of a database after the execution. The DBMS assumes that this property holds for each transaction. Ensuring this property of a transaction is the responsibility of the user.

- **Isolation: (concurrent changes invisible)**

The transactions must behave as if they are executed in isolation. It means that if several transactions are executed concurrently the results must be same as if they were executed serially in some order. The data used during the execution of a transaction cannot be used by a second transaction until the first one is completed.

- **Durability: (committed update persist)**

The effect of completed or committed transactions should persist even after a crash. It means once a transaction commits, the system must guarantee that the

result of its operations will never be lost, in spite of subsequent failures. The acronym ACID is sometimes used to refer above four properties of transaction that we have presented here: Atomicity, Consistency, Isolation, and Durability.

9.2.2 States of Transaction

A transaction must be in one of the following states:

- ✓ **Active** – In this state, the transaction is being executed. This is the initial state of every transaction.
- ✓ **Partially Committed** – When a transaction executes its final operation, it is said to be in a partially committed state.
- ✓ **Failed** – A transaction is said to be in a failed state if any of the checks made by the database recovery system fails. A failed transaction can no longer proceed further.
- ✓ **Aborted** – If any of the checks fails and the transaction has reached a failed state, then the recovery manager rolls back all its write operations on the database to bring the database back to its original state where it was prior to the execution of the transaction. Transactions in this state are called aborted. The database recovery module can select one of the two operations after a transaction abort.
  - Re-start the transaction
  - Kill the transaction
- ✓ **Committed** – If a transaction executes all its operations successfully, it is said to be committed. All its effects are now permanently established on the database system.

The state diagram corresponding to a transaction is shown in Figure.

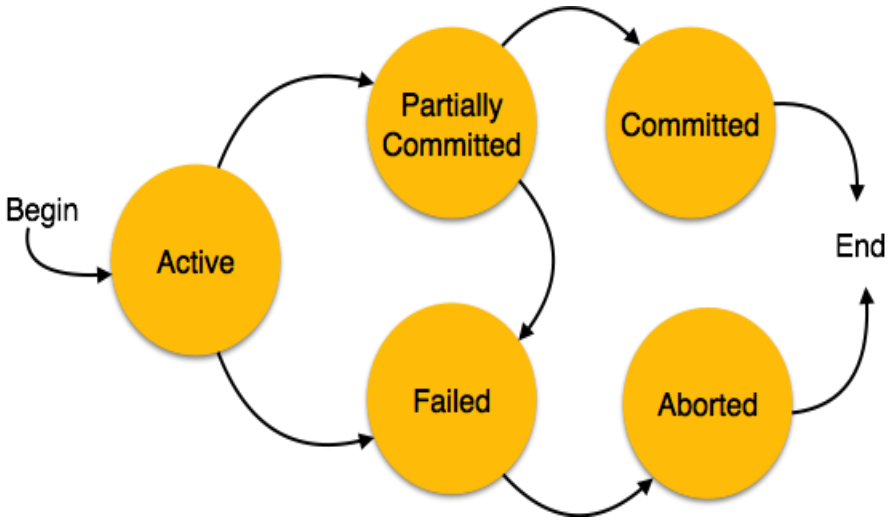


Figure:States of transaction

We can say that a transaction has committed only if it has entered the committed state. Similarly, we say that a transaction has aborted only if it has entered the aborted state. A transaction is said to have terminated if has either committed or aborted.

A transaction starts in the active state. When it finishes its final statement, it enters the partially committed state. At this point, the transaction has completed its execution, but it is still possible that it may have to be aborted, since the actual output may still be temporarily hiding in main memory and thus a hardware failure may preclude its successful completion

The database system then writes out enough information to disk that, even in the event of a failure, the updates performed by the transaction can be recreated when the system restarts after the failure. When the last of this information is written out, the transaction enters the committed state.

9.2.3 Advantages of Execution of Transaction

The DBMS interleaves the actions of different transactions to improve performance of system as discussed below:

1) Improved Throughput:

Consider that transaction are performed in serial order and active transaction is waiting for a page to be read in from disk, then instead of CPU waiting for a page, it can process another transaction. This is because Input/Output activity can be done in parallel with the CPU activity. The overlapping of Input/Output activities of CPU reduces the amount of time disks and processors are idle and increases system throughput (the average number of transaction completed in a given time.)

2) Reduced Waiting time:

Interleaved execution of a short transaction with a long transaction usually allows the short transaction to complete quickly. In serial execution a short transaction could get stuck behind a long transaction leading to unpredictable delays in response time or average time taken to complete a transaction.

9.3 Transaction Management with SQL

ANSI has defined standards that govern SQL database transactions

- ✓ Transaction support is provided by two SQL statements: COMMIT and ROLLBACK
- ✓ ANSI standards require that, when a transaction sequence is initiated by a user or an application program,
- ✓ it must continue through all succeeding SQL statements until one of four events occurs

If statements are executed successfully then the transaction is complete and then it is committed that saves the data in the database permanently. If any single statement fails then the entire transaction will fail and the complete transaction will be cancelled or rolled back. When a transaction starts, it locks all the table data that is used in the transaction. Hence during the transaction life cycle no one can modify this table data used by the transaction such that the integrity of the data for the transaction is maintained.

A transaction is used when more than one table or view related to each other at a time are affected. The main goal of a transaction is for either all operations will be done or nothing will be done. We can compare a transaction with a digital circuit that works on 0 and 1. Here:

1 indicates completeness of all tasks (T-SQL statements)

0 indicates no single tasks performed (T-SQL statements)

Example

A transaction is mainly used in banking or the transaction sector. Let us see an example of a bank that has two customers, Cust_A and Cust_B. In case Cust_A wants to transfer some money to Cust_B, then there are the following 3 possibilities:

Debiting from the Cust_A account is performed successfully and crediting in the Cust_B account is performed successfully. Neither debiting from the Cust_A account is performed nor is crediting in the Cust_B account performed. Debiting from the Cust_A account is performed successfully, but crediting in the Cust_B account is not performed.

The first condition indicates a successful transaction and the second condition is not so critical. We are not required to do any retransmission, but the third condition will create a problem if, due to a technical problem, the first operation is successful but the second one fails. The result here would be that the Cust_A account will be debited, but the Cust_B account will not be credited. This means that we will lose the information.

9.4 Transaction Log

Every SQL Server database has a transaction log that records all transactions and the database modifications made by each transaction. The transaction log is a

critical component of the database. If there is a system failure, you will need that log to bring your database back to a consistent state.

Warning

Never delete or move this log unless you fully understand the ramifications of doing so.

Tip

Known good points from which to begin applying transaction logs during database recovery are created by checkpoints.

9.4.1 The transaction log supports the following operations:

- ✓ Individual transaction recovery.
- ✓ Recovery of all incomplete transactions when SQL Server is started.
- ✓ Rolling a restored database, file, file group, or page forward to the point of failure.
- ✓ Supporting transactional replication.
- ✓ Supporting high availability and disaster recovery solutions: Always On availability groups, database mirroring, and log shipping.
- ✓ Individual transaction recovery

If an application issue a ROLLBACK statement, or if the Database Engine detects an error such as the loss of communication with a client, the log records are used to roll back the modifications made by an incomplete transaction.

9.4.2 Recovery of all incomplete transactions when SQL Server is started

If a server fails, the databases may be left in a state where some modifications were never written from the buffer cache to the data files, and there may be some modifications from incomplete transactions in the data files. When an instance of SQL Server is started, it runs a recovery of each database. Every modification

recorded in the log that may not have been written to the data files is rolled forward. Every incomplete transaction found in the transaction log is then rolled back to make sure the integrity of the database is preserved.

9.4.3 Rolling a restored database, file, file group, or page forward to the point of failure

After a hardware loss or disk failure affecting the database files, you can restore the database to the point of failure. You first restore the last full database backup and the last differential database backup, and then restore the subsequent sequence of the transaction log backups to the point of failure.

As you restore each log backup, the Database Engine reapplies all the modifications recorded in the log to roll forward all the transactions. When the last log backup is restored, the Database Engine then uses the log information to roll back all transactions that were not complete at that point.

9.4.4 Supporting transactional replication

The Log Reader Agent monitors the transaction log of each database configured for transactional replication and copies the transactions marked for replication from the transaction log into the distribution database.

9.4.5 Supporting high availability and disaster recovery solutions

The standby-server solutions, Always On availability groups, database mirroring, and log shipping, rely heavily on the transaction log.

In an **Always On availability group scenario**, every update to a database, the primary replica, is immediately reproduced in separate, full copies of the database, the secondary replicas. The primary replica sends each log record immediately to the secondary replicas, that applies the incoming log records to availability group databases, continually rolling it forward

- ✓ **In a log shipping scenario**, the primary server sends the active transaction log of the primary database to one or more destinations. Each secondary server restores the log to its local secondary database.

- ✓ **In a database mirroring scenario**, every update to a database, the principal database, is immediately reproduced in a separate, full copy of the database, the mirror database. The principal server instance sends each log record immediately to the mirror server instance, which applies the incoming log records to the mirror database, continually rolling it forward.

9.4.6 Transaction log characteristics

Characteristics of the SQL Server Database Engine transaction log:

✓ The transaction log is implemented as a separate file or set of files in the database. The log cache is managed separately from the buffer cache for data pages, which results in simple, fast, and robust code within the SQL Server Database Engine

✓ The format of log records and pages is not constrained to follow the format of data pages.

✓ The transaction log can be implemented in several files. The files can be defined to expand automatically by setting the FILEGROWTH value for the log. This reduces the potential of running out of space in the transaction log, while at the same time reducing administrative overhead

✓ The mechanism to reuse the space within the log files is quick and has minimal effect on transaction throughput.

9.4.7 Transaction log truncation

Log truncation frees space in the log file for reuse by the transaction log. You must regularly truncate your transaction log to keep it from filling the allotted space. Several factors can delay log truncation, so monitoring log size matters. Some operations can be minimally logged to reduce their impact on transaction log size.

Log truncation deletes inactive virtual log files (VLFs) from the logical transaction log of a SQL Server database, freeing space in the logical log for reuse by the Physical transaction log. If a transaction log is never truncated, it will eventually fill all the disk space allocated to physical log files.

To avoid running out of space, unless log truncation is delayed for some reason, truncation occurs automatically after the following events:

Under the simple recovery model, after a checkpoint.

Under the full recovery model or bulk-logged recovery model, if a checkpoint has occurred since the previous backup, truncation occurs after a log backup (unless it is a copy-only log backup).

9.4.8 What is Concurrency Control?

Concurrency Control: in Database Management System is a procedure of managing simultaneous operations without conflicting with each other. It ensures that Database transactions are performed concurrently and accurately to produce correct results without violating data integrity of the respective Database.

Concurrent access is quite easy if all users are just reading data. There is no way they can interfere with one another. Though for any practical Database, it would have a mix of READ and WRITE operations and hence the concurrency is a challenge.

DBMS Concurrency Control is used to address such conflicts, which mostly occur with a multi-user system. Therefore, Concurrency Control is the most important element for proper functioning of a Database Management System where two or more database transactions are executed simultaneously, which require access to the same data.

Potential problems of Concurrency

Here, are some issues which you will likely to face while using the DBMS Concurrency Control method:

1) **Lost Updates** occur when multiple transactions select the same row and update the row based on the value selected Uncommitted dependency issues occur when the second transaction selects a row which is updated by another transaction (**dirty read**)

2) **Non-Repeatable Read** occurs when a second transaction is trying to access the same row several times and reads different data each time.

3) **Incorrect Summary issue** occurs when one transaction takes summary over the value of all the instances of a repeated data-item, and second transaction update few instances of that specific data-item. In that situation, the resulting summary does not reflect a correct result.

9.4.9 Why use Concurrency method?

Reasons for using Concurrency control method is DBMS:

1) To apply Isolation through mutual exclusion between conflicting transactions.
2) To resolve read-write and write-write conflict issues.
3) To preserve database consistency through constantly preserving execution obstructions.
4) The system needs to control the interaction among the concurrent transactions. This control is achieved using concurrent-control schemes.
5) Concurrency control helps to ensure serializability.

Example

Assume that two people who go to electronic kiosks at the same time to buy a movie ticket for the same movie and the same show time.

However, there is only one seat left in for the movie show in that particular theatre. Without concurrency control in DBMS, it is possible that both moviegoers will end up purchasing a ticket. However, concurrency control method does not allow this to happen. Both moviegoers can still access information written in the movie seating database. But concurrency control only provides a ticket to the buyer who has completed the transaction process first.

9.4.10 Concurrency Control Protocols

Different concurrency control protocols offer different benefits between the amount of concurrency they allow and the amount of overhead that they impose. Following are the Concurrency Control techniques in DBMS:

1) Lock-Based Protocols
2) Two Phase Locking Protocol
3) Timestamp-Based Protocols
4) Validation-Based Protocols

1) Lock Based Protocols:

In DBMS is a mechanism in which a transaction cannot Read or Write the data until it acquires an appropriate lock. Lock based protocols help to eliminate the concurrency problem in DBMS for simultaneous transactions by locking or isolating a particular transaction to a single user.

A lock is a data variable which is associated with a data item. This lock signifies those operations that can be performed on the data item. Locks in DBMS help synchronize access to the database items by concurrent transactions.

All lock requests are made to the concurrency-control manager. Transactions proceed only once the lock request is granted.

- ✓ **Binary Locks:** A Binary lock on a data item can either locked or unlocked states.
- ✓ **Shared/exclusive:** This type of locking mechanism separates the locks in DBMS based on their uses. If a lock is acquired on a data item to perform a writeoperation, it is called an exclusive lock.
- ✓ **Shared Lock (S):**

A shared lock is also called a Read-only lock. With the shared lock, the data item can be shared between transactions. This is because you will never have permission to update data on the data item.

For example, consider a case where two transactions are reading the account balance of a person. The database will let them read by placing a shared lock.

However, if another transaction wants to update that account's balance, shared lock prevents it until the reading process is over.

✓ **Exclusive Lock (X):**

With the Exclusive Lock, a data item can be read as well as written. This is exclusive and can't be held concurrently on the same data item. X-lock is requested using lock-x instruction. Transactions may unlock the data item after finishing the 'write' operation.

For example, when a transaction needs to update the account balance of a person. You can allow this transaction by placing X lock on it. Therefore, when the second transaction wants to read or write, exclusive lock prevents this operation.

✓ **Simplistic Lock Protocol:**

This type of lock-based protocols allows transactions to obtain a lock on every object before beginning operation. Transactions may unlock the data item after finishing the 'write' operation.

✓ **Pre-claiming Locking**

Pre-claiming lock protocol helps to evaluate operations and create a list of required data items which are needed to initiate an execution process. In the situation when all locks are granted, the transaction executes. After that, all locks release when all of its operations are over.

✓ **Starvation**

Starvation is the situation when a transaction needs to wait for an indefinite period to acquire a lock.

Following are the reasons for Starvation:

When waiting scheme for locked items is not properly managed

In the case of resource leak the same transaction is selected as a victim repeatedly

2) Deadlock

Deadlock refers to a specific situation where two or more processes are waiting for each other to release a resource or more than two processes are waiting for the resource in a circular chain.

Two Phase Locking Protocol

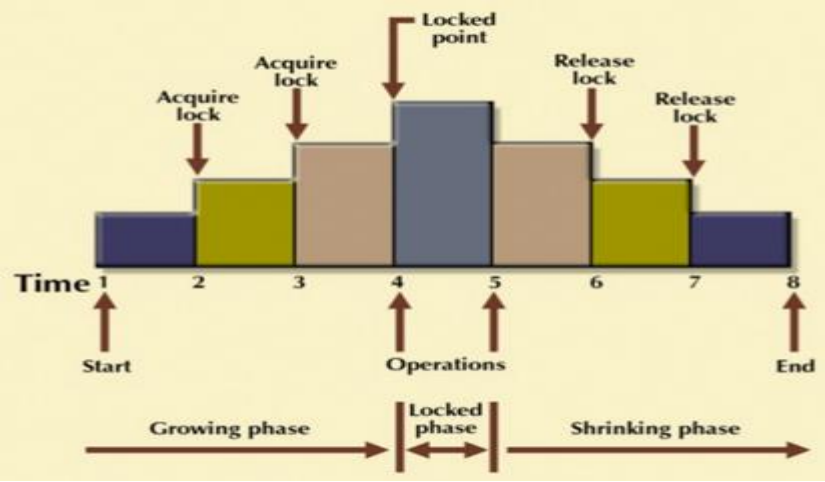
Also known as 2PL protocol is a method of concurrency control in DBMS that ensures serializability by applying a lock to the transaction data which blocks other transactions to access the same data simultaneously. Two Phase Locking protocol helps to eliminate the concurrency problem in DBMS.

This locking protocol divides the execution phase of a transaction into three different parts.

In the first phase, when the transaction begins to execute, it requires permission for the locks it needs.

The second part is where the transaction obtains all the locks. When a transaction release its first lock, the third phase starts.

In this third phase, the transaction cannot demand any new locks. Instead, it only releases the acquired locks.



The Two-Phase Locking protocol allows each transaction to make a lock or unlock request in two steps:

✓ **Growing Phase**: In this phase transaction may obtain locks but may not release any locks.

✓ **Shrinking Phase**: In this phase, a transaction may release locks but not obtain any new lock

It is true that the 2PL protocol offers serializability. However, it does not ensure that deadlocks do not happen.

In the above-given diagram, you can see that local and global deadlock detectors are searching for deadlocks and solve them with resuming transactions to their initial states.

9.5 Strict Two-Phase Locking Method

Strict-Two phase locking system is almost similar to 2PL. The only difference is that Strict-2PL never releases a lock after using it. It holds all the locks until the commit point and releases all the locks at one go when the process is over.

9.5.1 Centralized 2PL

In Centralized 2 PL, a single site is responsible for lock management process. It has only one lock manager for the entire DBMS.

9.5.2 Primary copy 2PL

Primary copy 2PL mechanism, many lock managers are distributed to different sites. After that, a particular lock manager is responsible for managing the lock for a set of data items. When the primary copy has been updated, the change is propagated to the slaves.

9.5.3 Distributed 2PL

In this kind of two-phase locking mechanism, Lock managers are distributed to all sites. They are responsible for managing locks for data at that site. If no data is replicated, it is equivalent to primary copy 2PL. Communication costs of Distributed 2PL are quite higher than primary copy 2PL.

9.5.4 Timestamp based Protocol

In DBMS is an algorithm which uses the System Time or Logical Counter as a timestamp to serialize the execution of concurrent transactions. The Timestamp-based protocol ensures that every conflicting read and write operations are executed in a timestamp order.

The older transaction is always given priority in this method. It uses system time to determine the time stamp of the transaction. This is the most commonly used concurrency protocol.

Lock-based protocols help you to manage the order between the conflicting transactions when they will execute. Timestamp-based protocols manage conflicts as soon as an operation is created.

Advantages:

Schedules are serializable just like 2PL protocols

No waiting for the transaction, which eliminates the possibility of deadlocks!

Disadvantages:

Starvation is possible if the same transaction is restarted and continually aborted

Validation Based Protocol

9.5.5 Validation based Protocol:

In DBMS also known as Optimistic Concurrency Control Technique is a method to avoid concurrency in transactions. In this protocol, the local copies of the transaction data are updated rather than the data itself, which results in less interference while execution of the transaction?

The Validation based Protocol is performed in the following three phases:

1. Read Phase
2. Validation Phase
3. Write Phase

1. Read Phase:

In the Read Phase, the data values from the database can be read by a transaction but the write operation or updates are only applied to the local data copies, not the actual database.

2. Validation Phase:

In Validation Phase, the data is checked to ensure that there is no violation of serializability while applying the transaction updates to the database.

3. Write Phase:

In the Write Phase, the updates are applied to the database if the validation is successful, else; the updates are not applied, and the transaction is rolled back.

Characteristics of Good Concurrency Protocol. An ideal concurrency control DBMS mechanism has the following objectives: Must be resilient to site and communication failures.

It allows the parallel execution of transactions to achieve maximum concurrency. Its storage mechanisms and computational methods should be modest to minimize overhead. It must enforce some constraints on the structure of atomic actions of transactions.

Lock Granularity

1) Indicates the level of lock use
2) Locking can take place at the following levels:
    - Database
    - Table
    - Page
    - Row
    - Field (attribute)

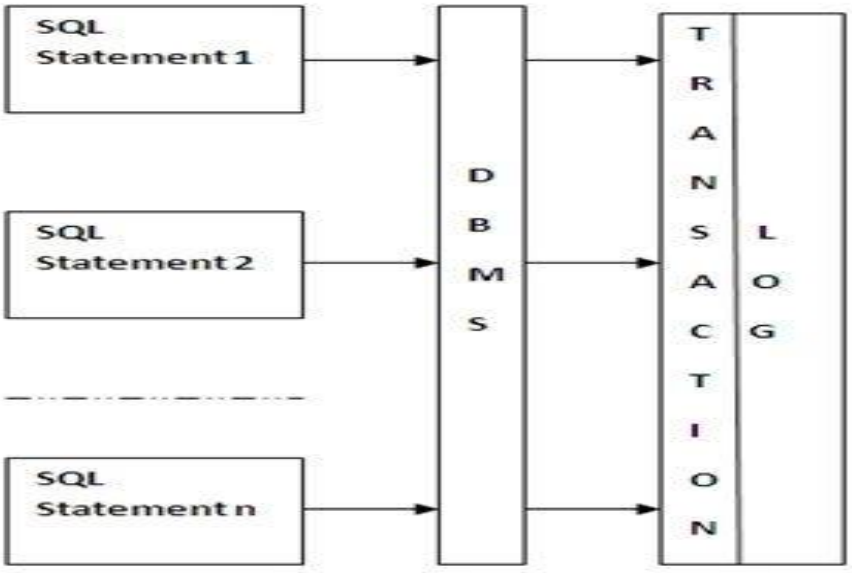
9.6 Types of Transactions:

In SQL, transactions are of the following two types:

1. Implicit Transections
2. Explicit Transections

9.6.1 Implicit Transactions

Implicit transactions in the SQL language are performed by a DML query (insert, update and delete) and DDL query (alter, drop, truncate and create) statements. All these queries are handled by Implicit Transactions.



When any DDL or DML query is performed then the system stores the information of all the operations in the log file. If any error occurs then the SQL Server will roll back the complete statement.

9.6.2 Explicit Transactions

An explicit transaction is defined and controlled by the user on a DML query (insert, update or delete). A transaction is not applied on a SELECT command because it doesn't affect the data. A transaction is not used in creating tables or dropping them because these operations are automatically committed in the database.

- **Transaction Control**

  Following commands are used in the transaction control mechanism.

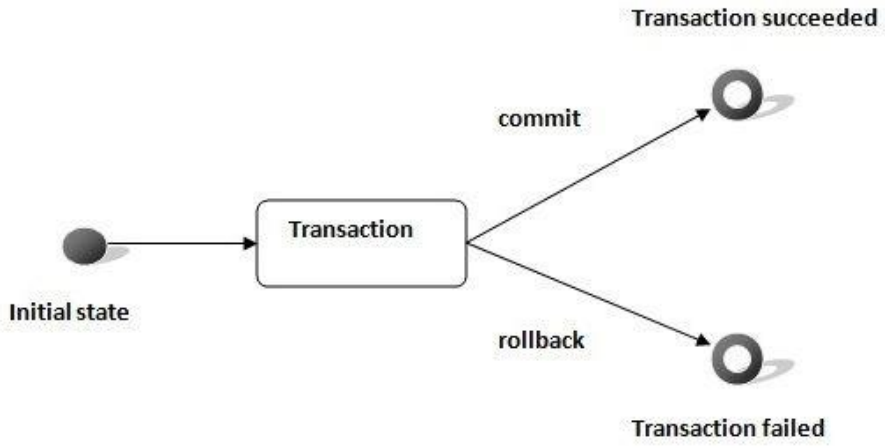


Figure: Transaction Control

- ✓ **BEGIN:** To initiate a transaction.
- ✓ **COMMIT:** To save changes. After the commit command, the transaction can't rollback.
- ✓ **SAVEPOINT:** Provides points where the transaction can rollback to.
- ✓ **ROLLBACK**: To roll back to a previous saved state.

Syntax of Transaction

Begin {Transaction| Tran }[Transaction_Name |@Trans Name]

Write Code Here
End
Here

- ✓ **Begin:** Initiate transaction.
- ✓ **Transaction| Tran:** We can use any one out of both.
- ✓ **Transaction Name:** Used for providing a name for a transaction.
- ✓ **@Trans_Name:** This is the name of a user-defined variable containing a valid transaction name.
- ✓ **End:** Indicates the end of the transaction.

EXERCISE No. 09

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1) Durability can be ensured in _____ ways.

A: 2 WAYS B: 3 ways

C: 4 ways D: 5 way

2) The state in which the transaction stays while it is executing is termed as

A: Active B: Partially committed

C: Initial D: BOTH A & C

3) Performing concurrent execution of transactions reduces

A: Waiting Time B: Buffer time

C: Queue time D: Evaluation time

4) When a transaction is rolled back and entered at the aborted stage, the available system's options are of

A: 2 Types B: 3 types

C: 4 types D: 5 types

5) The usage of concurrent execution of transactions improves

A: Utilization B: Response time

C: Throughput D: BOTH A & C

6) Atomicity is ensured by a component of the database called the

A: Evaluation manager B: Control manager

C: RECOVERY MANAGER D: Quality manager

7) A transaction is said to be a unit of program's

A: Evaluation B: Execution

C: Computation D: Controlling

8) A single transaction failure may result into a set of transaction rollbacks, is known to be

A: Iterated rollback B: Cascade less rollback

C: Cascading Rollback D: Serial rollback

9) Transactions access data using operations of

A: 2 TYPES B: 3 types

C: 4 types D: 5 types

10) Serializability of schedules can be ensured through a mechanism called

A: Concurrency Control Policy

B: Evaluation control policy

C: Execution control policy.

D: Cascading control policy

11) Schedules should preferably be

A: Cascade less B: Cascade

C: Dependent D: Non-modifying

12) Ensuring durability is the responsibility of

A: Quality manager B: Control manager

C: Evaluation manager D: recovery manager

13) The effects of a committed transaction can be undo by executing a

A: Compound transaction

B: Composite transaction

C: Compensating Transaction

D: Common transaction

14) Examples of nonvolatile storage are magnetic disks and

A: Flash Memory B: Main memory

C: Cache memory D: Primary memory

15) A schedule can be tested against the conflict serializability by constructing

A: Histogram B: Gantt chart

C: Precedency Graph D: Bar-graph

16) When the changes made by the aborted transaction has been undone, the transaction is said to be

A: Committed B: Derived

C: Rolled Back D: Depicted

17) The term stating either all operations of the transaction to be displayed at the database, or none at all is known to be

A: Atomicity B: Inconsistency

C: Isolation D: Durability

18) Stable storage that can be accessed online is approximated with

A: Drives B: Flashes

C: Mirrored disks D: Floppies

19) A transaction must be in one of the states from

A: 3 states B: 4 states

C: 5 states D: 6 states

20) By finding a linear order consistent with the partial order in the precedence graph, we can obtain the

A: Topological order

B: Serializability order

C: Precedence order

D: Conflict serializability order

ANSWER KEY

1.	A	2.	D	3.	A	4.	A	5.	D
6.	C	7.	B	8.	C	9.	A	10.	A
11.	A	12.	D	13	C	14.	A	15.	C
16	C	17	A	18	C	19	C	20	B

PART-II SAMPLE SHORT QUESTIONS

1. Define transaction.
2. Define consistency.
3. Name properties of transaction.
4. Name any 3 States of transaction.
5. Define transaction log.
6. Define atomicity.

7. Define durability with example.
8. What are series of process of transaction?
9. Define isolation.
10. Define individual transaction recovery.
11. What is Difference between shared lock & Exclusive lock?
12. Write any 2 characteristics of good currency protocol.
13. What is validation phase?
14. Write Name of two-phase locking protocol.
15. What is dead lock with example?

PART-III SAMPLE LONG QUESTIONS

1. Briefly explain transaction with example.
2. Explain Properties of transaction.
3. What are advantages of execution of transaction?
4. Explain 3 phases of validation-based protocol.
5. Explain each States of transaction.
6. What is Difference between currency method & currency control?
7. Briefly explain transaction log characteristics.